

分析 Stage2 结果

会话 ID: 019f3b5f-5533-7760-8231-bfe776cc71c3 开始时间: 2026-07-07 08:59:26 消息数: 256

用户 2026-07-07 08:59:27

Files mentioned by the user:

```
## [INFO] Stage2 ----- seed: 130 [INFO] 随机种子已设置: 130 [INFO] Stage2 ----- d...:
C:\Users\XiaoyanXiong\.codex/attachments/81915e1d-05cc-4c47-aa29-43aa5f4ff4d2/pasted-text.txt
```

My request for Codex:

```
!python /content/drive/MyDrive/s2/run_stage2.py \
--stage1_dir /content/drive/MyDrive/s1/0704/0704C_ar002_et12_20260511_001 \
--config
/content/drive/MyDrive/s2/0706ar002_et12_20260511_001/timeOnly_mlp/stage2_config_used.yaml \
--out_dir /content/drive/MyDrive/s2/0706ar002_et12_20260511_001/timeOnly_mlp \
--context_method time_only \
--context_policy online \
--window_size 128 \
--seed 130 \
--epochs 500 \
--batch_size 128
```

结果。

结合我s2下面的代码。详细解析我的结果。

助手 2026-07-07 08:59:37

我先把你贴的运行日志和本地 `s2` 代码一起读一遍，重点看配置加载、数据构造、训练/验证指标和最终保存逻辑，这样解析结果时能对上代码里的真实含义。

助手 2026-07-07 09:00:11

日志文件能读到，大小不大；`s2` 里也有完整的 `run\_stage2.py` 和 `stage2/` 模块。我接下来会把训练入口、数据集窗口、`time\_only` 上下文、模型和评估指标这几块对应起来。

助手 2026-07-07 09:00:28

初步读到一个很关键的点：虽然命令叫 `timeOnly\_mlp`，日志里实际构建的是 `model\_type = no\_context\_mlp`。也就是说这次结果主要是在评估 Stage1 每条 flow 自身 embedding 的线性/浅层分类能力，不是在使用 Transformer/LSTM 聚合 128 条上下文。

助手 2026-07-07 09:00:40

代码里 `ContextIndexBuilder` 的确生成了 128 长度的 time-only 序列；但是 `Stage2NoContextMLP.forward()` 只取最后一个有效 token，也就是当前 flow 本身。所以这次上下文诊断很健康，但模型没有真正消费历史 flow。

助手 2026-07-07 09:00:53

训练部分也对上了：每个 epoch 都先用 0.5 阈值拿概率，再在验证集上搜索最优阈值；保存 best model 用的是 `metric\_for\_best=pr\_auc`，不是单纯的 F1。最后测试集使用“验证集选出的阈值”，这点是合理的。

助手 2026-07-07 09:01:06

我还要核一下这次日志对应的配置值。日志里的 `sampler\_pos\_fraction=0.080` 和 `alpha=[1.0, 1.05]` 看起来和本地 `s2/0706/Atimeonly\_v5/stage2\_config\_used.yaml` 很像。

助手 2026-07-07 09:01:30

小细节：你命令里 `--epochs 500` 会覆盖 YAML 里的 epochs，所以即使某个本地 config 写的是 80，实际保存到本次 out_dir 的 `stage2\_config\_used.yaml` 应该是 500。日志里 early stopping 在第 43 轮，所以没有真的跑满 500。

助手 2026-07-07 09:03:00

结论

这次结果很强，但要先划重点：日志显示实际跑的是 `no\_context\_mlp`，不是 Transformer/LSTM 上下文模型。也就是说，虽然你构造了 `time\_only + window\_size=128` 的上下文序列，但模型最后只取“当前 flow 自己的 Stage1 embedding”做分类。因此这次更像是一个 **Stage1 embedding + 浅层分类头** 的强 baseline。

实际模型

日志里打印的是：

```
model_type = no_context_mlp
Stage2NoContextMLP(
  input_proj: Identity
  cls_head: LayerNorm(128) -> Dropout(0.25) -> Linear(128,2)
)
```

结合 [model.py](/C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/model.py:211), `Stage2NoContextMLP.forward()` 只做：

```
current_z = last_valid_token(context_z, mask)
return self.cls_head(current_z)
```

所以它忽略历史上下文，只用当前 flow 的 128 维 `z`。`cls\_head=1` 在你的代码里也不是深 MLP，而是 `LayerNorm + Dropout + Linear`，参数量非常小。这说明：**Stage1 产生的 embedding 本身已经非常可分**。

数据与上下文

总 flow 数：

```
train: 295329
val:   42189
test:  84380
total: 421898
z shape: (421898, 128)
```

类别分布：

```
train positive: 11330 / 295329 = 3.84%
val positive:   1794 / 42189   = 4.25%
test positive:  3520 / 84380   = 4.17%
```

这是明显类别不平衡任务，所以 accuracy / weighted F1 会偏乐观，应该重点看 `Class\_1`、`PR\_AUC`、混淆矩阵。

上下文方面, `time\_only + online + include\_target=True + window\_size=128` 表示每个样本最多包含前 127 条时间上更早的 flow, 加上当前 flow。代码在 [context.py](/C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/context.py:107) 先取历史, 再在 [context.py](/C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/context.py:134) append 当前 flow, 所以没有 future leakage。

不过因为这次模型是 `no\_context\_mlp`, 这些历史 flow 实际没有参与预测。`context\_policy=online` 对这次模型结果基本不产生影响。

****训练过程****

训练非常快收敛:

```
epoch 1: val_pr_auc=0.0225, val_f1_label1=0.0816
epoch 2: val_pr_auc=0.7550, val_f1_label1=0.7282
epoch 5: val_pr_auc=0.8977, val_f1_label1=0.8192
epoch 28: val_pr_auc=0.9024, val_f1_label1=0.8219 <- best
early stop at epoch 43
```

虽然命令设了 `--epochs 500`, 但 early stopping 在第 43 轮停止, best epoch 是 28。保存 best model 用的是 `metric\_for\_best=pr\_auc`, 阈值选择用的是验证集 `f1\_label1`。这个设计是合理的: PR-AUC 选排序能力, F1 阈值在 val 上定。

训练设置也比较保守:

```
WeightedRandomSampler target_pos_frac = 0.08
manual alpha = [1.0, 1.05]
FocalLoss gamma = 1.0
```

正类原始比例约 3.84%, 采样到 8%, 只是轻度提升正类, 不是激进过采样。

****测试集结果****

官方测试结果使用验证集选出的阈值:

```
threshold = 0.603939
test samples = 84380
positive = 3520
```

测试集混淆矩阵:

	pred 0	pred 1
true 0	79962	898
true 1	605	2915

也就是:

```
TP = 2915      /
FN = 605
FP = 898
TN = 79962
```

核心指标:

```
Accuracy:      0.9822
Macro F1:      0.8929
Class 1 F1:    0.7950
Class 1 Recall:0.8281
Class 1 Precision: 0.7645
AUC:           0.9891
PR_AUC:        0.8755
```

解释一下:

- `Recall\_class1 = 0.8281`: 3520 个正类里检出了 2915 个, 漏掉 605 个。
- `Precision\_class1 = 0.7645`: 模型报为正类的 3813 个里, 2915 个是真的正类。

- `FPR = 898 / 80860 = 1.11%`, 误报率不高。
- `FNR = 605 / 3520 = 17.19%`, 漏报率还有下降空间。
- `PR\_AUC = 0.8755` 在正类只有 4.17% 的测试集上是很强的, 远高于随机基线约 0.0417。

****验证集到测试集的泛化****

验证集:

```
Class 1 F1 = 0.8219
Precision = 0.8144
Recall    = 0.8294
PR_AUC    = 0.9024
```

测试集:

```
Class 1 F1 = 0.7950
Precision = 0.7645
Recall    = 0.8281
PR_AUC    = 0.8755
```

主要下降来自 precision: 测试集误报更多; recall

基本没掉。这说明模型识别正类的能力比较稳定, 但测试集中有更多正常 flow 的 embedding 和攻击 flow 更接近。

阈值迁移很好:

```
val threshold:      0.603939
test oracle threshold:0.608889
F1 gap:             0.000474
```

这说明你用验证集选的阈值几乎就是测试集最优阈值, 阈值没有明显过拟合。

****我对这个结果的判断****

这个结果可以作为一个很强的 no-context baseline: `Class1 F1=0.795`、`PR\_AUC=0.8755`, 说明 Stage1 embedding 已经包含大量可用于二分类的信息。

但它不能证明 `time\_only`

上下文有效, 因为当前模型没有使用历史上下文。下一步最有价值的是用同一份数据、同一阈值策略, 分别跑:

```
--model_type no_context_mlp
--model_type transformer
--model_type lstm
```

然后比较 `test PR\_AUC`、`test F1\_class1`、`FN/FP`。只有 Transformer/LSTM 显著超过这个 no-context baseline, 才能说明 Stage2 时间上下文真的带来了增益。

用户 2026-07-07 09:28:12

Files mentioned by the user:

```
## window size: 128 [INFO] Stage2 ----- seed: 130 [INFO] 随机种子已设置: 130 [INFO] S...:
C:\Users\XiaoyanXiong\.codex/attachments/f85eba2c-a4a4-4b19-85af-70e6259dd80c/pasted-text.txt
```

My request for Codex:

不同window size的运行结果。详细解析结果。我的代码是否还有优化空间? 是否需要换模型? 修改loss function? 或者任何其他的修正? 详细解析

助手 2026-07-07 09:28:30

我来把这份“不同 window size”的日志先结构化出来：每个 window 的模型类型、阈值、val/test 指标、混淆矩阵、early stopping 和上下文长度都拆开看。然后再结合 `s2` 的代码判断：结果差异到底来自 window，还是来自模型/训练配置。

助手 2026-07-07 09:29:50

关键结果已经出来了：这次四个 window 都是 `model\_type=transformer`，不是上一条日志里的 `no\_context\_mlp`。但四个 window 的测试指标几乎贴在一起，差异更多体现为 precision/recall 阈值取舍，而不是 PR-AUC 或总体排序能力的大幅变化。

助手 2026-07-07 09:31:31

****核心结论****

这组实验里，`window\_size=128/64/32/16` 都跑的是 `Stage2Transformer`，配置基本一致：`time\_only + online + include\_target=True + pooling=last + 1-layer Transformer`。结果显示：****window size 对最终测试性能影响很小****。最大的差异不是排序能力，而是阈值带来的 precision/recall 取舍。

window	Val F1_1	Val PR-AUC	Test P_1	Test R_1	Test F1_1	Test PR-AUC	混淆矩阵	`[TN FP; FN TP]`
---:	---:	---:	---:	---:	---:	---:	---	---
128	0.8245	0.9028	0.7574	0.8418	**0.7974**	**0.8758**	`[79911 949; 557 2963]`	
64	**0.8257**	0.9033	0.7802	0.8119	0.7958	0.8748	`[80055 805; 662 2858]`	
32	0.8227	0.9034	**0.7846**	0.8040	0.7942	0.8755	`[80083 777; 690 2830]`	
16	0.8255	**0.9040**	0.7826	0.8122	0.7972	0.8753	`[80066 794; 661 2859]`	

****怎么解读****

`window=128` 的测试 F1 和 PR-AUC 略高，但优势非常小：F1 只比 `window=16` 高约 `0.0002`，比 `window=64` 高约 `0.0016`。这个差距很可能低于随机种子波动。

`window=128` 更偏召回：漏报最少，`FN=557`，但误报最多，`FP=949`。

`window=32/16/64` 更偏精确率：误报少一些，但漏报更多。

如果你的 NIDS 目标是“尽量少漏报”，128 更合适；如果目标是“减少告警噪声/提高效率”，16 或 32 更划算。

测试 oracle gap 很小：

```
ws128 gap = 0.00149
ws64 gap ~= 0.00167
ws32 gap = 0.00335
ws16 gap = 0.00138
```

说明验证集选阈值基本能迁移到测试集，阈值没有严重过拟合。真正的问题是：Val PR-AUC 约 `0.903`，Test PR-AUC 约 `0.875`，有稳定的泛化下降。

****和 no-context baseline 对比****

你上一条 `timeOnly\_mlp/no\_context\_mlp` 的测试结果大约是：

```
F1_class1 = 0.7950
PR_AUC     = 0.8755
P1         = 0.7645
R1         = 0.8281
```

现在最好的 Transformer `window=128` 是：

```
F1_class1 = 0.7974
PR_AUC     = 0.8758
P1         = 0.7574
R1         = 0.8418
```

也就是说 Transformer 使用上下文后, F1 只提升约 `+0.0023`, PR-AUC 只提升约 `+0.0003`。这说明: **当前 time_only 上下文没有带来明显增益, Stage1 当前 flow embedding 已经承担了绝大多数分类能力。**

****是否需要换模型****

我不建议现在直接换成更大的 Transformer。当前结果不像 “模型容量不够”, 更像是
****上下文本身太泛、噪声太多****。

你的 `time\_only` 在

[context.py](/C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/context.py:107) 取的是全局时间上最近的 flow, 不管 source/destination 是否相关。对 NIDS 来说, 这些历史 flow 很多可能和当前 flow 没关系, 所以窗口变大只是在加入更多噪声。

更值得试的是:

1. endpoint / source_host / destination_host context
2. target-query attention flow query attend flow
3. concat flow + history attention summary +
4. log(delta_t) position age

现在的 Transformer 在

[model.py](/C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/model.py:602) 是整段 self-attention, 然后 `pooling=last` 取当前 token 表示。它能用历史, 但没有强迫模型学习 “当前 flow 与历史 flow 的关系”。target-query/cross-attention 会更直接。

****Loss 是否需要改****

暂时不把 loss 当第一优先级。你现在是:

```
WeightedRandomSampler:      8%
FocalLoss gamma=1.0
manual alpha=[1.0, 1.05]
```

这是比较温和的设置, 结果也稳定。强行提高 alpha 或 pos_weight, 大概率会提高 recall, 但 FP 会明显增加。你现在主要不是 loss 崩了, 而是上下文增益太小。

建议做小规模 ablation:

- A. focal gamma=1.0, alpha=[1.0,1.05], sampler=0.08
- B. CE loss, sampler=0.08, no class weight
- C. focal gamma=0.5, alpha=[1.0,1.0], sampler=0.08
- D. focal gamma=1.0, alpha=[1.0,1.05], sampler=0.05/0.10

不要一上来用很强的 class weight。你代码里 `class\_weighted\_loss=True` 会给少数类很大权重, 容易把 precision 打掉。

****代码里建议修正****

有一个实际代码问题值得修: 评估时 AMP 使用不一致。

训练 epoch 验证在

[trainer.py](/C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/trainer.py:410) 里用了 `amp\_enabled=(cuda)`; 但最终选阈值和最终保存 metrics 在 [trainer.py](/C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/trainer.py:612) 和 [trainer.py](/C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/trainer.py:672) 默认 `amp\_enabled=False`。而 `threshold\_oracle\_diagnosis()` 又用了 AMP。这会导致边界样本概率有极小差异, 所以你日志里 `TEST AT VAL THRESHOLD` 和最终 `[TEST] Full metrics` 偶尔差 1 个样本。

建议: 评估统一用 FP32, 或者统一用一个 `training.eval\_amp` 配置。论文/报告结果建议用 FP32。

第二个建议：你现在 `metric\_for\_best=pr\_auc`，但最终最关心的是 `F1\_class1`。日志里有些后期 epoch 的 val F1 比 best epoch 更高，但因为 PR-AUC 没涨，所以没保存。建议同时保存：

```
stage2_best_pr_auc.pt
stage2_best_f1_label1.pt
```

或者如果最终目标就是 F1，把 `metric\_for\_best` 改成 `f1\_label1`。

****下一步实验顺序****

我建议按这个顺序，不要先堆大模型：

```
1. window=16 128 3 seed
2. no_context_mlp baseline seed
3. context_method=endpoint/source_host/destination_host
4. AMP best_pr_auc best_f1 checkpoint
5. target-query attention gated context model
```

当前最稳的判断是：****window size 不是主要瓶颈；time_only 全局上下文的信息密度不够，模型和 loss 都不是第一优先级问题。****

用户 2026-07-07 09:44:35

如果修改模型里面的self-attention或者pooling会不会提高性能？后者任何其他的合理修改是否会提高性能？详细解析

助手 2026-07-07 09:45:14

会有提升空间，但我判断：****单纯把 pooling 从 `last` 改成 `mean/attention`，提升概率不高；真正更可能提升的是改 self-attention 的使用方式，让模型显式学习 “当前 flow 与历史 flow 的关系” 。****

你现在的结果已经告诉我们一件事：`window=128/64/32/16` 的 PR-AUC 都在 `0.875` 左右，F1_class1 都在 `0.794~0.797`。这说明瓶颈不是窗口长度，而是当前 `time\_only` 上下文没有被模型有效转化为额外判别信息。

****当前模型怎么用上下文****

你现在的 `Stage2Transformer` 是：

```
context_z = [history_1, history_2, ..., current_flow]
TransformerEncoder(context_z)
pooling = last
classifier(last_token)
```

因为 `include\_target=True`，当前 flow 被 append 到最后。`pooling=last` 就是取当前 flow 经过 self-attention 后的表示。

这个设计本身是合理的：分类目标是当前 flow，不是整段窗口，所以取最后 token 比 `mean pooling` 更符合任务。当前 token 在 self-attention 中可以 attend 到历史 token，因此理论上已经能利用上下文。

但实际结果只比 no-context baseline 好一点点，说明 self-attention 没学到很多有用历史模式，或者 `time\_only` 历史本身噪声太大。

****改 pooling 会怎样****

`last pooling`：最合理的默认选择。因为 label 属于当前 flow，最后 token 是当前 flow。

`mean pooling`：我不太推荐。它会把很多历史 flow 平均进去，尤其 `time\_only` 上下文里历史 flow 未必相关，容易稀释当前 flow 的强特征。

`attention pooling`：可以试，但不一定提升。它让模型自己选窗口里重要 token，不过如果没有明确告诉模型 “当前 flow 是查询目标”，attention pooling 可能学成 “整段窗口分类”，而不是 “当前 flow + 历史关系分类”。

所以 pooling 优先级：

```
last >= attention pooling > mean pooling
```

如果要试，我建议只试 `attention pooling`，不用优先试 `mean`。

****更值得改的是 self-attention 结构****

当前 Transformer 是普通 full self-attention。更适合你这个任务的结构应该是 ****target-query attention****：

```
current_z    query
history_z    key/value
  history_summary
  current_z
```

也就是让模型明确回答：

```
“    flow        ”
```

而不是让所有 token 混在一起 self-attend。

推荐结构：

```
current = last token
history = previous tokens

summary = MultiHeadAttention(query=current, key=history, value=history)
fused = concat(current, summary, current - summary, current * summary)
classifier(fused)
```

这比普通 self-attention 更贴合 NIDS

场景，因为攻击判断通常不是“整段窗口是否异常”，而是“当前连接相对于近期行为是否异常”。

****第二个强推荐：gated context****

你的 no-context baseline 已经很强，所以上下文不能强行覆盖当前 flow。应该让模型自己决定用多少上下文：

```
gate = sigmoid(MLP([current, summary]))
fused = current + gate * summary
```

这样如果历史上下文没用，模型可以退回当前

flow；如果上下文有用，才注入历史信息。这对你现在这种“上下文增益很小”的情况很合适。

****第三个推荐：加入时间差特征****

你现在的 `position\_mode=age` 只知道 token 顺序，不能知道真实时间间隔。对于流量检测，最近 10 条 flow 可能发生在 1 秒内，也可能发生在 10 分钟内，语义完全不同。

建议给每个历史 token 加：

```
log1p(current_time - history_time)
```

可以作为一个小 embedding 加到 token 上，或者拼接到每个 token 的 embedding 后再投影。这个可能比单纯改 pooling 更有价值。

****是否需要换模型****

暂时不建议直接换更大 Transformer。当前模型：

```
1 layer, d_model=128, nhead=4, dim_feedforward=256
```

已经足够表达简单上下文关系。结果没有明显提升，不像是容量不够，更像是：

```
time_only
  flow embedding
  target-history
```

如果要换模型，我建议优先顺序是：

1. Gated target-query attention

2. Transformer + current residual fusion
3. LSTM/GRU
4. Transformer

不要先上 2-4 层 Transformer。窗口 128 下算力更贵，也更容易过拟合噪声。

****Loss function 要不要改****

loss 可以微调，但不是第一瓶颈。你现在：

```
focal gamma=1.0
alpha=[1.0, 1.05]
sampler_pos_fraction=0.08
```

这是比较稳的设置。提高正类权重会提高 recall，但通常 FP 也会上升。你目前 `window=128` 已经 recall 高、precision 低；如果再加大 alpha，可能 F1 不一定涨。

更合理的是把 loss 当成第二阶段微调：

```
CE + sampler 0.08
Focal gamma 0.5
Focal gamma 1.0
alpha [1.0, 1.0]
alpha [1.0, 1.05]
```

先不要用很强的 class weight。

****我最建议的修改路线****

第一优先级：

```
model_type = target_attention
current query attends history
gated fusion
pooling      mean/last      current+summary
```

第二优先级：

```
delta_t embedding
```

第三优先级：

```
context_method = endpoint/source_host/destination_host
```

第四优先级：

```
best_pr_auc    best_f1_label1    checkpoint
eval AMP/FP32
```

****我的判断****

简单改 pooling，提升可能很小，甚至可能下降。

改 self-attention 为 ****target-query + gated fusion****，有真实提升机会。

再配合 endpoint/time-delta 上下文，比单纯调 loss 或加大模型更有希望。

用户 2026-07-07 10:10:08

请给我提供改 self-attention 为 target-query + gated

fusion，有真实提升机会。这个的代码，在我代码的基础上，尽量生成新的代码，少修改原有diamagnetic

助手 2026-07-07 10:10:33

我会按“少动原代码”的方式做：新增一个独立模型文件，原来的 `Transformer/LSTM/MLP`

都不碰；只在模型工厂和命令行 choices 里挂一个新的

`model\_type`。这样你可以直接用参数切换实验，不会破坏已有结果。

助手 2026-07-07 10:10:55

`git status` 因为 Windows sandbox 用户和仓库 owner 不一致被 Git 拦住了，我不会去改全局 Git 配置。代码结构已经够清楚：我会新增 `stage2/target\_query.py`，再在 `model.py` 工厂里 lazy import，避免循环依赖和大改原文件。

助手 2026-07-07 10:11:54

接下来我会写一个新的 `Stage2TargetQueryGatedAttention`：当前 flow 做 query，历史 flow 做 key/value；再用 gate 控制历史 summary 注入当前 flow 的强度。这样 no-history 样本会自动退回 current-only，不会产生 NaN。

助手 2026-07-07 10:13:01

新模型已经挂上了。现在我做两层验证：先做语法编译，再用一个很小的随机 batch 走一遍 forward，确认输出 shape、no-history fallback、mask 逻辑都正常。

助手 2026-07-07 10:13:15

本地 `python` 命令在这个 sandbox 里不可用，我会改用你机器上的 Python 路径验证。顺手把 `bool(no\_history.any())` 改成 `.item()`，这个在 CUDA 上更稳一点。

助手 2026-07-07 10:13:40

模型文件现在更稳了。下一步我用本机 Python 路径做语法编译；如果环境里有 torch，我再跑一次小 batch forward。

助手 2026-07-07 10:13:51

验证命令被 sandbox 拒绝执行用户目录下的 Python，可这一步对确认代码可运行有用。我会按权限流程请求一次执行许可，只做编译和一个小 forward 检查，不写用户目录外文件。

助手 2026-07-07 10:14:31

语法编译通过了。再做一个 forward smoke test：构造 3 条不同历史长度的假样本，包括一条 no-history，确认输出是 `[batch, 2]` 且没有 NaN。

助手 2026-07-07 10:15:05

小 forward 检查也通过了：输出 shape 是 `(3, 2)`，全部 finite。现在我整理一下改动点和你在 Colab/本地该怎么跑这个新模型。

助手 2026-07-07 10:15:29

已按“少改原代码”的方式加好了：新增模型文件，只在工厂和 CLI choices 里挂入口。

改动位置：

- 新增: [target_query.py](/C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/target_query.py:18)
- 挂到模型工厂: [model.py](/C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/model.py:641)
- CLI 支持 `--model\_type target\_query\_gated`: [run_stage2.py](/C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/run_stage2.py:95)
- 同步更新 tuned 入口: [run_stage2_tuned.py](/C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/run_stage2_tuned.py:95)

新模型逻辑是:

```
current flow = token
history flows = current token

current -> query
history -> key/value
summary = MultiHeadAttention(current, history)

gate = sigmoid(MLP([current, summary, current-summary, current*summary]))
fused = current + gate * context_update
classifier(fused)
```

它保留了你原来的 `SinusoidalPositionalEncoding`、`cls\_head`、`nhead`、`dropout`、`dim\_feedforward` 等配置。没有历史的样本会自动退回 current-only, 避免 attention 全 mask 后 NaN。

运行方式直接加参数即可:

```
!python /content/drive/MyDrive/s2/run_stage2.py \
--stage1_dir /content/drive/MyDrive/s1/0704/0704C_ar002_et12_20260511_001 \
--config /content/drive/MyDrive/s2/0706ar002_et12_20260511_001/timeOnly_v5/stage2_config_used.yaml \
--out_dir /content/drive/MyDrive/s2/0706ar002_et12_20260511_001/target_query_gated_ws128 \
--context_method time_only \
--context_policy online \
--window_size 128 \
--model_type target_query_gated \
--seed 130 \
--epochs 500 \
--batch_size 128
```

建议先跑 `window\_size=128` 和 `window\_size=16` 两个点, 对比你已有 Transformer:

```
test PR_AUC    > 0.8758
F1_class1    > 0.7974
FN          557  FP          949
```

验证已做:

```
py_compile:
batch forward: shape = (3, 2), finite = True
```

这个版本最适合验证“上下文是否真的有用”。如果它仍然只比 no-context 高一点点, 那下一步就不是继续堆模型, 而是改 `context\_method=endpoint/source\_host/destination\_host` 或加入真实 `delta\_t` 时间差特征。

用户 2026-07-07 10:50:02

这是stage1,方案C抛出的结果: {

```
"best_model_path": "/content/drive/MyDrive/s1/0704/0704C_ar002_et12_20260511_001/seqLen128heads
tage1_best_model.pt",
```

```
"best_epoch": 46,
```

```
"best_score": 0.9048145780941358,
```

```

"test_metrics": {
  "accuracy": 0.9834202417634511,
  "macro_precision": 0.8989505209974933,
  "macro_recall": 0.8918983147078002,
  "macro_f1": 0.8953896222507791,
  "weighted_precision": 0.9832784957495156,
  "weighted_recall": 0.9834202417634511,
  "weighted_f1": 0.9833463190159758,
  "precision_label1": 0.8069464544138929,
  "recall_label1": 0.7920454545454545,
  "f1_label1": 0.7994265232974911,
  "auc": 0.9890371580790593,
  "pr_auc": 0.8762558340285889,
  "per_class_f1": {
    "0": 0.9913527212040671,
    "1": 0.7994265232974911
  },
  "confusion_matrix": [
    [
      80193,
      667
    ],
    [
      732,
      2788
    ]
  ],
  "num_classes": 2,
  "num_samples": 84380,
  "loss": 0.06821018314045486,
  "threshold": 0.6840000000000005
}

```

详细解析

助手 2026-07-07 10:50:48

这个 Stage1 方案 C 结果很漂亮，而且它其实已经非常接近、甚至略微超过你目前的 Stage2 结果。重点不是 accuracy，而是 `Class\_1`、`PR\_AUC` 和混淆矩阵。

****核心指标****

测试集共有：

```
: 84380
0: 80860
1: 3520
```

混淆矩阵：

	pred 0	pred 1
true 0	80193	667
true 1	732	2788

也就是：

```
TP = 2788
FN = 732
FP = 667
TN = 80193
```

换算一下：

```
Precision_class1 = 0.8069
Recall_class1     = 0.7920
F1_class1         = 0.7994
PR_AUC            = 0.8763
AUC               = 0.9890
Accuracy          = 0.9834
```

`FPR = 667 / 80860 = 0.825%`，误报率很低。

`FNR = 732 / 3520 = 20.80%`，每 100 个异常大约漏掉 21 个。

所以这个模型是 ****偏保守、高 precision、低误报**** 的风格。

****和 Stage2 对比****

你之前最好的 Stage2 time-only Transformer `window=128` 大概是：

```
P1 = 0.7574
R1 = 0.8418
F1 = 0.7974
PR_AUC = 0.8758
CM = [[79911, 949], [557, 2963]]
```

这次 Stage1 C 是：

```
P1 = 0.8069
R1 = 0.7920
F1 = 0.7994
PR_AUC = 0.8763
CM = [[80193, 667], [732, 2788]]
```

对比非常关键：

Stage1 C	Stage2	: 949 -> 667	282	FP
Stage1 C	Stage2	: 557 -> 732	175	FN
Stage1 C	F1	: 0.7974 -> 0.7994		
Stage1 C	PR_AUC	: 0.8758 -> 0.8763		

所以 Stage1 C 目前不是“被 Stage2 明显提升”的 baseline，反而是 ****当前最强或并列最强的单模型结果****。

****threshold 的含义****

你的阈值是：

```
threshold = 0.684
```

这个阈值偏高，所以模型更谨慎：只有概率较高才判为异常。这解释了为什么 precision 高、FP 低，但 recall 没有 Stage2 高。

如果你把阈值降低，可能得到类似 Stage2 的风格：

```
recall
FN
FP
precision
```

所以 Stage1 C 和 Stage2 目前更像是 ****不同阈值/模型风格的 trade-off****，不是 Stage2 明显更强。

****best_score 解读****

`best\_score = 0.9048` 很可能是验证集上的最佳选择指标，不等于测试集最终 F1 或 PR-AUC。测试集最终：

```
F1_class1 = 0.7994
PR_AUC = 0.8763
```

验证到测试存在下降，这是正常的，也说明测试集比验证集更难一些。

****我的判断****

这个 Stage1 方案 C 很强，说明 Stage1 的 flow-level 表征已经捕获了主要判别信息。Stage2 time-only 上下文目前没有带来明显 ranking 增益，因为 PR-AUC 几乎没变。

如果你的目标是：

```
Stage1 C
Stage2 window=128
F1 Stage1 C
```

****下一步建议****

先不要急着换 loss。你应该把 Stage1 C 当作当前强 baseline，然后用刚加的 `target\_query\_gated` 去验证 Stage2 是否真的能超过它。

重点看这三个数：

```
PR_AUC    > 0.8763
F1_class1  > 0.7994
FP        667      FN
```

如果 `target\_query\_gated` 仍然不能超过 Stage1 C，那说明瓶颈不是模型结构，而是 Stage2 的 `time\_only` 上下文信息密度不足，需要改 `endpoint/source\_host/destination\_host` 或加入真实时间差特征。

用户 2026-07-07 11:01:10

Files mentioned by the user:

```
## class Stage2ResidualContextTransformer(nn.Module): """ Residual context Transfo...:
C:\Users\XiaoyanXiong\codex\attachments\b7367e35-69cf-44e9-93e8-0aae79b3e59f/pasted-text.txt
```

My request for Codex:

这个和你给我的那个Stage2TargetQueryGatedAttention，有什么异同？详细解析

助手 2026-07-07 11:01:31

我先读一下你贴的 `Stage2ResidualContextTransformer` 代码，再和我们新增的 `Stage2TargetQueryGatedAttention` 逐层对照：输入、上下文建模、融合方式、pooling、对 no-history 的处理，以及它们各自更可能改善哪类指标。

助手 2026-07-07 11:02:38

****一句话区别****

`Stage2ResidualContextTransformer` 是 ****“保留 current-only 分类器，在 logits 上加一个上下文修正项”****；我给你的 `Stage2TargetQueryGatedAttention` 是 ****“用当前 flow 作为 query 去主动检索历史 flow，然后在表示层 gated fusion”****。

两者都想解决同一个问题：Stage1/current flow 已经很强，历史上下文不能乱加，必须可控。但它们“怎么用上下文”不一样。

****结构对比****

维度	ResidualContextTransformer	TargetQueryGatedAttention
核心思想	`base_logits(current) + context_delta`	`current + gate * context_update`
上下文编码	TransformerEncoder 编码历史序列	当前 flow query, history 做 key/value
当前 flow 是否主动查询历史	否, history summary 基本独立于 current	是, attention 权重由 current 决定
融合位置	logits 层融合	representation 层融合
gate 形状	标量 gate `[B,1]`	向量 gate `[B,D]`
上下文影响强度	`context_scale * gate * delta_logits`	`gate * context_update`
是否保留 no-context 分支	很明确, `base_head(current_z)`	隐式保留, `current + gated update`
计算复杂度	Transformer self-attention, 约 $O(L^2)$	单 query cross-attention, 约 $O(L)$
更可能改善	稳定性、precision、减少上下文伤害	真正挖掘 current-history 关系

****Residual 这个方案的优点****

它非常保守，这一点对你当前任务是有意义的。因为你的 Stage1 C 已经很强：

```
F1_class1 = 0.7994
PR_AUC     = 0.8763
```

而之前普通 Stage2 Transformer 并没有明显超过它。所以 Residual 结构的想法是对的：不要让上下文完全重写当前 flow 判断，只让上下文学一个修正项。

它的公式是：

```
logits = base_logits + context_scale * gate * delta_logits
```

这会让模型天然更接近 no-context baseline。如果上下文没用，gate 可以变小；如果上下文有用，delta 才修正结果。这个设计很适合你现在这种“current embedding 很强，上下文增益不稳定”的情况。

****Residual 的局限****

它不是 target-query attention。它的 context summary 是：

```
TransformerEncoder(history)
context_summary = mean(history_hidden)
```

也就是说，它更像是在问：

但不是明确问：

```
flow      flow
```


对于 `time\_only` 上下文，这个区别很关键。因为 time-only 历史里很多 flow 可能和当前 flow 没有关系。平均池化容易把有用历史和噪声历史混在一起。

所以 Residual 可能更稳定，但不一定能带来真正大的上下文增益。

****TargetQuery 的优点****

我给你的 `Stage2TargetQueryGatedAttention` 明确做：

```
query = current flow
key/value = history flows
summary = attention(current, history)
```

它问的是：

```
flow          token
```

这比 mean pooling 更贴合 NIDS。比如当前 flow 是某个 endpoint 的异常连接，模型有机会从历史里找相似/相关/突变的 flow，而不是平均全部历史。

它还用了：

```
[current, summary, current-summary, current*summary]
```

`current-summary` 看差异，`current\*summary` 看相似/交互。这个对“当前行为相对历史是否异常”更直接。

****一个重要代码问题****

你贴的 `Stage2ResidualContextTransformer` 对 no-history 样本处理不完整。

代码里写了：

```
# If a sample has no history, keep one safe token masked out.
```

但实际没有做 safe token。它直接：

```
key_padding_mask = ~history_mask
h = self.encoder(x_ctx, src_key_padding_mask=key_padding_mask)
```

如果某个样本没有 history，`history\_mask` 全 False，`key\_padding\_mask` 全 True。PyTorch attention 在全 mask 时可能产生 NaN。你的 online time_only 下只有极少 train 样本无 history，但如果换 `endpoint/source\_host/split\_isolated`，no-history 会更多，这个问题会变严重。

我给你的 TargetQuery 里专门处理了这个：

```
no_history      safe token
attention      summary      0
gate/context_update  0
```

所以它在 no-history 场景更稳。

****哪个更值得跑****

我建议两个都值得，但目的不同：

```
ResidualContextTransformer
“      ”      Stage1 C
precision

TargetQueryGatedAttention
“  flow      ”
recall  PR-AUC
```

如果只选一个，我更偏向

****TargetQueryGatedAttention****，因为你现在的问题不是“模型不稳定”，而是“time_only 上下文没有被有效利用”。Target-query 更直接解决这个问题。

但如果你的目标是“不许比 Stage1 C 差太多，尽量稳”，Residual 更安全。

****最佳方案其实是二者结合****

我最推荐的最终结构是 hybrid:

```
base_logits = base_head(current)

summary = target_query_attention(current, history)

fused = [current, summary, current-summary, current*summary]
delta_logits = delta_head(fused)
gate = sigmoid(gate_head(fused))

logits = base_logits + context_scale * gate * delta_logits
```

也就是:

```
Residual    logits correction
+
TargetQuery  current-aware history summary
```

这会同时拥有:

1. current-only baseline
2. context
3. summary flow
4. context_scale

这比两者单独用都更符合你现在的实验现象。当前 Stage1 已经很强, Stage2 应该做 “有约束的上下文修正”, 而不是重新学习一个完全上下文模型。

用户 2026-07-07 11:06:54

Files mentioned by the user:

```
## [INFO] Stage2 ----- seed: 130 [INFO] 随机种子已设置: 130 [INFO] Stage2 ----- d...:
C:\Users\XiaoyanXiong\codex\attachments\344c8d88-1e8a-4b96-b99f-a2254c3d2999\pasted-text.txt
```

```
## My request for Codex:
model_type=target_query_gated
详细解析, 还能继续优化吗?
```

助手 2026-07-07 11:07:16

我先把这次 `target\_query\_gated` 的完整日志拆出来, 尤其看: best epoch、val 阈值、test 混淆矩阵、和普通 Transformer/Stage1 C 的差异。然后我会判断它是 “结构有效但阈值/训练需调”, 还是 “上下文本身仍然信息不足”。

助手 2026-07-07 11:08:27

****结论****

`target\_query\_gated` 没有带来真实提升。它的行为很清楚: **召回略高, 但 precision 掉了, 整体 F1/PR-AUC 没超过普通 Transformer, 也没超过 Stage1 方案 C**。

这说明 target-query 结构本身没有坏, 但在当前 `time\_only + window=128` 上, 它更像是在引入一些历史噪声, 帮助多抓一点正类, 同时也多报了不少正常流量。

****本次结果****

配置:

```
model_type = target_query_gated
context_method = time_only
```

```
window_size = 128
context_policy = online
lr = 3e-5
sampler_pos_fraction = 0.08
focal_gamma = 1.0
alpha = [1.0, 1.05]
best_epoch = 5
```

测试集:

```
Accuracy      = 0.9819
P_class1      = 0.7538
R_class1      = 0.8420
F1_class1     = 0.7955
PR_AUC        = 0.8751
AUC           = 0.9888
threshold     = 0.5841
```

混淆矩阵:

	pred 0	pred 1
true 0	79892	968
true 1	556	2964

也就是:

```
TP = 2964
FN = 556
FP = 968
TN = 79892
```

这个模型偏“多抓异常”：漏报少，但误报明显多。

****和之前模型对比****

和普通 `Stage2 Transformer window=128` 比:

```
Transformer:
P1=0.7574, R1=0.8418, F1=0.7974, PR_AUC=0.8758
CM=[[79911, 949], [557, 2963]]
```

```
target_query_gated:
P1=0.7538, R1=0.8420, F1=0.7955, PR_AUC=0.8751
CM=[[79892, 968], [556, 2964]]
```

差异非常小，但 `target\_query\_gated` 稍差:

```
F1      0.0019
PR_AUC   0.0007
FP      19
FN       1
```

所以它没有证明 target-query 比普通 self-attention 更好。

和 Stage1 方案 C 比:

```
Stage1 C:
P1=0.8069, R1=0.7920, F1=0.7994, PR_AUC=0.8763
CM=[[80193, 667], [732, 2788]]
```

```
target_query_gated:
P1=0.7538, R1=0.8420, F1=0.7955, PR_AUC=0.8751
CM=[[79892, 968], [556, 2964]]
```

解释:

```
target_query_gated      176
                        301
F1      0.0039
PR_AUC   0.0012
```

如果你更重视 recall，它有价值；如果看综合 F1/PR-AUC，Stage1 C 仍然更强。

****训练现象****

验证集最好：

```
Val F1_class1 = 0.8258
Val PR_AUC    = 0.9036
```

测试集：

```
Test F1_class1 = 0.7955
Test PR_AUC    = 0.8751
```

val 到 test 的 drop 很明显：

```
F1 drop      ≈ 0.0303
PR_AUC drop ≈ 0.0286
```

这和之前 Stage2 一样，说明模型在 val 上能调出很漂亮的阈值，但 test 上没有真正提升排序能力。`threshold transfer gap=0.0011` 很小，所以问题不是阈值选错，而是 ****ranking 本身没有变强****。

****为什么没提升****

核心原因还是 `time\_only` 上下文太泛。`target\_query\_gated` 会让当前 flow 去查询最近 127 条 flow，但这些 flow 不一定和当前 flow 的 endpoint、source、destination 有关。模型可能学到一些“近期全局流量状态”，但这对单条 flow 分类帮助有限，还可能把正常样本推成异常。

第二个原因是我给你的版本是 representation-level fusion：

```
fused = current + gate * context_update
classifier(fused)
```

它没有显式保留一个强 no-context logits 分支。你的 Stage1/current embedding 很强，所以更理想的是让上下文只做“修正”，而不是重新改写表示。

****还能优化吗****

可以，但我建议优化方向不是继续加大模型，而是让上下文更保守、更相关。

第一优先级：改成 hybrid residual target-query。

推荐结构：

```
base_logits = base_head(current)

summary = target_query_attention(current, history)

fused = [current, summary, current-summary, current*summary]
delta_logits = delta_head(fused)
gate = sigmoid(gate_head(fused))

logits = base_logits + context_scale * gate * delta_logits
```

这比当前版本更适合你的结果，因为它保留 Stage1 C/current-only 的强 precision，同时让上下文只做小幅修正。`context\_scale` 建议先试：

```
0.1, 0.2, 0.3, 0.5
```

第二优先级：让 context 初始影响更小。

当前 gate 初始大概可能在 0.5 附近，context_update 是随机的，训练早期可能扰动 current embedding。建议：

```
gate final bias      -2.0  -3.0
context_update       zero-init
```

这样模型一开始接近 no-context，只有当上下文真的有时才逐渐打开 gate。

第三优先级：换 context，而不是换大模型。

优先试：

```
context_method=endpoint
context_method=source_host
context_method=destination_host
```

尤其是 `endpoint`，比 `time\_only` 更可能提供有意义的历史行为。现在 time-only 全局历史噪声太多。

第四优先级：跑小窗口 target-query。

你现在 `window=128`，历史多但不一定相关。建议试：

```
window_size=16
window_size=32
window_size=64
```

如果小窗口 precision 上来，说明大窗口确实在引入噪声。

第五优先级：训练设置轻调。

当前 best epoch=5，后面 PR-AUC 开始下降，说明容易早期过拟合。可以试：

```
lr = 1e-5 2e-5
dropout = 0.15 0.20
weight_decay = 1e-4
sampler_pos_fraction = 0.05 0.08
alpha = [1.0, 1.0] [1.0, 1.05]
```

你当前模型 FP 偏多，所以不建议继续加大正类权重。

****我的建议路线****

下一步不要继续堆 Transformer 层数。最值得做的是：

1. hybrid residual target-query
2. gate bias context_update zero-init
3. context_scale 0.1/0.2/0.3/0.5
4. time_only ws16/ws32/ws128
5. endpoint ws16/ws32/ws128

目前这个 `target\_query\_gated` 的结论是：****结构没有明显赢，但暴露出一个方向：上下文可以提高 recall；下一步要控制它对 precision 的破坏。****

用户 2026-07-07 11:37:48

Files mentioned by the user:

```
## [INFO] Stage2 ----- seed: 130 [INFO] 随机种子已设置: 130 [INFO] Stage2 ----- d...:
C:\Users\XiaoyanXiong\codex/attachments/6d8b58ca-bf4e-42b1-9603-b2f701f5c514/pasted-text.txt
```

My request for Codex:

详细解析运行结果

助手 2026-07-07 11:38:03

我先把这份新日志拆出来，确认这次跑的是哪一个模型、窗口和配置，再和你前面的 Stage1 C、普通 Transformer、target_query_gated 做横向对比。重点会看 test PR-AUC、F1_class1、precision/recall 和混淆矩阵的取舍。

助手 2026-07-07 11:38:50

****结论****

这次是 `target\_query\_gated + time\_only + window\_size=16`。结果比同模型 `window=128` 略好，但没有明显超过普通 Transformer，也没有超过 Stage1 方案 C。它的特点很明确：****提高召回、降低漏报，但误报偏多，precision 被拉低****。

****本次配置****

```
model_type = target_query_gated
context_method = time_only
window_size = 16
context_policy = online
seed = 130
lr = 3e-5
weight_decay = 3e-4
sampler_pos_fraction = 0.08
loss = FocalLoss(gamma=1.0, alpha=[1.0, 1.05])
metric_for_best = pr_auc
best_epoch = 5
```

上下文长度正常：

```
15      flow +      flow = 16
test/val  history_len=15
train      history
```

所以这次不是数据构造问题，模型确实在用短窗口历史。

****测试集结果****

测试集总数：

```
num_samples = 84380
positive_count = 3520
positive_rate = 4.17%
```

最终指标：

```
Accuracy      = 0.9821
Macro F1      = 0.8938
P_class1      = 0.7546
R_class1      = 0.8446
F1_class1     = 0.7971
AUC           = 0.9888
PR_AUC        = 0.8754
threshold     = 0.5693
```

混淆矩阵：

	pred 0	pred 1
true 0	79893	967
true 1	547	2973

也就是：

```
TP = 2973
FN = 547
FP = 967
TN = 79893
```

换算：

```
FNR = 547 / 3520 = 15.54%
FPR = 967 / 80860 = 1.20%
    = 3940
    = 2973
```

它抓异常抓得比较积极，所以 recall 高，但误报也多。

****阈值表现****

验证集选出的阈值：

```
val threshold = 0.5693
```

测试 oracle threshold 也是：

```
test oracle threshold = 0.5693
threshold transfer gap = 0.0000
```

这点很好，说明不是阈值迁移失败。验证集选出的阈值在测试集上刚好也是 F1 最优网格阈值。问题在于：****排序能力 PR-AUC 没有明显提升****。

****训练过程****

训练到：

```
best_epoch = 5
early stopping at epoch = 20
```

PR-AUC 在前 5 轮上升：

```
epoch 1: val_pr_auc = 0.8751
epoch 2: val_pr_auc = 0.8992
epoch 3: val_pr_auc = 0.9026
epoch 4: val_pr_auc = 0.9032
epoch 5: val_pr_auc = 0.9035 <- best
```

后面 val F1 有时更高，例如 epoch 12/13 附近 `val\_f1\_label1=0.8272`，但 `metric\_for\_best=pr\_auc`，所以最终保存的是 epoch 5。

如果你的目标是最终 `F1\_class1`，可以单独试一次：

```
metric_for_best: f1_label1
```

这可能改变最终 checkpoint，不过不保证测试会更好。

****和 target_query_gated window=128 对比****

之前 `target\_query\_gated ws128`：

```
P1 = 0.7538
R1 = 0.8420
F1 = 0.7955
PR_AUC = 0.8751
CM = [[79892, 968], [556, 2964]]
```

这次 `target\_query\_gated ws16`：

```
P1 = 0.7546
R1 = 0.8446
F1 = 0.7971
PR_AUC = 0.8754
CM = [[79893, 967], [547, 2973]]
```

变化：

```
FP 1
FN 9
F1 +0.0016
PR_AUC +0.0003
```

所以短窗口略好。说明 `window=128` 的远历史没有提供增益，甚至可能有噪声。对这个模型来说，`ws16` 更合理。

****和普通 Transformer window=16 对比****

之前普通 Transformer `ws16` 大概是：

```
P1 = 0.7826
```

```
R1 = 0.8122
F1 = 0.7972
PR_AUC = 0.8753
CM = [[80066, 794], [661, 2859]]
```

现在 target-query `ws16`:

```
P1 = 0.7546
R1 = 0.8446
F1 = 0.7971
PR_AUC = 0.8754
CM = [[79893, 967], [547, 2973]]
```

变化:

```
114
173
F1
PR_AUC
```

也就是说, target-query 确实把模型推向了 “更高 recall” , 但没有提高整体排序质量。

****和 Stage1 方案 C 对比****

Stage1 C:

```
P1 = 0.8069
R1 = 0.7920
F1 = 0.7994
PR_AUC = 0.8763
CM = [[80193, 667], [732, 2788]]
```

target_query_gated ws16:

```
P1 = 0.7546
R1 = 0.8446
F1 = 0.7971
PR_AUC = 0.8754
CM = [[79893, 967], [547, 2973]]
```

解释:

```
185
300
F1 0.0024
PR_AUC 0.0009
```

如果你的目标是安全检测尽量少漏报, target-query 有价值。

如果目标是综合 F1 或告警质量, Stage1 C 仍然更稳。

****我的判断****

这次结果说明:

```
1. target_query_gated
2. window=16 window=128 time_only
3. PR-AUC
4. time_only recall precision
```

****还能怎么优化****

最值得做的是保守化上下文影响:

```
base_logits = current-only classifier
delta_logits = target-query context correction
logits = base_logits + context_scale * gate * delta_logits
```

也就是把现在的 `target\_query\_gated` 改成 hybrid residual target-query。这样可以保住 Stage1/current-only 的 precision, 让上下文只做小幅修正。

建议实验优先级：

1. target_query_gated + ws16 + metric_for_best=f1_label1
2. hybrid residual target-query + context_scale=0.1/0.2/0.3
3. context_method=endpoint + ws16/ws32
4. target_query_gated gate bias=-2

现在这个结果已经很有信息量：**结构方向不是完全错，但 time_only 上下文太泛，直接融合会增加误报。下一步要让上下文“更相关”或“更保守”。**

用户 2026-07-07 12:09:35

Files mentioned by the user:

```
## [INFO] Stage2 ----- seed: 130 [INFO] 随机种子已设置: 130 [INFO] Stage2 ----- d...:
C:\Users\XiaoyanXiong\codex/attachments/3eef8f7e-7afd-4917-9165-d6bb16a63097/pasted-text.txt
```

My request for Codex:

```
!python /content/drive/MyDrive/s2/run_stage2.py \
--stage1_dir /content/drive/MyDrive/s1/0704/0704C_ar002_et12_20260511_001 \
--config /content/drive/MyDrive/s2/0707ar002_et12_20260511_001/timeOnly_residual_transformer/stage
2_config_used.yaml \
--out_dir /content/drive/MyDrive/s2/0707ar002_et12_20260511_001/timeOnly_residual_transformer/ws16
\
--context_method time_only \
--context_policy online \
--window_size 16 \
--model_type residual_transformer \
--seed 130 \
--epochs 500 \
--batch_size 128
```

运行结果。

详细解析运行结果。还有什么提升的可能吗？代码或配置中任何可以修改的可能？详细解析

助手 2026-07-07 12:09:49

我先把这次 `residual\_transformer ws16` 日志读出来，并和你前面几组关键 baseline 对齐：Stage1 C、普通 Transformer ws16/128、target_query_gated ws16。这样能看清它到底是在改善 precision、recall，还是只是阈值风格变化。

助手 2026-07-07 12:10:04

我已经读到一个很关键的异常：这次 `residual\_transformer` 在最终保存 train predictions 时出现了 NaN warning。它不会直接影响你打印的 test 指标，但说明模型代码对“无历史样本”处理有 bug，尤其在 `time\_only` 的第一条 train flow 上会触发。这个要优先修。

助手 2026-07-07 12:10:53

****结论****

这次 `residual\_transformer + time\_only + window\_size=16` 没有提升。它比 `target\_query\_gated ws16` 更保守一些，precision 高一点、recall 低一点；但整体 `F1\_class1` 和 `PR\_AUC` 都没有超过你已有的强 baseline。

更重要的是：日志里出现了 ****NaN warning****。这说明当前 `residual\_transformer` 代码有一个需要修的稳定性问题，优先级比继续调参更高。

****本次配置****

```
model_type = residual_transformer
context_method = time_only
window_size = 16
context_policy = online
context_scale = 0.5
dropout = 0.25
nhead = 4
num_layers = 1
metric_for_best = pr_auc
best_epoch = 8
early_stop = epoch 23
```

数据和窗口都正常：

```
train = 295329
val   = 42189
test  = 84380
z_dim = 128
context length = 16
```

也就是每条样本大多是：

```
15      flow +      flow
```

****测试集结果****

官方测试结果使用验证集选出的阈值 `0.6138`：

```
Accuracy      = 0.9824
P_class1      = 0.7727
R_class1      = 0.8179
F1_class1     = 0.7946
AUC           = 0.9886
PR_AUC        = 0.8734
```

混淆矩阵：

	pred 0	pred 1
true 0	80013	847
true 1	641	2879

也就是：

```
TP = 2879
FN = 641
FP = 847
TN = 80013
```

漏报率：

```
641 / 3520 = 18.21%
```

误报率：

```
847 / 80860 = 1.05%
```

这个模型比 `target\_query\_gated` 更谨慎，误报少一些，但漏报更多。

****Test Oracle****

测试集 oracle 阈值是 `0.6534`，只是诊断用，不能作为正式结果：

```
P_class1 = 0.7973
R_class1 = 0.7969
F1_class1 = 0.7971
CM = [[80147, 713], [715, 2805]]
```

这说明如果阈值更高，模型会更接近 Stage1 C 的风格：precision 上升、recall 下降。
但 PR-AUC 不变，仍是 `0.8734`，说明排序能力本身没有提升。

****和已有结果对比****

```
| 模型 | P1 | R1 | F1_1 | PR_AUC | 混淆矩阵 |  
|---|---|---|---|---|---|  
| Stage1 C | **0.8069** | 0.7920 | **0.7994** | **0.8763** | `[[80193,667],[732,2788]]` |  
| Transformer ws16 | 0.7826 | 0.8122 | 0.7972 | 0.8753 | `[[80066,794],[661,2859]]` |  
| target_query_gated ws16 | 0.7546 | **0.8446** | 0.7971 | 0.8754 | `[[79893,967],[547,2973]]` |  
| residual ws16 | 0.7727 | 0.8179 | 0.7946 | 0.8734 | `[[80013,847],[641,2879]]` |  
| residual ws16 oracle | 0.7973 | 0.7969 | 0.7971 | 0.8734 | `[[80147,713],[715,2805]]` |
```

所以：

```
residual_transformer      Stage1 C  
Transformer ws16  
target_query_gated ws16
```

它只是改变了 precision/recall 取舍。

****关键问题：NaN****

日志里有：

```
[WARNING] NaN detected in loss!  
[WARNING] AUC calculation failed: Input contains NaN.
```

这个大概率来自 residual 模型里 history mask 的处理。你的代码中 current token 被移除后，第一条或少数样本可能没有任何 history：

```
history_mask      False  
key_padding_mask  True
```

TransformerEncoder 遇到全 mask attention 可能产生 NaN。你前面 `target\_query\_gated` 我专门做了 no-history safe token，但 residual 版本没有真正处理。

这个必须修，否则 train predictions / train metrics 可能污染，换 `endpoint/source\_host` 时 no-history 样本会更多，问题会放大。

****代码建议 1：修 no-history****

在 residual forward 里加 safe history mask：

```
history_mask = self._history_mask_without_current(mask)  
no_history = ~history_mask.any(dim=1)  
safe_history_mask = history_mask.clone()  
  
if no_history.any():  
    safe_history_mask[no_history, 0] = True
```

然后 encoder 用：

```
key_padding_mask = ~safe_history_mask
```

最后 summary 后把 no-history 的 context 置零：

```
context_summary = masked_mean_pooling(h, safe_history_mask)  
context_summary = context_summary.masked_fill(no_history.unsqueeze(-1), 0.0)
```

这样无历史样本会退回：

```
logits = base_logits(current)
```

****代码建议 2：让上下文初始影响更小****

现在：

```
context_scale = 0.5
gate           0.5
```

这意味着一开始上下文修正强度可能不小。建议：

```
context_scale = 0.1  0.2
```

并把 gate 最后一层 bias 初始化成负数：

```
nn.init.constant_(self.gate[-2].bias, -2.0)
```

这样初始 gate 约 $\text{sigmoid}(-2)=0.119$ ，模型先接近 Stage1/current-only，再慢慢学习是否打开上下文。

****配置建议****

优先试：

```
model:
  context_scale: 0.1  # 0.2, 0.3
training:
  metric_for_best: f1_label1
```

这次 best checkpoint 用 `pr_auc`，但最终你关心的是 `F1_class1`。日志里 epoch 9/12/17 的 val F1 比 best epoch 8 更高，但没有被保存。可以单独跑一次 `metric_for_best=f1_label1`。

不建议现在加大正类权重。当前模型不是 recall 太低的问题，而是 PR-AUC 已经下降，说明排序能力没变好。

****下一步优先级****

1. 先修 no-history NaN，这是必须项。
2. 跑 `context_scale=0.1/0.2/0.3`。
3. 跑 `metric_for_best=f1_label1`。
4. 跑 `context_method=endpoint` 或 `source_host`，因为 `time_only` 仍然太泛。
5. 如果继续做 residual，建议改成 `target-query summary + residual logits correction`，而不是 TransformerEncoder + mean pooling。

当前判断：`residual_transformer` 的想法是合理的，但这次实现还没有赢，且有 no-history NaN bug。提升空间主要在 ****修稳定性、减小上下文修正强度、换更相关的 context****，不是继续加大模型。

用户 2026-07-07 13:56:38

Files mentioned by the user:

```
## [INFO] Stage2 ----- seed: 130 [INFO] 随机种子已设置: 130 [INFO] Stage2 ----- d...:
C:\Users\XiaoyanXiong\codex/attachments/daff1aaf-61be-4b96-9548-e1255a519ba8/pasted-text.txt
```

My request for Codex:

```
--context_method time_only \
--context_policy online \
--window_size 128 \
--model_type residual_transformer \
--seed 130 \
--epochs 500 \
--batch_size 128
```

详细解析结果

助手 2026-07-07 13:56:54

我先把这次 `residual\_transformer + window\_size=128` 的日志读出来，再和刚才 `residual\_ws16`、`target\_query\_gated\_ws16/128`、Stage1 C 做对比。重点看它是因为长窗口带来上下文增益，还是只是改变了阈值下的 precision/recall。

助手 2026-07-07 13:57:38

****结论****

这次 `residual\_transformer + time\_only + window\_size=128` 比 `residual\_transformer\_ws16` 略好，但仍然没有超过你已有的强 baseline。它的主要效果是：****更偏 recall，漏报减少，但误报增加；PR-AUC 仍然偏低，说明排序能力没有真正提升。****

还有一个必须注意的问题：日志中再次出现 ****NaN warning****，说明 `residual\_transformer` 代码里 no-history 样本处理仍有 bug，需要先修。

****本次配置****

```
model_type = residual_transformer
context_method = time_only
context_policy = online
window_size = 128
context_scale = 0.5
seed = 130
batch_size = 128
metric_for_best = pr_auc
best_epoch = 8
early_stopping = epoch 23
```

上下文构造正常：

```
test history_len = 127
val history_len = 127
train mean history_len = 126.97
```

所以绝大多数样本都是：

```
127      flow +      flow
```

****测试集结果****

官方测试结果使用验证集阈值：

```
threshold = 0.5693
```

测试集指标：

```
Accuracy      = 0.9821
Macro F1      = 0.8935
P_class1      = 0.7573
R_class1      = 0.8395
F1_class1     = 0.7963
AUC           = 0.9885
PR_AUC        = 0.8740
```

混淆矩阵：

	pred 0	pred 1
true 0	79913	947
true 1	565	2955

也就是：

```
TP = 2955
FN = 565
FP = 947
TN = 79913
```

换算：

```
= 565 / 3520 = 16.05%
= 947 / 80860 = 1.17%
```

这个模型比 Stage1 C 更积极，会抓到更多异常，但也带来更多误报。

****阈值诊断****

测试 oracle 阈值是：

```
0.5643
```

和验证集阈值 `0.5693` 非常接近：

```
threshold transfer gap = 0.000655
```

这说明阈值迁移很好，不是“val 阈值选坏了”。问题主要是模型排序能力没有提高：

```
Val PR_AUC = 0.9029
Test PR_AUC = 0.8740
drop ≈ 0.0289
```

val 到 test 的下降仍然明显。

****和 residual ws16 对比****

模型	P1	R1	F1_1	PR_AUC	CM
residual ws16	0.7727	0.8179	0.7946	0.8734	`[[80013,847],[641,2879]]`
residual ws128	0.7573	0.8395	0.7963	0.8740	`[[79913,947],[565,2955]]`

`ws128` 相比 `ws16`：

```
FN 76
FP 100
F1 +0.0016
PR_AUC +0.0005
```

所以长窗口有一点 recall 增益，但非常小，而且靠牺牲 precision 换来。没有证明 128 窗口真正带来了强上下文信息。

****和其他模型对比****

模型	P1	R1	F1_1	PR_AUC	评价
Stage1 C	**0.8069**	0.7920	**0.7994**	**0.8763**	综合最好，误报少
Transformer ws128	0.7574	0.8418	0.7974	0.8758	recall 高，略强于 residual
target_query ws16	0.7546	**0.8446**	0.7971	0.8754	recall 最高
residual ws128	0.7573	0.8395	0.7963	0.8740	没有优势
residual ws16	0.7727	0.8179	0.7946	0.8734	更保守但仍弱

结论很明确：

```
residual_transformer ws128      Stage1 C
Transformer ws128
target_query_gated ws16
```

它目前不是最佳方向。

****关键代码问题****

日志里有：

```
[WARNING] NaN detected in loss!
[WARNING] AUC calculation failed: Input contains NaN.
```

这通常来自 residual 模型里移除 current token 后, 某些样本没有 history, 导致 Transformer attention 全 mask。你代码中的相关位置在 [model.py](/C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/model.py:743) 附近。

这不会明显影响 test 指标, 因为 test 几乎都有完整 history; 但会污染 train predictions/metrics, 而且换成 `endpoint/source\_host` 后 no-history 会更多, 风险会放大。

****优先修正****

1. residual 模型必须加 no-history safe mask。

没有历史时, 让 context summary 为 0, 并让 logits 退回 base logits。

2. 降低上下文影响:

```
model:
  context_scale: 0.1 # 0.2, 0.3
```

现在 `context\_scale=0.5` 偏大, 容易让上下文修正拉低 precision。

3. gate 初始化更保守:

```
nn.init.constant_(self.gate[-2].bias, -2.0)
```

让初始 gate 约等于 0.12, 而不是默认接近 0.5。

4. 如果目标是 F1, 不要只用 `pr\_auc` 保存 best:

```
training:
  metric_for_best: f1_label1
```

这次后面一些 epoch 的 val F1 不低, 但 best checkpoint 按 PR-AUC 固定在 epoch 8。

****下一步建议****

不要继续加大 residual Transformer。当前瓶颈不是容量, 而是 `time\_only` 上下文太泛。

更值得试:

1. residual no-history NaN
2. residual ws128 + context_scale=0.1/0.2/0.3
3. residual + metric_for_best=f1_label1
4. context_method=endpoint + ws16/ws32/ws128
5. hybrid: target-query summary + residual logits correction

目前最强结论是: ****time_only 长窗口没有带来足够有效的排序增益; residual 结构思路合理, 但当前实现和配置还没有超过 Stage1 C。****

用户 2026-07-07 14:14:31

Files mentioned by the user:

1、--context_method source_host \ --context_policy online \ --window_size 128 \...:

C:\Users\XiaoyanXiong\codex/attachments/555916b2-91df-4dce-8b72-4c78178a1f74/pasted-text.txt

My request for Codex:

这是修正context_method后产生的结果。详细解析

助手 2026-07-07 14:14:46

我先把这份“修正 context_method 后”的日志拆成几组实验, 确认每组到底是 `source\_host / destination\_host / endpoint`、window、模型、阈值和混淆矩阵。然后我会和之前的 `time\_only`、Stage1 C 对比, 看修正 context 是否真的带来了信息增益。

助手 2026-07-07 14:15:01

日志里三组都是 `target\_query\_gated + window\_size=128`，区别只在
`context\_method`：`source\_host`、`destination\_host`、`endpoint`。最重要的结论已经很清楚：****修正成
host/endpoint 相关上下文后，Stage2 开始真正超过 time_only 和 Stage1 C，尤其 source_host 最好。****

助手 2026-07-07 14:15:52

****核心结论****

这次修正 `context\_method` 后，Stage2 终于出现了真正的上下文增益。尤其是：

source_host + target_query_gated + window_size=128

这是目前最值得保留的 Stage2 结果。它在 `F1\_class1` 和 `PR\_AUC` 上都超过了之前的 `time\_only`，也超过了 Stage1 C 的 PR-AUC。

****三组结果总览****

	context	P1	R1	F1_1	PR_AUC	AUC	混淆矩阵
	---	---	---	---	---	---	---
source_host	**0.7945**	0.8074	**0.8009**	**0.8832**	**0.9903**		`[[80125,735],[678,2842]]`
destination_host	0.7690	**0.8332**	0.7998	0.8797	0.9895		`[[79979,881],[587,2933]]`
endpoint	0.7843	0.8139	0.7988	0.8776	0.9894		`[[80072,788],[655,2865]]`

排序：

```
PR_AUC: source_host > destination_host > endpoint
F1_1:   source_host > destination_host > endpoint
Recall: destination_host > endpoint > source_host
Precision: source_host > endpoint > destination_host
```

所以如果按综合质量看，`source\_host` 最好；如果你更重视少漏报，`destination\_host` 更激进。

****和之前 baseline 对比****

Stage1 C:

```
P1=0.8069
R1=0.7920
F1=0.7994
PR_AUC=0.8763
CM=[[80193,667],[732,2788]]
```

这次 source_host:

```
P1=0.7945
R1=0.8074
F1=0.8009
PR_AUC=0.8832
CM=[[80125,735],[678,2842]]
```

相比 Stage1 C:

```
PR_AUC +0.0069
F1_class1 +0.0015
54 : FN 732 -> 678
68 : FP 667 -> 735
```

这是很重要的，因为之前 `time\_only` 的提升基本只是 precision/recall 风格变化；这次 source_host 的 PR-AUC 明确上升，说明排序能力真的变强了。

和之前 `time\_only target\_query\_gated ws16`:

```
time_only:   F1=0.7971, PR_AUC=0.8754
```



```
source_host: F1=0.8009, PR_AUC=0.8832
```

这个差距已经不是纯阈值波动，更像是真上下文信息。

****为什么 source_host 最好****

source_host 的上下文诊断很有意思：

```
test label 0 history mean = 98.0
test label 1 history mean = 117.9
train label 0 history mean = 76.1
train label 1 history mean = 98.9
val label 0 history mean = 95.9
val label 1 history mean = 115.6
```

正类在 source_host 上明显有更多历史。这说明攻击/异常 flow 更可能来自活跃 source，或者同一 source 的行为历史更有辨识度。`target\_query\_gated` 正好是 “当前 flow 去查询同 source 历史”，所以它能利用这个结构。

这不是 future leakage，因为都是历史 flow；但它确实说明模型可能部分利用了 “source 历史活跃度” 这个信号。这个信号在 NIDS 里通常是合理的，但建议之后做一个 length-only/availability baseline 来确认不是单纯靠 history length 取巧。

****destination_host 的含义****

destination_host 结果：

```
P1=0.7690
R1=0.8332
F1=0.7998
PR_AUC=0.8797
CM=[[79979,881],[587,2933]]
```

它比 source_host 更偏 recall：

```
source_host    91 : 678 -> 587
               146 : 735 -> 881
```

destination_host 的诊断与 source 相反：正类 destination history 更少。

```
test label 0 mean = 90.8
test label 1 mean = 65.4
```

这可能表示攻击目标更分散，或者恶意 destination 不一定有稳定历史。模型仍然得到提升，但 precision 被拉低。

****endpoint 为什么不最好****

endpoint 几乎所有样本都有完整历史：

```
test mean = 125.9
val mean  = 125.6
```

但结果反而不如 source_host：

```
endpoint PR_AUC=0.8776
source   PR_AUC=0.8832
```

原因很可能是 endpoint 把 source 和 destination 两侧历史混在一起了。历史更多，但噪声也更多；target-query attention 虽然能筛选，但混合上下文仍然稀释了 source_host 的强信号。

所以 “更多上下文” 不等于 “更有效上下文”。这次结果很清楚地支持：

```
source_host
```

****阈值诊断****

三组 threshold transfer gap 都很小：

```
source_host gap = 0.0011
destination_host gap = 0.0010
endpoint gap = 0.0000
```

说明验证集选出的阈值迁移到测试集是稳定的。当前问题不是阈值选坏了，而是不同 context 的排序能力不同。

source_host 的 test oracle:

```
threshold=0.6237
F1_class1=0.8022
P1=0.7876
R1=0.8173
CM=[[80084,776],[643,2877]]
```

这说明 source_host 还有一点阈值提升空间，但不大。

****下一步建议****

优先保留并围绕 `source\_host + target\_query\_gated` 优化。

建议实验顺序:

1. source_host window_size = 16 / 32 / 64 / 128
2. source_host metric_for_best = f1_label1
3. source_host best_pr_auc best_f1 checkpoint
4. source_host + gate bias = -2
5. source_host + hybrid residual target-query

另外很值得做一个新模型：分别建 source 和 destination 两个 summary，而不是 endpoint 混合：

```
summary_src = attention(current, source_history)
summary_dst = attention(current, destination_history)
fused = [current, summary_src, summary_dst, current-summary_src, current-summary_dst]
```

这可能比 endpoint 更好，因为 endpoint 当前是“混合历史”，而结果显示 source/destination 的作用不同。

****最终判断****

修正 context 后，Stage2 不再是“没有用的上下文实验”。现在结论变成：

```
source_host
destination_host      recall
endpoint      source_host
      source_host      time_only
```

目前最推荐汇报的 Stage2 结果是：

```
source_host + target_query_gated + ws128
F1_class1 = 0.8009
PR_AUC    = 0.8832
AUC       = 0.9903
```

用户 2026-07-07 14:26:13

结合我给你的结果，给我生成 Full metrics 的所有模型的对比表格，包括：stage1C, source_host, destination_host, endpoint, timeOnly。让我可以直观的看所有 metric 的结果对比

助手 2026-07-07 14:26:26

我先把前面给过的日志指标补齐成同一套口径：能从 confusion matrix 精确反推的 accuracy、macro/weighted 指标我会统一计算，这样表格不会因为不同日志打印字段不一致而断层。

助手 2026-07-07 14:26:42

我找到了最新那份 `source\_host / destination\_host / endpoint` 的运行日志。接下来我会从日志里抓 “TEST SET” 的完整指标，再把缺失的宏平均/加权指标用同一个混淆矩阵公式补齐。

助手 2026-07-07 14:26:48

日志里的 `Full metrics` 行是完整的，我会直接抽取这些字典，避免手抄造成 1 个 TP/FN 的小误差。这里也能顺便确认：同一份日志里 source 的最终 Full metrics 是 `TP=2843, FN=677`，不是我之前粗读时的 `TP=2842, FN=678`。

助手 2026-07-07 14:27:10

为了让 `timeOnly` 和后面三个 context 方法公平对比，我会优先用同一个 `model\_type=target\_query\_gated, window\_size=128` 的 time_only 结果；否则会把 “换 context” 和 “换模型” 混在一起。

助手 2026-07-07 14:28:08

我会把表格分成两层：第一张是最直观的核心指标和混淆矩阵，第二张是完整的 macro/weighted/full metrics。宽表很挤，但这样你可以一眼看排名，同时又保留完整数值。

助手 2026-07-07 14:29:11

下面统一用 `\*\*TEST [Full metrics]\*\*` 口径；Stage2 的 `source\_host / destination\_host / endpoint / timeOnly` 都按 `\*\*model\_type=target\_query\_gated, window\_size=128, seed=130\*\*` 对比。不是 test oracle threshold。

所有行：`num\_classes=2`, `num\_samples=84380`, 正类数 `3520`, 正类比例 `0.041716`。

****核心对比****

Model / Context	Stage	Acc	P1	R1	F1_1	PR_AUC	AUC	TN	FP	FN	TP	Threshold
stage1C	Stage1	0.983420	0.806946	0.792045	0.799427	0.876256	0.989037	80193	667	732	2788	0.684000
source_host	Stage2	0.983254	0.794521	0.807386	0.800902	0.883189	0.990287	80125	735	678	2842	0.638586
destination_host	Stage2	0.982603	0.769009	0.833239	0.799836	0.879687	0.989453	79979	881	587	2933	0.589091
endpoint	Stage2	0.982899	0.784287	0.813920	0.798829	0.877569	0.989360	80072	788	655	2865	0.663333
timeOnly	Stage2	0.981939	0.753815	0.842045	0.795491	0.875063	0.988823	79892	968	556	2964	0.584141

****Full Metrics****

Model / Context	Macro P	Macro R	Macro F1	Weighted P	Weighted R	Weighted F1	F1_0	F1_1	Loss
stage1C	0.898951	0.891898	0.895390	0.983278	0.983420	0.983346	0.991353	0.799427	0.068210
source_host	0.893065	0.899148	0.896081	0.983387	0.983254	0.983319	0.991260	0.800902	0.030160

```
| **destination_host** | 0.880861 | 0.911172 | 0.895371 | 0.983382 | 0.982603 | 0.982935 | 0.990906 |  
0.799836 | 0.030907 |  
| **endpoint** | 0.888087 | 0.902088 | 0.894949 | 0.983226 | 0.982899 | 0.983050 | 0.991070 | 0.798829 |  
0.031747 |  
| **timeOnly** | 0.873452 | **0.915037** | 0.893022 | 0.983107 | 0.981939 | 0.982415 | 0.990552 | 0.795491 |  
0.031644 |
```

****相对 Stage1C 的变化****

```
| Model / Context |  $\Delta F1\_1$  |  $\Delta PR\_AUC$  |  $\Delta P1$  |  $\Delta R1$  |  $\Delta FP$  |  $\Delta FN$  |  
|---|---:|---:|---:|---:|---:|  
| **source_host** | **+0.001475** | **+0.006933** | -0.012426 | +0.015341 | +68 | -54 |  
| **destination_host** | +0.000410 | +0.003431 | -0.037938 | +0.041193 | +214 | -145 |  
| **endpoint** | -0.000598 | +0.001313 | -0.022660 | +0.021875 | +121 | -77 |  
| **timeOnly** | -0.003935 | -0.001193 | -0.053132 | +0.050000 | +301 | -176 |
```

****直观结论****

`source\_host` 是目前综合最好: `F1\_1=0.800902`、`PR\_AUC=0.883189` 都是最高, 而且相对 Stage1C 少了 `54` 个 FN, 只多了 `68` 个 FP。

`destination\_host` 召回更激进, FN 最少之一, 但 FP 明显增加, 所以 precision 被拉低。`endpoint` 历史最多, 但混入噪声后没有超过 `source\_host`。`timeOnly` 召回最高, 但 FP 太多, 整体 F1 和 PR_AUC 都不如修正后的 host-context。Stage1C 仍然是 precision/FP 最稳的强基线。

用户 2026-07-07 14:33:07

Files mentioned by the user:

```
## [INFO] Stage2 ----- seed: 130 [INFO] 随机种子已设置: 130 [INFO] Stage2 ----- d...:  
C:\Users\XiaoyanXiong\codex/attachments/1f1ef92c-c2a5-43aa-a242-88d13e43fb10/pasted-text.txt
```

My request for Codex:

解决这里的nan问题, 该如何修正代码? 详细解析

助手 2026-07-07 14:33:20

我先对照你这份最新日志和本地 `s2` 代码定位 NaN 出现的位置。根据前面的症状我怀疑是 residual transformer 在 “没有历史 token” 的样本上把 attention 全 mask 掉了, 但我会先用日志和源码确认一下。

助手 2026-07-07 14:33:32

日志确认: NaN 不是训练一开始爆炸, 而是在保存 train predictions/计算 train AUC 时出现; 模型是 `residual\_transformer`, `context\_method=time\_only`。我现在看 residual 的 forward, 重点检查 history mask 如何传给 `TransformerEncoder`。

助手 2026-07-07 14:33:45

源码里问题很清楚了: `history\_mask` 会把 “当前 flow” 去掉; 对第一个样本或没有历史的样本, `history\_mask` 全 False, 然后 `key\_padding\_mask = ~history\_mask` 就变成全 True。PyTorch 的 self-attention 在某一行所有 key 都被 mask 时, softmax 会得到 NaN。接下来我会在 residual

模型里加一个安全分支，让无历史样本走纯 current-flow baseline。

助手 2026-07-07 14:34:26

修正点不只是“让 encoder 不 NaN”，还要保证无历史样本不被 residual 分支偷偷改 logits。也就是说：无历史时可以给 attention 一个临时安全 key 来避免 softmax 全 mask，但最终 residual correction 的 gate 要置 0，让输出严格回退到 `base\_logits(current\_z)`。

助手 2026-07-07 14:34:31

我现在直接改 `s2/stage2/model.py` 里的 `Stage2ResidualContextTransformer.forward`，范围很小：只加 `safe\_history\_mask/no\_history` 和无历史 gate 清零。

助手 2026-07-07 14:34:53

补丁已经加上。接下来我做两个验证：先确认源码能编译，再用一个包含“只有当前 token、无历史”的 batch 做 forward，检查输出是否 finite。

助手 2026-07-07 14:35:04

编译验证被 Windows 沙箱拦住了：`python.exe Access is denied`，这不是代码语法错误反馈。补丁位置我已经复核到 735-790 行，接下来我用已批准的命令方式再尝试一次；如果仍被拦，我会把验证状态明确告诉你。

助手 2026-07-07 14:35:18

`py\_compile` 已通过。现在做一个更贴近问题的 forward smoke test：batch 里混合“无历史”和“有历史”的样本，确认 residual transformer 输出不再含 NaN。

助手 2026-07-07 14:35:35

刚才 smoke test 的 Python 字符串被 PowerShell 引号吃坏了，补丁本身没跑。这个是命令包装问题，我用单行表达式重跑一次。

助手 2026-07-07 14:36:00

smoke test 通过：输出 shape 是 `(4, 2)`，finite=True。再做一个更严格的检查：无历史样本的最终 logits 是否等于 `base\_head(current\_z)`，也就是 residual 分支确实没有介入。

助手 2026-07-07 14:36:16

严格检查也通过：无历史样本输出和 `base\_head(current\_z)` 的最大差值是 `0.0`。我再看一下 diff，确保只改了 residual transformer 这一处。

助手 2026-07-07 14:36:41

已修正。问题在
[s2/stage2/model.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/model.py:743) 的
`Stage2ResidualContextTransformer.forward()`。

****NaN 原因****

`residual\_transformer` 会先把当前 flow token 从 `mask` 里移除，只保留历史 token：

```
history_mask = self._history_mask_without_current(mask)
key_padding_mask = ~history_mask
h = self.encoder(x_ctx, src_key_padding_mask=key_padding_mask)
```

但如果某条样本没有历史，例如 context 只有当前 flow，一个 token 被移除后 `history\_mask` 就全是 `False`。这会导致 `TransformerEncoder` 这一行的所有 key 都被 mask 掉。attention softmax 面对全 `-inf` 时会产生 `NaN`，于是后面 `context\_summary` -> `delta\_logits` -> `logits` -> FocalLoss/AUC 全部被污染。

所以日志里的：

```
[WARNING] NaN detected in loss!
[WARNING] AUC calculation failed: Input contains NaN.
```

不是 loss function 本身坏了，而是模型 forward 已经输出了 NaN。

****我做的修正****

在无历史样本上：

1. 用 `safe\_history\_mask` 临时保留当前 token 作为 attention 的安全 key，避免 encoder 全 mask。
2. encoder 之后把这些样本的 `context\_summary` 清零。
3. 最关键：把这些样本的 `gate` 清零，使最终输出严格退回到：

```
logits = base_logits
```

也就是无历史时不使用 residual context correction。

核心修改逻辑：

```
history_mask = self._history_mask_without_current(mask)
no_history = ~history_mask.any(dim=1)

safe_history_mask = history_mask
if has_no_history:
    safe_history_mask = history_mask.clone()
    safe_history_mask[no_history, current_idx[no_history]] = True

h = self.encoder(x_ctx, src_key_padding_mask=~safe_history_mask)

context_summary = masked_mean_pooling(h, safe_history_mask)
context_summary[no_history] = 0

delta_logits = self.delta_head(fused)
gate = self.gate(fused)
gate[no_history] = 0

logits = base_logits + self.context_scale * gate * delta_logits
```

****验证结果****

我做了两个本地检查：

```
py_compile passed
forward smoke test: output shape (4, 2), finite=True
```

还验证了无历史样本：

```
max(abs(output - base_head(current))) = 0.0
```

说明无历史样本现在不会 NaN，并且会严格回退到 current-flow baseline。

补充：日志里的 `enable\_nested\_tensor ... norm\_first was True` 只是 PyTorch 性能提示，不是 NaN 的原因。你需要重新跑一次 `residual\_transformer` 实验或至少重新生成 predictions；旧的 `stage2\_predictions\_train.csv` 里已经保存过 NaN，不会自动修复。

用户 2026-07-07 14:42:10

如何修正代码才能让stage2的结果变的更好？给我提供尽可能多的修正方案。改模型、改attention、任何可以修改的都要尝试。详细解析

助手 2026-07-07 14:42:27

我会先把 Stage2 的训练、模型、context、loss 代码再过一遍，然后给你一组“可以直接落地实验”的修改方案。这里重点不是随便堆复杂度，而是围绕你已有结果：`source\_host + target\_query\_gated` 已经有提升，下一步要尽量保留 Stage1C 的 precision，同时继续吃到 source 历史带来的 recall/PR_AUC。

助手 2026-07-07 14:42:44

我看到你的当前 best 配置本质上是：`source\_host + target\_query\_gated + focal(gamma=1) + sampler` 正类比例 0.08 + best by PR_AUC。从结果看，Stage2 已经主要赢在 PR_AUC 和 recall，输在 precision/FP；所以代码优化要分两类：一类提升排序能力，另一类把 FP 压回来。

助手 2026-07-07 14:42:55

有两个明显的代码层面机会：第一，`evaluate()` 最终阶段默认没开 AMP，而训练时/threshold diagnosis 有时开 AMP，会让阈值选择和最终 metrics 有轻微不一致；第二，当前模型只支持单一路径 context，不能同时建 source/destination 两套历史。模型层面更大的机会是“target-query 注意力 + residual logit correction”的混合模型。

助手 2026-07-07 14:43:20

我会直接在代码里落地几类“低破坏、高价值”的实验入口：1) target-query + residual logit correction 混合模型；2) 更偏向压 FP 的 asymmetric focal loss；3) 最终评估 AMP 口径一致。更大的数据结构改造，比如 source/destination 双路 context，我会给清楚方案和代码方向，但不贸然把 Dataset 改成多输入，避免影响你现有所有实验。

助手 2026-07-07 14:43:53

下面我会做三个代码补丁。补丁会保持向后兼容：你原来的 `target\_query\_gated`、`residual\_transformer` 还能照常跑；新增模型/损失只有在 config 或命令行指定时才启用。

助手 2026-07-07 14:44:00

我先新增一个最值得跑的模型：`target\_query\_residual`。它的核心思想是：保留 Stage1/current-flow 的强 baseline，只让 context 预测一个小的 `delta\_logits`，并用 gate 控制修正幅度。这个设计是为你当前“recall 提了但 FP 增多”的现象准备的。

助手 2026-07-07 14:44:37

新模型已经加好。现在把它接到 `build\_stage2\_model` 和命令行 `--model\_type`，这样你能直接用现有 `run\_stage2.py` 跑，不需要改训练脚本结构。

助手 2026-07-07 14:44:50

刚才 patch 因为本地 factory 参数写法是 ``input_dim=input_dim``，上下文没对上。我按实际代码再补一次。

助手 2026-07-07 14:44:59

接下来更新两个 runner 的 choices。之后你可以直接加 ``--model_type target_query_residual`` 开跑。

助手 2026-07-07 14:45:13

现在加一个可选的新

loss: ``asymmetric_focal``。它的作用是更集中惩罚“高置信度误报”的负类样本，适合你现在 FP 偏多、precision 比 Stage1C 低的问题。原来的 focal loss 不受影响。

助手 2026-07-07 14:45:32

loss 类加好了。现在把 trainer 的 ``loss_type`` 分支接上，并顺手修正 AMP 配置，让训练/阈值选择/最终评估使用同一个 ``training.amp`` 开关。

助手 2026-07-07 14:45:46

我继续把 AMP 一致性补上。这个不会让模型“真实能力”突增，但会减少你看到的 val threshold、test metrics、oracle diagnosis 之间那些 1 个样本级别的漂移，方便判断实验。

助手 2026-07-07 14:45:55

训练阶段已经统一了 AMP。我再把 ``threshold_oracle_diagnosis()`` 和最终保存 predictions 的评估也接到同一个开关。

助手 2026-07-07 14:46:05

接下来把新参数写进默认 config，主要是为了 ``stage2_config_used.yaml`` 能记录完整实验设置，后续复现实验不靠记忆。

助手 2026-07-07 14:46:24

为了让你在 Colab 里不必反复手改 YAML，我再给 runner 加几个常用命令行 override: ``context_scale``、``gate_bias_init``、``loss_type``、``focal_gamma``、``sampler_pos_fraction`` 等。这样你可以用同一个 config 快速扫实验。

助手 2026-07-07 14:46:30

我加的 CLI override 不会改变默认行为；只有你在命令里写了对应参数才会覆盖 YAML。

助手 2026-07-07 14:47:16

我还补一个 ``--asym_gamma_pos``，不然 asymmetric loss 的正类 focal 参数只能改 YAML。然后跑编译和 forward/loss smoke test。

助手 2026-07-07 14:47:32

现在做验证：编译 `model/target\_query/trainer/losses/runner`，再用新模型和新 loss 做最小 forward。

助手 2026-07-07 14:47:47

编译通过。接下来做新模型和 asymmetric loss 的 smoke test，确认 batch 混合有历史/无历史样本时输出 finite，loss 也 finite。

助手 2026-07-07 14:48:21

smoke test 也通过了。新模型输出 `(4, 2)` 且没有 NaN，`asymmetric\_focal` loss 是有限值。最后我会整理一份“优先级实验矩阵”，把已经写进代码的方案和还值得进一步改的数据/attention 方案分开。

助手 2026-07-07 14:48:32

还有一个很实用的修正：你现在阈值选择默认最大化 `f1\_label1`，它不偏 precision。因为 Stage2 当前主要问题是 FP 增多，我加 `threshold\_metric=f0.5`，用于选择更保守的阈值；这不改模型，但常常能把 precision 拉回来。

助手 2026-07-07 14:48:59

`f0.5/f2` 阈值选择也加好了：`f0.5` 偏 precision，`f2` 偏 recall。现在再编译一次确认没有语法问题。

助手 2026-07-07 14:49:18

代码补丁完成并通过编译。最后我会给你一个详细的方案清单：哪些已经写进代码、为什么可能提升、怎么跑、以及后续更大改造方向。

助手 2026-07-07 14:50:15

我已经在你现有 `s2` 代码上加了几类可直接跑的改革，不影响原模型。核心目标是：**保留 `source\_host + target\_query\_gated` 带来的 PR_AUC/recall 提升，同时把 FP 压下来，让 precision 接近或超过 Stage1C。**

已修改代码

- 新模型：`target\_query\_residual`

在 [target_query.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/target_query.py:174)。公式是：

```
logits = base_logits(current) + context_scale * gate * delta_logits(context)
```

它比 `target\_query\_gated` 更保守。`target\_query\_gated` 是直接改 representation，容易提高 recall 但增加 FP；新模型只让 context 修正 logits，并且默认 `gate\_bias\_init=-2.0`、`context\_scale=0.2`、`zero\_init\_delta=True`，一开始几乎等价于 current-flow baseline，然后逐步学习 context correction。

- 新 loss：`asymmetric\_focal`

在 [losses.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/losses.py:116)。用来更集中惩罚高置信度 false positive，适合你现在 Stage2 precision 低于 Stage1C 的问题。

- 新阈值指标：`f0.5` / `f2`

在 [trainer.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/trainer.py:310)。`f0.5` 偏 precision，`f2` 偏 recall。你现在建议优先试 `f0.5`。

- AMP 评估口径一致

在 [trainer.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/trainer.py:388)。

训练、val threshold、oracle diagnosis、final metrics 现在使用同一个 `training.amp` 开关，减少 metrics 轻微漂移。

- CLI 支持新增参数

在 [run_stage2.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/run_stage2.py:92) 和 `run\_stage2\_tuned.py`。

现在可以直接传：

```
`--model_type target_query_residual`  
`--context_scale`  
`--gate_bias_init`  
`--use_context_length_feature`  
`--loss_type asymmetric_focal`  
`--threshold_metric f0.5`
```

等。

我也做了验证：`py\_compile` 通过；新模型 forward 输出 finite；`asymmetric\_focal` loss finite。

****最推荐先跑的实验****

1. ****最高优先级：source_host + target_query_residual****

```
!python /content/drive/MyDrive/s2/run_stage2.py \  
  --stage1_dir /content/drive/MyDrive/s1/0704/0704C_ar002_et12_20260511_001 \  
  --config /content/drive/MyDrive/s2/0707ar002_et12_20260511_001/src_newM/stage2_config_used.yaml \  
  --out_dir /content/drive/MyDrive/s2/0707ar002_et12_20260511_001/src_tq_residual/ws128_s02 \  
  --context_method source_host \  
  --context_policy online \  
  --window_size 128 \  
  --model_type target_query_residual \  
  --context_scale 0.2 \  
  --gate_bias_init -2.0 \  
  --seed 130 \  
  --epochs 500 \  
  --batch_size 128
```

2. ****如果 precision 还是低：用 f0.5 选阈值****

```
--threshold_metric f0.5 \  
--metric_for_best fbeta_0.5_label1
```

这会牺牲一点 recall，但通常能减少 FP。你当前 `source\_host` 相对 Stage1C 是 `FP +68, FN -54`，所以这个方向很合理。

3. ****扫 context_scale****

```
context_scale = 0.1, 0.2, 0.3, 0.5
```

建议顺序：`0.2 -> 0.1 -> 0.3 -> 0.5`。

如果 `0.5` FP 上升，说明 context correction 太强；如果 `0.1` 没提升，说明 context 被压得太弱。

4. ****扫 gate_bias_init****

```
gate_bias_init = -3.0, -2.0, -1.0, 0.0
```

越负越保守。你现在 Stage2 precision 不如 Stage1C，所以先试 `-2.0`、`-3.0`。

5. ****source_host 窗口大小重新扫****

```
window_size = 16, 32, 64, 128
```

之前 time_only 的 window 差异不大，但 source_host 是有效 context，值得重新扫。我的预期：`32/64` 可能比 `128` 更稳，因为 source_host 长历史虽然有用，但也可能带来旧行为噪声。

****loss / sampler 方向****

你当前配置：

```
use_weighted_sampler: true
sampler_pos_fraction: 0.08
class_weighted_loss: false
loss_type: focal
alpha: [1.0, 1.05]
focal_gamma: 1.0
```

可以试：

| 目标 | 修改 |

|---|---|

| 压 FP / 提 precision | `--sampler\_pos\_fraction 0.05` 或 `0.06` |

| 保持当前平衡 | `0.08` |

| 拉 recall | `0.10` 或 `0.12` |

| 压高置信误报 | `--loss\_type asymmetric\_focal --asym\_gamma\_neg 2.0 --asym\_neg\_weight 1.1` |

| 更强压 FP | `--asym\_gamma\_neg 3.0 --asym\_neg\_weight 1.2` |

| 避免 recall 掉太多 | `--asym\_pos\_weight 1.05` 或 `1.1` |

推荐组合：

```
--model_type target_query_residual \
--context_scale 0.2 \
--gate_bias_init -2.0 \
--loss_type asymmetric_focal \
--asym_gamma_neg 2.0 \
--asym_neg_weight 1.1 \
--asym_pos_weight 1.05 \
--sampler_pos_fraction 0.06 \
--threshold_metric f0.5 \
--metric_for_best fbeta_0.5_label1
```

****还值得做但我没有贸然改 Dataset 的大方案****

1. ****source/destination 双路 attention****

比 `endpoint` 更合理。`endpoint` 把 source 和 destination 混在一起，结果反而弱；说明两类上下文语义不同。理想模型是：

```
summary_src = attention(current, source_history)
summary_dst = attention(current, destination_history)
logits = base + gated_delta(current, summary_src, summary_dst)
```

这需要 Dataset 同时返回两套 context，改动较大，但提升机会真实存在。

2. ****Stage1 + Stage2 概率融合****

Stage1C precision 强，Stage2 source_host PR_AUC 强。可以在 val 上扫：

```
p_final = λ * p_stage1 + (1 - λ) * p_stage2
λ = 0.1 ... 0.9
```

这非常值得做，可能比继续堆模型更稳。

3. ****Hard negative mining****

先跑 source_host，收集 val/train 中 Stage2 的 FP，再在第二轮训练中提高这些 negative 的采样权重。目标非常明确：压误报。

4. ****context length 诊断****

我已经给新模型加了 `--use\_context\_length\_feature`。它可能提升，也可能学到“历史长度捷径”。建议只作为 ablation 跑：

```
--use_context_length_feature
```

如果提升明显，要再做 length-only baseline 确认不是脆弱 shortcut。

****我的优先级排序****

1. `source\_host + target\_query\_residual + context\_scale/gate\_bias`
2. `threshold\_metric=f0.5` 压 FP
3. `sampler\_pos\_fraction=0.05/0.06/0.08`
4. `asymmetric\_focal`
5. source_host 的 `window\_size=16/32/64/128`
6. Stage1 + Stage2 概率融合
7. source/destination 双路 attention 数据结构改造

最可能超过你当前最好结果的，不是更大的 Transformer，而是 ****保守 residual context correction + 更合适的阈值/采样策略****。

用户 2026-07-07 15:46:20

如何修正代码才能让stage2的结果变的更好？给我提供尽可能多的修正方案。改模型、改attention、任何可以修改的都要尝试。详细解析

助手 2026-07-07 15:46:40

我会先把现有 Stage2 的模型、loss、trainer、配置入口再扫一遍，然后给你一个“能直接落地”的改进清单。这里我会把建议分成：最有希望立刻提升的、需要小实验验证的、风险较高但值得尝试的。

助手 2026-07-07 15:46:54

我看到你本地其实已经多了一个 `Stage2TargetQueryResidualAttention` 类，但 factory 现在还不一定接上了。这个方向正是“保留 Stage1 精度 + 让 context 只做修正”的高优先级方案，我先读一下它的实现再决定是否需要补齐入口或修正细节。

助手 2026-07-07 15:47:07

代码里已经具备一个“target-query + residual logit correction”的雏形，但它还没有在 `build\_stage2\_model()` / CLI `choices` 里注册。这个可以直接变成第一个可跑实验。我还会检查 trainer 里是否有影响结果可比性的点，比如 AMP 评估口径和 sampler/loss 叠加。

助手 2026-07-07 15:47:18

下一步我会做一个小的代码补齐：把已有的 `target\_query\_residual` 注册到模型工厂和命令行参数里，这样你可以直接跑。然后我会给出一组实验矩阵，包含模型、attention、loss、sampler、threshold、context 结构这些层面的修改。

助手 2026-07-07 15:47:25

目前最小但很有价值的代码修正是“让实验入口能跑已有的新模型”。我会先补这个，不动训练主流程；其它更激进方案我在最终回答里给你实验顺序和具体配置。

助手 2026-07-07 15:47:36

模型工厂已经接上 `target\_query\_residual`。接下来把两个运行脚本的 `--model\_type` choices 也补上，避免命令行直接报 “invalid choice”。

助手 2026-07-07 15:47:43

`run\_stage2.py` 已补齐。现在同步补 `run\_stage2\_tuned.py`，保持两个入口一致。

助手 2026-07-07 15:48:01

入口补完后，我做编译和 forward 检查，确认新增 model_type 能真正实例化并处理无历史样本。

助手 2026-07-07 15:48:14

编译通过。接着做一次 `build\_stage2\_model` 的 smoke test，验证新注册的 `target\_query\_residual` 在混合历史长度 batch 上输出 finite。

助手 2026-07-07 15:48:48

我还想补一个小便利：把 `context\_scale / gate\_bias\_init / use\_context\_length\_feature` 做成命令行可覆盖。否则你每试一次 residual gate 都要手动改 YAML，实验会很慢。

助手 2026-07-07 15:48:56

我会在两个入口里加 3 个参数：`--context\_scale`、`--gate\_bias\_init`、`--use\_context\_length\_feature`。这不会改变默认行为，但能让你直接从命令行扫参数。

助手 2026-07-07 15:49:01

现在我把这些 override 写进去，默认值都是 `None`，所以不会影响你旧实验复现。

助手 2026-07-07 15:49:12

`run\_stage2.py` 已加参数。同步到 tuned 入口。

助手 2026-07-07 15:49:24

最后再编译一次，确认这几个 CLI 参数没有语法问题。

助手 2026-07-07 15:49:59

还有一个关键判断：Stage2 现在的 `base\_head(current)` 是重新训练的 current-flow head，并不等于 Stage1C 已经训练好的分类头。所以如果你想把 Stage1C 的高 precision 真正带进 Stage2，后续最值得做的是把 Stage1 的 logit/prob 一起保存进 Stage2，作为 base logit 或 extra feature。这个我会放在最高优先级方案里。

助手 2026-07-07 15:50:33

我先说结论: **Stage2 现在最有提升空间的方向不是盲目加大 Transformer, 而是 “保留 Stage1C 的强 current-flow 判断, 再让 context 做小幅、可控修正” 。** 你的结果已经说明: `source\_host` context 有真实信号, `timeOnly` 太噪, `target\_query\_gated` 容易提高 recall 但损 precision。

我已经顺手补了一个可直接跑的代码入口:

```
-  
[s2/stage2/model.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/model.py:808): 注册  
`target_query_residual`  
- [s2/run_stage2.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/run_stage2.py:92): CLI 支持  
`--model_type target_query_residual`  
- [s2/run_stage2.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/run_stage2.py:127): 新增  
`--context_scale`、`--gate_bias_init`、`--use_context_length_feature`  
- [s2/run_stage2_tuned.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/run_stage2_tuned.py:92  
) : 同步支持
```

已验证: `py\_compile` 通过, `target\_query\_residual` forward 输出 finite=True。

****第一优先级****

先跑这个:

```
!python /content/drive/MyDrive/s2/run_stage2.py \  
  --stage1_dir /content/drive/MyDrive/s1/0704/0704C_ar002_et12_20260511_001 \  
  --config /content/drive/MyDrive/s2/0707ar002_et12_20260511_001/src_newM/stage2_config_used.yaml \  
  --out_dir /content/drive/MyDrive/s2/0707ar002_et12_20260511_001/src_target_query_residual/ws128_s02_g-2_len  
 \  
  --context_method source_host \  
  --context_policy online \  
  --window_size 128 \  
  --model_type target_query_residual \  
  --context_scale 0.2 \  
  --gate_bias_init -2.0 \  
  --use_context_length_feature \  
  --seed 130 \  
  --epochs 500 \  
  --batch_size 128
```

为什么它最值得试:

`target\_query\_gated` 是 representation-level 强融合, 容易把 context 噪声注入 current embedding; `target\_query\_residual` 是 logit-level correction:

```
final_logits = base_logits(current) + context_scale * gate * delta_logits(context)
```

也就是说它更保守, 更适合你现在这种 “Stage1C precision 很强, Stage2 source_host 只需要补一点 recall/PR_AUC” 的情况。

****推荐实验矩阵****

	优先级	修改	建议值	目的
1	1	`context_method`	`source_host`	目前最强, PR_AUC/F1 都最好
1	1	`model_type`	`target_query_residual`	保 precision, 减少 FP
1	1	`context_scale`	`0.1, 0.2, 0.3, 0.5`	控制 context 修正强度
1	1	`gate_bias_init`	`-3.0, -2.0, -1.0`	初始 gate 小一点, 避免过早信 context
1	1	`window_size`	`16, 32, 64, 128`	找 source_host 最佳历史长度
2	2	`use_context_length_feature`	on/off	检查 “历史长度” 是否本身有预测力
2	2	`metric_for_best`	`pr_auc` vs `f1_label1`	PR_AUC 更稳, F1 更贴最终阈值

2	`threshold\_steps`	`501` 或 `1001`	阈值更细，减少 transfer gap
2	`sampler\_pos\_fraction`	`0.06, 0.08, 0.10, 0.12`	控制 recall/precision
2	`focal\_gamma`	`0.5, 1.0, 1.5, 2.0`	调整难样本关注强度
3	`destination\_host`	residual 模型再跑	召回高，但 FP 偏多，适合找 recall 上限
3	`endpoint`	不建议优先	历史多但噪声更大
3	`timeOnly`	不建议主线	recall 高但 precision 损失太大

****模型层面还能改什么****

1. ****Stage1-logit residual****

最高潜力。现在 Stage2 的 `base\_head(current)` 是重新训练的，不等于 Stage1C 的原始分类头。最好把 Stage1C 的 `prob/logit\_label1` 保存进 Stage2，然后：

```
final_logit = stage1_logit + context_scale * gate * delta_logit
```

这会真正继承 Stage1C 的高 precision，而不是重新学一个近似 current-flow head。

2. ****source/destination 双路 attention****

不要把 `endpoint` 混在一起。更合理是：

```
summary_src = attention(current, source_history)
summary_dst = attention(current, destination_history)
fused = [current, summary_src, summary_dst, current-summary_src, current-summary_dst]
```

你的结果里 `source\_host` 和 `destination\_host` 行为不同：source 更均衡，destination 更召回。双路比 endpoint 混合更可解释，也更可能提升。

3. ****recency-biased attention****

历史越近越重要，可以给 attention logits 加 age bias：

```
attn_score += -lambda * age
```

或者把 `age / delta\_time` 作为额外 feature。现在 sinusoidal position 只告诉模型顺序，不直接表达“越近越可信”。

4. ****context dropout****

训练时随机丢一部分历史 token，防止模型过度依赖 context 长度或少数噪声历史：

```
history_mask = history_mask & (torch.rand_like(history_mask.float()) > p)
```

建议 `p=0.1/0.2`。

5. ****top-k attention / sparse attention****

只取 attention 权重最高的前 k 个历史 summary，减少 endpoint/timeOnly 噪声。适合 `window\_size=128` 时尝试。

****Loss 和训练建议****

我不建议第一步就大改 loss。你的 PR_AUC 差异主要来自 context 质量和融合方式，不是 loss 崩了。可以按顺序试：

```
Focal gamma: 1.0 -> 0.5 -> 1.5 -> 2.0
label_smoothing: 0.0 -> 0.02 -> 0.05
sampler_pos_fraction: 0.08 -> 0.06/0.10/0.12
```

注意不要同时太激进：`WeightedRandomSampler` + 高 alpha focal + 高 gamma 会把模型推向高 recall/高 FP，这正是 `timeOnly` 和部分 Stage2 的问题。

****我建议的跑法****

先只跑 8 个最关键组合：


```
source_host + target_query_residual + ws16 + scale 0.2
source_host + target_query_residual + ws32 + scale 0.2
source_host + target_query_residual + ws64 + scale 0.2
source_host + target_query_residual + ws128 + scale 0.2
source_host + target_query_residual + ws128 + scale 0.1
source_host + target_query_residual + ws128 + scale 0.3
source_host + target_query_residual + ws128 + gate_bias -3.0
destination_host + target_query_residual + ws128 + scale 0.2
```

判断标准：如果 `source\_host + target\_query\_residual` 能把 `FP` 从 735 往 Stage1C 的 667 拉近，同时保持 `FN < 732`，那就是实质提升。当前最好的目标不是单纯提高 recall，而是把 `source\_host` 的 `PR\_AUC=0.8832` 转换成更好的实际阈值点。

用户 2026-07-07 16:24:31

source/destination 双路 attention

不要把 endpoint 混在一起。更合理是：

```
summary_src = attention(current, source_history)
```

```
summary_dst = attention(current, destination_history)
```

```
fused = [current, summary_src, summary_dst, current-summary_src, current-summary_dst]
```

你的结果里 source_host 和 destination_host 行为不同：source 更均衡，destination 更召回。双路比 endpoint 混合更可解释，也更可能提升。

该如何实现这个功能？详细解析

助手 2026-07-07 16:24:50

我先把现有 Stage2 的数据流重新对齐一下：现在 Dataset/Trainer 都假设只有一条 `context\_z + mask`。双路 attention 要么改 Dataset/Trainer 支持两条历史，要么把 source/destination 编码到同一条序列里。我会优先做“结构清楚、改动可控”的双输入方案。

助手 2026-07-07 16:25:02

数据入口确认了：`run\_stage2.py` 固定调用 `ContextIndexBuilder`，`Stage2Dataset` 返回 `context\_z/mask`，trainer 固定 `model(x, mask)`。所以实现双路有两种路线：小改动是让 batch 额外带 `source\_context\_z/destination\_context\_z`，trainer 检测到就调用新模型；大改动是重写整套 Dataset。我会采用小改动。

助手 2026-07-07 16:25:58

实现上我会加入一个新的

`context\_method=source\_destination`，它不是把两类历史合并，而是为每个样本生成两个数组：`source` 历史和 `destination` 历史。对应新模型 `source\_destination\_attention` 会分别对两路做 target-query attention，再用 gated residual 修正 logits。

助手 2026-07-07 16:26:13

我先改 context builder：新增 `source\_destination` 方法。它会独立维护 `source\_hist[src]` 和 `dest\_hist[dst]`，每个样本返回 `{"source": ..., "destination": ...}`，两路最后都 append 当前 flow，避免 self/future leakage。

助手 2026-07-07 16:26:38

接下来改 Dataset/collate。旧单路样本保持原样；当 context entry 是 dict 时，batch 会多出 `source\_context\_z/source\_mask/destination\_context\_z/destination\_mask` 四个字段。

助手 2026-07-07 16:27:06

现在 trainer 要认识双路 batch。实现方式是加一个小 helper：如果 batch 有 `source\_context\_z`，就调用 `model(src, src\_mask, dst, dst\_mask)`；否则保持旧的 `model(x, mask)`。

助手 2026-07-07 16:27:23

还要让最终 `stage2\_metrics.json` 的 context length 统计不被 dict 弄坏。我会把单路统计和双路统计都封装成同一个 helper。

助手 2026-07-07 16:27:53

现在新增模型类。它是“双路 target-query + gated residual”结构：source/destination 分别 attention，summary 不混合；最后用一个小 gate 控制 context 对 logits 的修正幅度。

助手 2026-07-07 16:28:40

模型类已加。现在注册 `model\_type=source\_destination\_attention`，并把 CLI 的 context/model choices 放开。

助手 2026-07-07 16:28:50

模型工厂已注册。接下来补 `run\_stage2.py` 和 `run\_stage2\_tuned.py` 的命令行 choices：`--context\_method source\_destination` 和 `--model\_type source\_destination\_attention`。

助手 2026-07-07 16:28:57

`run\_stage2.py` 已更新。同步到 tuned 入口。

助手 2026-07-07 16:29:03

再补一个小注释，把默认配置里的 context method 注释更新，避免以后看 YAML 时忘了新方法。

助手 2026-07-07 16:29:15

还要修日志统计：双路 context 不是一个 array，旧的 `len(ctx)` 会误读成 dict key 数。现在我让 summary 分别打印 source/destination 的长度分布和无历史比例。

助手 2026-07-07 16:29:43

`run\_stage2.py` 的诊断已适配。现在给 tuned 版本也补同样的 data summary，保持入口一致。

助手 2026-07-07 16:29:55

双路 batch、trainer、日志适配都完成了。下一步我做注册/编译和一个最小 end-to-end smoke test：builder -> dataset -> collate -> model forward。

助手 2026-07-07 16:30:00

我现在运行编译检查；如果通过，再跑一个小样本 smoke test。

助手 2026-07-07 16:30:18

编译通过。现在做一个小型端到端测试：6 条假 flow，构造 source/destination 历史，collate 成双路 batch，再喂给 `source\_destination\_attention`。

助手 2026-07-07 16:30:50

端到端 smoke test 通过：双路 context builder 输出 dict，collate 后两路张量都是 `(6, 4, 128)`，新模型输出 `(6, 2)` 且 finite=True。现在我收尾整理具体实现、使用命令和为什么这样设计。

助手 2026-07-07 16:31:02

我再取一下关键代码位置，方便你在本地打开核对。

助手 2026-07-07 16:31:35

已实现 `source/destination 双路 attention`，并且保持旧的单路 Stage2 流程不受影响。

****核心思路****

新增：

```
context_method = source_destination
model_type = source_destination_attention
```

每条 flow 会生成两路 context：

```
source_context      = previous flows with same source_id + current
destination_context = previous flows with same destination_id + current
```

模型内部分别做：

```
summary_src = attention(current, source_history)
summary_dst = attention(current, destination_history)
```

```
fused = [
    current,
    summary_src,
    summary_dst,
    current - summary_src,
    current - summary_dst,
    length features
]
```

```
logits = base_logits(current) + context_scale * gate * delta_logits(fused)
```

也就是说：****source 和 destination 不再像 endpoint 那样混在一起****，而是分别建模，再由 gated residual 控制 context 修正强度。

****已修改的代码位置****

- 双路 context 构造：

[s2/stage2/context.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/context.py:9)

- Dataset / collate 支持双路 batch:

[s2/stage2/dataset.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/dataset.py:42)

- Trainer 自动识别双路 batch:

[s2/stage2/trainer.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/trainer.py:135)

- 新模型 `Stage2SourceDestinationAttention`:

[s2/stage2/target_query.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/target_query.py:347)

- 模型工厂注册:

[s2/stage2/model.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/model.py:808)

- CLI 支持:

[s2/run_stage2.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/run_stage2.py:39)

[s2/run_stage2_tuned.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/run_stage2_tuned.py:39)

****推荐运行命令****

```
!python /content/drive/MyDrive/s2/run_stage2.py \
--stage1_dir /content/drive/MyDrive/s1/0704/0704C_ar002_et12_20260511_001 \
--config /content/drive/MyDrive/s2/0707ar002_et12_20260511_001/src_newM/stage2_config_used.yaml \
--out_dir /content/drive/MyDrive/s2/0707ar002_et12_20260511_001/source_destination_attention/ws128_s02_g-
2_len \
--context_method source_destination \
--context_policy online \
--window_size 128 \
--model_type source_destination_attention \
--context_scale 0.2 \
--gate_bias_init -2.0 \
--use_context_length_feature \
--seed 130 \
--epochs 500 \
--batch_size 128
```

****解释几个关键参数****

`--window\_size 128`: 现在表示 source 一路最多 `127` 条历史 + 当前 flow, destination 一路也是最多 `127` 条历史 + 当前 flow。所以计算量约等于单路的两倍。显存紧张可以先试 `64`。

`--context\_scale 0.2`: 控制 context 修正 logits 的强度。建议扫:

0.1, 0.2, 0.3, 0.5

`--gate\_bias\_init -2.0`: 让 gate 初始偏小, 避免模型一开始过度相信 context。建议扫:

-3.0, -2.0, -1.0

`--use\_context\_length\_feature`: 把 source/destination 历史长度也作为特征。你的结果里正负样本 context length 分布不同, 所以这个有机会提升。

****验证****

我已经做了:

```
py_compile passed
end-to-end smoke test passed
source_context_z shape: (6, 4, 128)
destination_context_z shape: (6, 4, 128)
model output shape: (6, 2)
finite=True
```

****建议优先实验组合****

```
source_destination_attention + ws32 + scale 0.2
source_destination_attention + ws64 + scale 0.2
```

```
source_destination_attention + ws128 + scale 0.2
source_destination_attention + ws128 + scale 0.1
source_destination_attention + ws128 + scale 0.3
source_destination_attention + ws128 + gate_bias -3.0
```

目标不是单纯提高 recall，而是看它能否比当前 `source\_host` 更好地平衡：

```
source_host    FP=735
Stage1C       FN=732
PR_AUC        source_host    0.8832
F1_1          0.8009
```

用户 2026-07-08 10:20:37

Files mentioned by the user:

```
## [INFO] Stage2 ----- seed: 130 [INFO] 随机种子已设置: 130 [INFO] Stage2 ----- d...:
C:\Users\XiaoyanXiong\codex\attachments\c9fe9bcc-d914-48b3-88c6-514c617d11f6\pasted-text.txt
```

My request for Codex:

```
!python /content/drive/MyDrive/s2/run_stage2.py \
--stage1_dir /content/drive/MyDrive/s1/0704/0704C_ar002_et12_20260511_001 \
--config /content/drive/MyDrive/s2/0708/dual_host_attention/stage2_config_used.yaml \
--out_dir /content/drive/MyDrive/s2/0708/dual_host_attention/ws128 \
--context_method source_destination \
--context_policy online \
--window_size 128 \
--model_type source_destination_attention \
--context_scale 0.2 \
--gate_bias_init -2.0 \
--use_context_length_feature \
--seed 130 \
--epochs 500 \
--batch_size 128
```

运行结果:Stage2 run summary

=====

```
best_epoch: 11
metric_for_best: pr_auc
best_score: 0.9117405122279064
context_method: source_destination
context_policy: online
window_size: 128
```

Final metrics:

```
train: loss=0.014475, acc=0.989307, macro_precision=0.938461, macro_recall=0.913132,
macro_f1=0.925385, weighted_precision=0.989054, weighted_recall=0.989307, weighted_f1=0.989148,
auc=0.9962204929820448, pr_auc=0.9421956488401709, precision_label1=0.8836619718309859,
recall_label1=0.8306266548984995, f1_label1=0.8563239308462238, cm=[[282760, 1239], [1919, 9411]]
val: loss=0.022505, acc=0.985541, macro_precision=0.909690, macro_recall=0.913616,
macro_f1=0.911643, weighted_precision=0.985612, weighted_recall=0.985541, weighted_f1=0.985576,
auc=0.9921021343993947, pr_auc=0.9117467496524614, precision_label1=0.826710816777042,
recall_label1=0.835005574136009, f1_label1=0.8308374930671104, cm=[[40081, 314], [296, 1498]]
test: loss=0.027916, acc=0.982425, macro_precision=0.879418, macro_recall=0.910943,
```

macro_f1=0.894481, weighted_precision=0.983249, weighted_recall=0.982425, weighted_f1=0.982775, auc=0.9909568814927036, pr_auc=0.8849652031004898, precision_label1=0.766135354063235, recall_label1=0.832954545454555, f1_label1=0.7981489043146862, cm=[[79965, 895], [588, 2932]]
结合我的代码和配置详细解析结果。是否还有提升空间？详细解析

助手 2026-07-08 10:20:55

我先把这次 dual-host 的完整日志和我们刚加的双路 attention 代码对齐一下，重点看三件事：context 长度分布、best epoch/阈值转移、以及 test 相比 source_host/Stage1C 的 FP/FN 变化。

助手 2026-07-08 10:21:31

我已经确认这次 dual-host 的 ranking 指标很强，但正式 test 点被阈值拖住了。最关键的线索是：val 选了 `0.599`，而 test oracle 最优在 `0.678`，这说明现在主要问题已经从“模型有没有学到”变成了“如何把分数校准到更稳的阈值”。

助手 2026-07-08 10:21:41

我再看一下 `find\_best\_threshold` 目前支持哪些 threshold metric。这样我可以判断“直接改配置”够不够，还是需要补一个 F0.5 / precision-constrained threshold 选择。

助手 2026-07-08 10:23:18

这次 `source\_destination\_attention` 的结论很清楚：**模型的排序能力变强了，但正式 test 阈值点没有选好，所以 F1 没有同步提升。** 也就是说，这次不是“模型没学到”，而是 **PR_AUC/AUC 提升了，threshold transfer 拖了最终分类点**。

****核心结果****

模型	Test P1	Test R1	Test F1_1	PR_AUC	AUC	CM
Stage1C	**0.8069**	0.7920	0.7994	0.8763	0.9890	[[80193,667],[732,2788]]
source_host	0.7945	0.8074	**0.8009**	0.8832	0.9903	[[80125,735],[678,2842]]
destination_host	0.7690	**0.8332**	0.7998	0.8797	0.9895	[[79979,881],[587,2933]]
dual source/destination	0.7661	0.8330	0.7981	**0.8850**	**0.9910**	[[79965,895],[588,2932]]

`dual\_host` 的 **PR_AUC=0.884965 是目前最高**，比 `source\_host` 高 `+0.00178`，比 Stage1C 高 `+0.00871`。这说明双路 source/destination attention 的 ranking 确实更强。

但正式 test F1 是 `0.79815`，低于 `source\_host=0.80090`。原因主要在阈值。

****最关键线索****

官方阈值来自 val：

```
val threshold = 0.59899
test official:
P1=0.7661, R1=0.8330, F1=0.7981
CM=[[79965, 895], [588, 2932]]
```

但是 test oracle 阈值是：

```
test oracle threshold = 0.67818
test oracle:
```

```
P1=0.8101, R1=0.7915, F1=0.8007
CM=[[80207, 653], [734, 2786]]
```

这个非常重要：如果阈值调到 `0.678`，dual-host 会变成一个 **高 precision、低 FP** 的模型：

- FP 从 `895` 降到 `653`
- 比 Stage1C 的 FP=667 还少 `14`
- P1 从 `0.7661` 升到 `0.8101`
- F1 从 `0.7981` 升到 `0.8007`

所以 dual-host 的潜力是有的，只是 val 选出来的阈值偏低。

****Context 诊断****

你的双路 context 分布也符合之前判断：

```
| Split/Test | label0 mean | label1 mean | 解释 |
|---|---|---|---|
| source_history_len | 98.0 | **117.9** | 正类 source 历史更多，source 分支是强正向信号 |
| destination_history_len | **90.8** | 65.4 | 正类 destination 历史更少，destination
分支是另一种反向/召回信号 |
| total_history_len | 188.8 | 183.3 | 合起来以后正负差异反而变小 |
```

这说明你“不混 endpoint、分开 source/destination”这个方向是合理的。source 和 destination 的信号方向不同，混在 endpoint 里会互相稀释。

****结合代码看这次模型****

当前模型在

[target_query.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/target_query.py:347) 里是：

```
source branch:      current query -> source history
destination branch: current query -> destination history
fusion: current, summary_src, summary_dst, current-summary_src, current-summary_dst
logits = base_logits + context_scale * gate * delta_logits
```

你的配置：

```
context_scale=0.2
gate_bias_init=-2.0
use_context_length_feature=True
use_multiplicative_fusion=False
metric_for_best=pr_auc
```

这个设计比较保守，`context\_scale=0.2` 和 `gate\_bias=-2` 都是在防止 context 过度改写 current-flow 判断。结果上也确实稳定，PR_AUC 提升了。

****主要问题****

1. `metric\_for\_best=pr\_auc` 选的是 ranking 最好的 epoch，不一定是最终 F1 最好的 epoch。
2. `threshold\_metric=f1\_label1` 在 val 上选了 `0.599`，但 test 更适合 `0.678`。
3. `use\_context\_length\_feature=True` 可能提高 PR_AUC，但也可能让概率校准更依赖 val/test 的长度分布。
4. destination 分支提高 recall，但也带来 FP；现在 single gate 没法分别控制 source 和 destination 的贡献。

****最值得继续试的配置****

先不要换大模型，优先做这些：

1. source_destination_attention + ws64
2. source_destination_attention + ws32
3. source_destination_attention + ws128 + context_scale 0.1

```

4. source_destination_attention + ws128 + context_scale 0.3
5. source_destination_attention + ws128 + gate_bias_init -3.0
6. source_destination_attention + ws128 + use_context_length_feature=False
7. source_destination_attention + ws128 + use_multiplicative_fusion=True
8. source_destination_attention + metric_for_best=f1_label1

```

尤其推荐：

```

--window_size 64 \
--model_type source_destination_attention \
--context_scale 0.2 \
--gate_bias_init -2.0

```

因为 destination 正类 median 大约 `54`，`ws64` 可能已经足够；`ws128` 对 destination 可能引入更多负类噪声。

****代码层面最有提升机会的修改****

第一，给 source/destination 分开 gate：

```

logits = base_logits
        + scale_src * gate_src * delta_src
        + scale_dst * gate_dst * delta_dst

```

现在是一个 fused gate 同时控制两路。source 更稳，destination 更召回，应该分开控制。

第二，加 `F0.5` 或 precision-constrained threshold。现在

[trainer.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/trainer.py:296) 只支持：

```
f1, recall, precision, macro_f1
```

但你的场景明显需要“别让 FP 太多”。可以加：

```
threshold_metric = f0.5_label1
```

让阈值更偏 precision，可能会把 dual-host 的正式 test 点推向 `0.65~0.68`。

第三，做 length feature ablation。当前 length 特征很可能是双刃剑：它能提高 PR_AUC，但可能让阈值转移不稳。建议直接跑一版不带：

```

model:
    use_context_length_feature: false

```

第四，打开 multiplicative fusion：

```

model:
    use_multiplicative_fusion: true

```

这会加入：

```

current * summary_src
current * summary_dst

```

它有机会提高“当前 flow 与历史是否匹配”的表达能力。

****最终判断****

这次 dual-host 是有价值的：****它没有让 official F1 变最好，但把 PR_AUC/AUC 推到了当前最高****。下一步重点不是继续堆 attention 层，而是：

```

1. threshold
2. window_size
3. ablation length feature
4. source/destination gate

```

如果目标是论文/实验结果，`dual\_host` 可以作为“ranking 最优模型”；如果目标是最终分类 F1，目前 `source\_host` 仍是 official F1 最稳的模型。

用户 2026-07-08 10:30:15

现尝试这个4. source/destination 分开 gate，请帮我修正代码

助手 2026-07-08 10:30:28

我来直接把 `Stage2SourceDestinationAttention` 改成两条 residual correction: source 一条 gate/delta, destination 一条 gate/delta。这样 source 分支可以偏稳、destination 分支可以偏召回，不再被一个总 gate 绑在一起。

助手 2026-07-08 10:30:48

我先读当前模型相关代码，确认 fusion_dim、gate、delta 的定义和 forward 拼接方式，避免把已有的 `use\_context\_length\_feature` / `use\_multiplicative\_fusion` 破坏掉。

助手 2026-07-08 10:31:12

我会把原来的 `fused -> delta\_head + gate` 替换成两套分支: `source\_delta/source\_gate` 和 `destination\_delta/destination\_gate`。同时保留 `source\_context\_scale` / `destination\_context\_scale` 两个可选配置，后面可以单独压低 destination 分支来减少 FP。

助手 2026-07-08 10:31:47

模型主体已经改成双 gate。接下来我会给 `run\_stage2.py` 和 `run\_stage2\_tuned.py` 加两个命令行参数，便于你单独调 `source\_context\_scale` 和 `destination\_context\_scale`。

助手 2026-07-08 10:31:54

这两个 scale 很有用：你的 destination 分支明显更偏召回、也更容易带来 FP，所以后续可以跑 `source=0.2, destination=0.1` 这种更稳的组合。

助手 2026-07-08 10:32:06

`run\_stage2.py` 已补。同步到 tuned 入口，避免两个脚本行为不一致。

助手 2026-07-08 10:32:17

CLI 覆盖也完成了。现在我运行编译检查和一个 forward smoke test，确认新结构维度拼接没有错。

助手 2026-07-08 10:32:24

先编译 `target\_query.py` 和两个入口脚本。

助手 2026-07-08 10:32:38

编译通过。现在做最小 forward: source/destination 两路都含有无历史和有历史样本，检查输出 finite。

助手 2026-07-08 10:33:04

forward 测试通过。最后我会把关键代码位置和建议跑法整理给你，尤其是先压低 destination scale 的实验，因为你上次结果里 destination 分支明显更偏召回。

助手 2026-07-08 10:33:21

已改好：`source\_destination\_attention` 现在已经从 **一个总 gate** 改成 **source gate + destination gate 两套独立 residual correction**。

核心修改在：

[stage2/target_query.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/target_query.py:347)

现在逻辑变成：

```
base_logits = base_head(current)

source_delta = source_delta_head(source_fused)
source_gate = source_gate(source_fused)

destination_delta = destination_delta_head(destination_fused)
destination_gate = destination_gate(destination_fused)

logits = (
    base_logits
    + source_context_scale * source_gate * source_delta
    + destination_context_scale * destination_gate * destination_delta
)
```

这样 source 和 destination 不再被一个 gate 绑死。source 分支可以学“稳定修正”，destination 分支可以学“召回型修正”，互相不强行共享 gate。

我也补了两个 CLI 参数：

[run_stage2.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/run_stage2.py:130)
[run_stage2_tuned.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/run_stage2_tuned.py:130)

```
--source_context_scale
--destination_context_scale
```

****建议优先跑这个****

你的上次 dual-host 结果里 destination 分支偏召回、FP 较多，所以第一版我建议 **source 保持 0.2，destination 压低到 0.1**：

```
!python /content/drive/MyDrive/s2/run_stage2.py \
--stage1_dir /content/drive/MyDrive/s1/0704/0704C_ar002_et12_20260511_001 \
--config /content/drive/MyDrive/s2/0708/dual_host_attention/stage2_config_used.yaml \
--out_dir /content/drive/MyDrive/s2/0708/dual_host_attention/separate_gate_ws128_src02_dst01 \
--context_method source_destination \
--context_policy online \
--window_size 128 \
--model_type source_destination_attention \
--context_scale 0.2 \
--source_context_scale 0.2 \
--destination_context_scale 0.1 \
--gate_bias_init -2.0 \
--use_context_length_feature \
--seed 130 \
--epochs 500 \
--batch_size 128
```

然后再跑两个对照：

```
--source_context_scale 0.2 --destination_context_scale 0.2
```

```
--source_context_scale 0.3 --destination_context_scale 0.1
```

****为什么这样试****

上次 official 结果:

```
P1=0.7661, R1=0.8330, FP=895, FN=588
```

test oracle 阈值下:

```
P1=0.8101, R1=0.7915, FP=653, FN=734
```

说明模型 ranking 很强, 但 destination 召回倾向把默认阈值点推得太激进。分开 gate + 降低 destination scale 的目标就是:

FP			
source			
official threshold	oracle	precision	

****验证****

已通过:

```
py_compile passed
forward smoke test passed
output shape = (4, 2)
finite = True
```

这版可以直接复制到 Colab 跑。

用户 2026-07-08 10:44:44

```
!python /content/drive/MyDrive/s2/run_stage2.py \
--stage1_dir /content/drive/MyDrive/s1/0704/0704C_ar002_et12_20260511_001 \
--config
/content/drive/MyDrive/s2/0708ar002_et12_20260511_001/dual_host_attention/stage2_config_used.yaml \
--out_dir /content/drive/MyDrive/s2/0708ar002_et12_20260511_001/dual_host_attention/separate_gate_ws128_src02_dst01 \
--context_method source_destination \
--context_policy online \
--window_size 128 \
--model_type source_destination_attention \
--context_scale 0.2 \
--source_context_scale 0.2 \
--destination_context_scale 0.1 \
--gate_bias_init -2.0 \
--use_context_length_feature \
--seed 130 \
--epochs 500 \
--batch_size 128
```

运行结果: [Stage2Trainer]---fit lr=3e-05, weight_decay=0.0003, patience=15, threshold=0.5, metric_for_best=pr_auc
fit-----best threshold = 0.5841414141414141
[EPOCH 001] train_loss=0.221015 val_loss=0.091916 val_f1_label1=0.5228 val_macro_f1=0.7511
val_weighted_f1=0.9599989759947591 val_pr_auc=0.4596 val_auc=0.9444lr=0.00000720 *
Traceback (most recent call last):

[illegible]

这个 traceback 我先判断一下：`KeyboardInterrupt` 不是模型报错，也不是 NaN/OOM。它表示 Python 收到了“中断执行”的信号，通常是 Colab 里手动点了停止、页面/运行时打断，或者某一步太慢你主动/系统中断了。它发生在第二个 epoch 的 backward 中，说明第 1 个 epoch 已经正常完成了。

这不是模型代码错误。`KeyboardInterrupt` 的意思是：**运行被中断了**。

```
scaler.scale(loss).backward()
KeyboardInterrupt
```

```
NaN
CUDA out of memory
shape mismatch
```

第 1 个 epoch 已经正常跑完:

所以模型、双路 context、separate gate 至少已经能正常 forward/backward 一个 epoch。

最直接：重新运行同一条命令即可。建议换一个新的`out dir`，避免和已经中断的半成品目录混在一起：

```
--out_dir /content/drive/MyDrive/s2/0708ar002_et12_20260511_001/dual_host_attention/  
separate_gate_ws128_src02_dst01_retry
```

如果它每次都在第 2 个 epoch 附近卡很久，再做这些调整：

1. 先把 `window\_size` 降到 `64`

```
--window_size 64
```

双路模型实际会处理 source 一路 + destination 一路，`ws128` 等于最多 `127 + 127` 条历史，计算量比单路大。

2. 保持 separate gate，但降低 destination 分支强度

```
--source_context_scale 0.2 \  
--destination_context_scale 0.1
```

这个你已经用了，是合理的。

3. 如果 Colab 显存够，可以试 `batch\_size 256`，减少 step 数；如果显存紧张，就用 `batch\_size 64`

```
--batch_size 256
```

或：

```
--batch_size 64
```

4. 先跑小实验确认趋势，不要一开始 500 epochs

```
--epochs 80
```

你的 early stopping 是 `patience=15`，一般不会真的跑满 500。

****我建议下一条命令****

先跑更稳、更快的 `ws64`：

```
!python /content/drive/MyDrive/s2/run_stage2.py \  
--stage1_dir /content/drive/MyDrive/s1/0704/0704C_ar002_et12_20260511_001 \  
--config /content/drive/MyDrive/s2/0708ar002_et12_20260511_001/dual_host_attention/stage2_config_used.yaml \  
--out_dir /content/drive/MyDrive/s2/0708ar002_et12_20260511_001/dual_host_attention/  
separate_gate_ws64_src02_dst01 \  
--context_method source_destination \  
--context_policy online \  
--window_size 64 \  
--model_type source_destination_attention \  
--context_scale 0.2 \  
--source_context_scale 0.2 \  
--destination_context_scale 0.1 \  
--gate_bias_init -2.0 \  
--use_context_length_feature \  
--seed 130 \  
--epochs 500 \  
--batch_size 128
```

如果 `ws64` 正常跑完，再回头跑 `ws128`。这次 traceback 本身没有暴露代码 bug，只是运行被中断了。

用户 2026-07-08 11:02:27

Final metrics:

```
train: loss=0.014475, acc=0.989307, macro_precision=0.938461, macro_recall=0.913132,  
macro_f1=0.925385, weighted_precision=0.989054, weighted_recall=0.989307, weighted_f1=0.989148,  
auc=0.9962204929820448, pr_auc=0.9421956488401709, precision_label1=0.8836619718309859,  
recall_label1=0.8306266548984995, f1_label1=0.8563239308462238, cm=[[282760, 1239], [1919, 9411]]  
val: loss=0.022505, acc=0.985541, macro_precision=0.909690, macro_recall=0.913616,  
macro_f1=0.911643, weighted_precision=0.985612, weighted_recall=0.985541, weighted_f1=0.985576,
```

auc=0.9921021343993947, pr_auc=0.9117467496524614, precision_label1=0.826710816777042, recall_label1=0.835005574136009, f1_label1=0.8308374930671104, cm=[[40081, 314], [296, 1498]]
test: loss=0.027916, acc=0.982425, macro_precision=0.879418, macro_recall=0.910943, macro_f1=0.894481, weighted_precision=0.983249, weighted_recall=0.982425, weighted_f1=0.982775, auc=0.9909568814927036, pr_auc=0.8849652031004898, precision_label1=0.766135354063235, recall_label1=0.8329545454545455, f1_label1=0.7981489043146862, cm=[[79965, 895], [588, 2932]]
为什么precision从train到test减少那么多? 详细解析

助手 2026-07-08 11:03:03

核心原因: **不是 test recall 明显变差, 而是 test 上负类被误报成正类的比例明显升高了.** 所以 precision 掉得很明显。

你这组结果:

Split	TP	FP	FN	TN	P1	R1
train	9411	1239	1919	282760	0.8837	0.8306
val	1498	314	296	40081	0.8267	0.8350
test	2932	895	588	79965	0.7661	0.8330

Recall 基本没变:

```
train R1 = 0.8306
val   R1 = 0.8350
test  R1 = 0.8330
```

但 FP rate 明显升高:

```
train FPR = 1239 / 283999 = 0.436%
val   FPR = 314 / 40395  = 0.777%
test  FPR = 895 / 80860  = 1.107%
```

也就是说: **test 上每 10000 个正常流, 大约有 111 个被误报; train 只有 44 个.** 在正类比例只有约 4% 的数据里, FPR 从 0.44% 升到 1.11%, precision 会被非常明显地拉低。

为什么会这样

1. **模型对 train 的负类学得更好, 对 test 的负类泛化弱一些**

train:

```
FP=1239, TP=9411
precision = 9411 / (9411 + 1239) = 0.8837
```

test:

```
FP=895, TP=2932
precision = 2932 / (2932 + 895) = 0.7661
```

test 的 TP/正类召回没问题, 问题是更多负类样本被打到阈值以上。

2. **dual source/destination 模型偏 recall**

你这个模型的 test recall 是 `0.8330`, 比 Stage1C 的 `0.7920` 高很多。代价就是 FP 增加:

```
Stage1C FP=667
dual_host FP=895
```

所以 precision 从 Stage1C 的 `0.8069` 掉到 `0.7661`。这不是异常, 是当前阈值下模型更激进。

3. **val 选出来的阈值偏低**

这次 official threshold 是 val 选的, 大概 `0.599`。之前你这组结果里 test oracle 显示:

```
test oracle threshold ≈ 0.678
test oracle P1 ≈ 0.810
test oracle R1 ≈ 0.791
test oracle F1 ≈ 0.8007
```

这说明模型排序能力很好, `PR\_AUC=0.88497` 也是目前很高的; 但 val 选的阈值对 test 来说偏低, 导致 test FP 多。

4. **source/destination context 分布对负类影响不同**

你前面的诊断里:

```
source_history_len:    >
destination_history_len:  <
```

source 分支比较像稳定正向信号, destination 分支更复杂, 容易提高 recall, 也容易把一些负类抬高。你现在的 single official threshold 下, destination 相关噪声会反映成 FP 增加。

5. **train 与 val/test 的 context 长度分布不同**

train 的 source/destination history 平均更短, val/test 更长、更接近满窗口。模型在 train 上看到的负类 context 分布和 test 上不完全一样。双路 attention 在长历史下更容易找到“看起来像攻击”的历史片段, 所以 test 负类分数更容易被抬高。

****是否过拟合? ****

有一点, 但不是严重崩坏式过拟合。

看 ranking:

```
train PR_AUC = 0.9422
val   PR_AUC = 0.9117
test  PR_AUC = 0.8850
```

train 到 test 有明显 gap, 说明模型确实学到了一些 train-specific pattern。但 test AUC/PR_AUC 仍然很高, 说明不是模型无效, 而是 ****calibration/threshold 和 FP 控制问题****。

****建议修正方向****

优先尝试:

1. threshold threshold_metric precision
2. separate gate
3. source_context_scale=0.2, destination_context_scale=0.1
4. window_size 64 destination
5. use_context_length_feature ablation

你现在最该关注的不是 recall, 而是把 test FPR 从 `1.107%` 压回到 `0.8%` 以下。只要 FP 从 `895` 降到大约 `750`, 同时 TP 保持在 `2850+`, F1 和 precision 都会明显改善。

用户 2026-07-08 11:30:32

Files mentioned by the user:

```
## [INFO] Stage2 ----- seed: 130 [INFO] 随机种子已设置: 130 [INFO] Stage2 ----- d...:
C:\Users\XiaoyanXiong\codex\attachments\dfa5bbac-f869-408e-82ff-1effe4f7ab4a\pasted-text.txt
```

My request for Codex:

```
!python /content/drive/MyDrive/s2/run_stage2.py \
--stage1_dir /content/drive/MyDrive/s1/0704/0704C_ar002_et12_20260511_001 \
```

```
--config /content/drive/MyDrive/s2/0707ar002_et12_20260511_001/src_newM/stage2_config_used.yaml \
--out_dir /content/drive/MyDrive/s2/0707ar002_et12_20260511_001/src_newM/ws128 \
--context_method source_host \
--context_policy online \
--window_size 128 \
--model_type target_query_gated \
--seed 130 \
--epochs 500 \
--batch_size 128
```

运行结果: Stage2 run summary

=====

best_epoch: 12
metric_for_best: pr_auc
best_score: 0.9127801911628701
context_method: source_host
context_policy: online
window_size: 128

Final metrics:

train: loss=0.014011, acc=0.989618, macro_precision=0.955888, macro_recall=0.898168,
macro_f1=0.924908, weighted_precision=0.989255, weighted_recall=0.989618, weighted_f1=0.989268,
auc=0.9964784184455083, pr_auc=0.9452900132842286, precision_label1=0.9197480698902885,
recall_label1=0.7991173874669021, f1_label1=0.8551997733068858, cm=[[283209, 790], [2276, 9054]]
val: loss=0.022961, acc=0.986110, macro_precision=0.924041, macro_recall=0.901396,
macro_f1=0.912376, weighted_precision=0.985790, weighted_recall=0.986110, weighted_f1=0.985920,
auc=0.9911598991177286, pr_auc=0.9127873554175314, precision_label1=0.8565525383707202,
recall_label1=0.8088071348940914, f1_label1=0.8319954128440367, cm=[[40152, 243], [343, 1451]]
test: loss=0.030160, acc=0.983254, macro_precision=0.893065, macro_recall=0.899148,
macro_f1=0.896081, weighted_precision=0.983387, weighted_recall=0.983254, weighted_f1=0.983319,
auc=0.990286994707463, pr_auc=0.883188786200487, precision_label1=0.7945205479452054,
recall_label1=0.8073863636363636, f1_label1=0.8009017894885163, cm=[[80125, 735], [678, 2842]]

详细解析, 是否能证明比stage1C 有提升?

助手 2026-07-08 11:31:18

结论: **可以说 Stage2 `source\_host + target\_query\_gated` 相比 Stage1C
有“部分明确提升”, 但不能说“全面提升”。**
更准确的表述是:

> Stage2 source_host 在 **正类召回、正类 F1、Macro F1、AUC、PR_AUC** 上优于 Stage1C; 但在
accuracy、正类 precision、FP 数量 上弱于 Stage1C。

Stage1C vs Stage2 Source Host

Metric	Stage1C	Stage2 source_host	变化
Accuracy	**0.983420**	0.983254	-0.000166
Macro Precision	**0.898951**	0.893065	-0.005886
Macro Recall	0.891898	**0.899148**	+0.007250

Macro F1	0.895390	**0.896081**	+0.000691
Weighted F1	**0.983346**	0.983319	-0.000028
P1	**0.806946**	0.794521	-0.012426
R1	0.792045	**0.807386**	+0.015341
F1_1	0.799427	**0.800902**	+0.001475
AUC	0.989037	**0.990287**	+0.001250
PR_AUC	0.876256	**0.883189**	+0.006933

****Confusion Matrix 对比****

Stage1C:

TN=80193, FP=667
FN=732, TP=2788

Stage2 source_host:

TN=80125, FP=735
FN=678, TP=2842

变化:

TP +54
FN -54
FP +68
TN -68

也就是说, Stage2 多抓到了 `54` 个正类攻击流, 但多误报了 `68` 个负类正常流。

****如何理解****

Stage1C 更保守:

Precision FP
P1=0.8069, FP=667

Stage2 source_host 更偏召回:

Recall FN
R1=0.8074, FN=678

对入侵检测任务来说, 这个 trade-off

是合理的, 因为漏报攻击通常比多一点误报更危险。但如果你的系统更重视低误报, Stage1C 仍然更稳。

****最有说服力的提升是 PR_AUC****

你的数据正类比例只有大约 `4.17%`, 所以 PR_AUC 比 accuracy 更重要。Stage2 source_host:

PR_AUC: 0.8763 -> 0.8832
: +0.00693

这个提升比 F1 的 `+0.00148` 更有意义, 因为 PR_AUC 是 threshold-independent ranking metric, 说明 Stage2 的 source history 让模型整体排序能力变强, 而不是只靠某个阈值碰巧变好。

****能不能“证明”比 Stage1C 好? ****

现在只能说: **在 seed=130、当前 test split、当前 validation threshold 策略下, Stage2 source_host 在关键少数类 ranking 和 recall/F1 上优于 Stage1C。 **

但严格意义上还不能“证明全面优于”, 原因是:

1. 只跑了一个 seed。
2. F1_1 提升很小: `+0.001475`。

3. Accuracy 和 precision 下降。
4. FP 增加了 `68` 个。
5. 还没有做 bootstrap / McNemar / 多 seed 显著性检验。

****论文里建议这样写****

可以写:

> Compared with Stage1C, the source-host Stage2 model improves minority-class recall from 0.7920 to 0.8074, F1 from 0.7994 to 0.8009, AUC from 0.9890 to 0.9903, and PR-AUC from 0.8763 to 0.8832. This indicates that source-host temporal context provides additional discriminative signal for attack detection, especially in ranking positive flows. The improvement is obtained at the cost of a small increase in false positives.

中文就是:

> 相比 Stage1C, Stage2 source_host 利用源主机历史上下文提升了正类召回、正类 F1、AUC 和 PR_AUC, 说明源主机历史行为对攻击检测有额外判别信息。但这种提升伴随 precision 下降和 FP 增加, 因此不是全面优于 Stage1C, 而是偏向提高检出能力的改进。

****下一步怎么让结论更硬****

建议至少跑:

```
seed = 130, 131, 132, 133, 134
```

然后汇总:

```
mean ± std of F1_1, PR_AUC, AUC, P1, R1
```

如果 `source\_host` 的 PR_AUC 和 F1_1 在多数 seed 都高于 Stage1C, 就可以更有底气说 Stage2 有稳定提升。当前结果已经支持 “有提升趋势”, 尤其是 PR_AUC。

用户 2026-07-08 11:36:30

Files mentioned by the user:

```
## [INFO] Stage2 ----- seed: 130 [INFO] 随机种子已设置: 130 [INFO] Stage2 ----- d...:
C:\Users\XiaoyanXiong\.codex/attachments/826cc3fd-d102-495e-81b3-4caefcbc626a/pasted-text.txt
```

My request for Codex:

```
!python /content/drive/MyDrive/s2/run_stage2.py \
--stage1_dir /content/drive/MyDrive/s1/0704/0704C_ar002_et12_20260511_001 \
--config
/content/drive/MyDrive/s2/0708ar002_et12_20260511_001/dual_host_attention/stage2_config_used.yaml \
--out_dir /content/drive/MyDrive/s2/0708ar002_et12_20260511_001/dual_host_attention/separate_gate_ws128_src02_dst01 \
--context_method source_destination \
--context_policy online \
--window_size 128 \
--model_type source_destination_attention \
--context_scale 0.2 \
--source_context_scale 0.2 \
--destination_context_scale 0.1 \
```

```
--gate_bias_init -2.0 \
--use_context_length_feature \
--seed 130 \
--epochs 500 \
--batch_size 128
运行结果详细解析，还有提升空间吗？
```

助手 2026-07-08 11:36:47

我先把这次 `separate\_gate\_ws128\_src02\_dst01` 的完整日志指标抓出来，尤其看 official test、test oracle、val threshold、以及和上一版 single-gate / source_host / Stage1C 的差异。

助手 2026-07-08 11:37:10

我抓到了这次 separate-gate 的关键结果：official test
F1=0.80154、PR_AUC=0.88771、AUC=0.99124，已经超过你之前的 `source\_host` 和 single-gate dual-host。现在我把它和 Stage1C、source_host、single-gate dual-host 放在同一口径下解读。

助手 2026-07-08 11:38:15

****结论****

这次 `source\_destination\_attention + separate gate + src\_scale=0.2 + dst\_scale=0.1` 是目前最好的 Stage2 结果之一，而且比上一版 single-gate dual-host 明显更稳。

最重要的变化是：

```
PR_AUC = 0.8877
AUC     = 0.9912
F1_1    = 0.8015   Stage1C / source_host / single-gate dual-host
FP      = 686     Stage1C 667       single-gate 895
```

所以这次修改是有效的。****分开 source/destination gate 的方向是对的。****

****核心结果****

Model P1 R1 F1_1 AUC PR_AUC FP FN TP
--- --- --- --- --- --- --- --- ---
Stage1C **0.8069** 0.7920 0.7994 0.9890 0.8763 **667** 732 2788
source_host target_query_gated 0.7945 0.8074 0.8009 0.9903 0.8832 735 **678** **2842**
dual-host single gate 0.7661 **0.8330** 0.7981 0.9910 0.8850 895 588 2932
dual-host separate gate 0.8039 0.7991 **0.8015** **0.9912** **0.8877** 686 707 2813

****相比 Stage1C****

Stage1C:

```
CM = [[80193, 667], [732, 2788]]
```

separate gate:

```
CM = [[80174, 686], [707, 2813]]
```

变化:

```
TP +25
FN -25
```

```
FP +19
TN -19
```

这个 trade-off 很好：只多了 `19` 个误报，但少漏报 `25` 个攻击流。同时：

```
F1_1: 0.7994 -> 0.8015 (+0.0021)
PR_AUC: 0.8763 -> 0.8877 (+0.0115)
AUC: 0.9890 -> 0.9912 (+0.0022)
```

这比之前 `source\_host` 的提升更有说服力，因为这次不仅 PR_AUC 提升，official F1 也更高，而且 FP 控制得更接近 Stage1C。

****相比 source_host****

source_host:

```
CM = [[80125, 735], [678, 2842]]
```

separate gate:

```
CM = [[80174, 686], [707, 2813]]
```

变化：

```
FP -49
FN +29
P1: 0.7945 -> 0.8039
R1: 0.8074 -> 0.7991
F1: 0.8009 -> 0.8015
PR_AUC: 0.8832 -> 0.8877
```

也就是说，separate gate 用一点 recall 换来了更高 precision，并且 F1/PR_AUC 都更好。这个结果说明：****source_host 单路虽然召回更高，但双路 separate gate 的排序能力和整体平衡更好。****

****相比 single-gate dual-host****

single-gate dual-host:

```
CM = [[79965, 895], [588, 2932]]
P1=0.7661, R1=0.8330, F1=0.7981
```

separate gate:

```
CM = [[80174, 686], [707, 2813]]
P1=0.8039, R1=0.7991, F1=0.8015
```

变化：

```
FP -209
FN +119
Precision +0.0378
Recall -0.0338
F1 +0.0034
PR_AUC +0.0027
```

这就是分开 gate 的价值：single-gate 过度偏 recall，把很多正常流推成正类；separate-gate 把 destination 分支压低到 `0.1` 后，明显减少误报。

****阈值稳定性也明显变好****

这次：

```
Val selected threshold = 0.6683
Test oracle threshold = 0.6435
Threshold transfer gap = 0.000076
```

上一版 single-gate 的 threshold transfer gap 大约是 `0.002541`。这次几乎没 gap，说明 val 选出来的阈值能很好迁移到 test。

这是一个非常好的信号：`\*\*模型不只是 test oracle 好，而是 official test 也好。\*\*`

****训练过程****

best epoch 是 `12`：

```
best_epoch=12
metric_for_best=pr_auc
val_pr_auc=0.91349
```

后面 epoch 的 val F1 有时更高，例如 epoch 24 的 `val\_f1\_label1=0.8337`，但 PR_AUC 下降到 `0.9091`，所以 trainer 按 `pr\_auc` 选择了 epoch 12。这是合理的，因为你的最终 test PR_AUC 也很高。

****小注意****

日志里有一个 1 个样本级别的小差异：

```
TEST AT VAL THRESHOLD:
CM = [[80175, 685], [706, 2814]]

Final TEST Full metrics:
CM = [[80174, 686], [707, 2813]]
```

只差 1 个样本，通常来自 AMP / evaluation 口径细微差异，不影响结论。之后可以统一 final evaluate 和 threshold diagnosis 的 AMP 设置，避免这种小漂移。

****是否还有提升空间？有****

优先尝试这几组：

1. src=0.2, dst=0.05

目标：进一步减少 FP，看 precision 能不能超过 Stage1C，同时保持 F1 不掉太多。

2. src=0.3, dst=0.1

目标：增强 source 分支，试图找回一点 recall，同时不放大 destination 噪声。

3. window_size=64, src=0.2, dst=0.1

目标：减少长历史噪声，尤其 destination 历史噪声。这个很值得跑。

4. use_context_length_feature=False

目标：检查 length feature 是否带来过拟合。现在 length feature 有用，但也可能影响泛化。

5. use_multiplicative_fusion=True

目标：加入 `current \* summary\_src` / `current \* summary\_dst`，增强当前 flow 和历史是否匹配的表达。

****我建议下一步最优先跑****

```
--window_size 64 \
--source_context_scale 0.2 \
--destination_context_scale 0.1
```

然后跑：

```
--window_size 128 \
--source_context_scale 0.2 \
--destination_context_scale 0.05
```

****最终判断****

这次结果已经可以作为你目前的 strongest Stage2 candidate：

```
Best PR_AUC
Best AUC
Best official F1_1
FP      Stage1C
FN      Stage1C
```

如果写论文，可以说：

> Separate source/destination gated fusion improves the precision-recall trade-off over single-gate dual-host attention. It achieves the highest PR-AUC and AUC while also improving minority-class F1 over Stage1C, with only a small increase in false positives.

用户 2026-07-08 11:42:30

我想知道target_query_gated在这个模型的基础上，我还能继续做什么优化吗？使它的性能更好？

助手 2026-07-08 11:42:45

我会基于你已经跑出的结果来判断，而不是泛泛说“加层”。`target\_query\_gated`当前最大问题不是没有表达能力，而是它把 context 直接注入 representation，容易提高 recall 但压低 precision；所以优化方向应该是“让 context 更有选择地介入”。

助手 2026-07-08 11:43:30

可以继续优化，但要先明确一点：**原始`target\_query\_gated`已经不是最强方向了。**
从你目前结果看，性能排序大致是：

```
separate source/destination gate > source_host target_query_gated > single-gate dual-host / time_only
```

所以如果目标是“最终性能更好”，我建议不要只死守原始`target\_query\_gated`，而是在它的基础上做更保守、更可控的改造。

****当前 target_query_gated 的问题****

你的`target\_query\_gated`大致是：

```
current = flow_embedding
summary = attention(current, history)
gate = sigmoid(...)
fused = current + gate * context_update
logits = cls_head(fused)
```

问题是：context 是直接改 representation 的。

一旦 history 有噪声，`fused` 会偏离原本很强的 Stage1 current-flow 表征，容易出现：

```
Recall
FP
Precision
```

这就是你之前`timeOnly`、`destination\_host`、single-gate dual-host 的典型现象。

****优先级 1：改成 residual-logit correction****

最推荐。也就是不要让 context 直接改 embedding，而是让 context 只修正 logits：

```
base_logits = base_head(current)
delta_logits = delta_head([current, summary, current-summary, current*summary])
gate = sigmoid(gate_head(...))

final_logits = base_logits + context_scale * gate * delta_logits
```

优点:

1. Stage1/current-flow
2. context
3. FP
4. threshold

你后面的 `separate\_gate` 实验其实已经证明这个方向有效。

推荐配置:

```
--model_type target_query_residual \  
--context_method source_host \  
--context_scale 0.2 \  
--gate_bias_init -2.0 \  
--use_context_length_feature
```

如果想更保守:

```
--context_scale 0.1 \  
--gate_bias_init -3.0
```

****优先级 2: 给 target_query_gated 加 “保守初始化” ****

原始 `target\_query\_gated` 的 gate 默认不一定很小, 所以训练初期 context 可能介入太强。可以加:

```
gate_bias_init = -2.0 -3.0
```

让初始 gate 接近:

```
sigmoid(-2) ≈ 0.119  
sigmoid(-3) ≈ 0.047
```

也就是说, 模型一开始主要相信 current-flow, 训练中再慢慢学会什么时候用 context。

同时可以把 `context\_update` 最后一层 zero-init:

```
nn.init.zeros_(self.context_update[-1].weight)  
nn.init.zeros_(self.context_update[-1].bias)
```

这样模型初始近似:

```
fused ≈ current
```

不会一上来破坏 Stage1 表征。

****优先级 3: source/destination 分开 gate****

你已经跑出结果证明这个方向有效。

原始 target_query_gated 是:

```
summary  
gate
```

更好的结构是:

```
summary_src = attention(current, source_history)  
summary_dst = attention(current, destination_history)  
  
logits = base_logits  
+ scale_src * gate_src * delta_src  
+ scale_dst * gate_dst * delta_dst
```

你的结果已经说明:

```
source
destination      recall
```

所以应该分开控制。当前最好的配置：

```
--source_context_scale 0.2 \
--destination_context_scale 0.1
```

后续建议继续扫：

```
src=0.2, dst=0.05
src=0.3, dst=0.1
src=0.2, dst=0.0 #          source branch  destination
```

****优先级 4：调 window_size****

你现在 `ws128` 很强，但不一定最优。

双路模型里 `ws128` 实际最多是：

```
source 127 history + destination 127 history
```

历史非常多，destination 分支尤其容易引入噪声。

建议跑：

```
source_host target_query_gated: ws16, ws32, ws64, ws128
dual separate gate: ws32, ws64, ws128
```

我最看好：

```
--window_size 64 \
--source_context_scale 0.2 \
--destination_context_scale 0.1
```

因为 destination 正类 median history 大约 54，`ws64` 可能已经够了。

****优先级 5：加入 recency bias****

现在 attention 只知道 position encoding，但没有明确鼓励“越近越重要”。可以在 attention score 里加一个时间/年龄惩罚：

```
attention_score += -lambda * age
```

直觉：

```
source
      history
```

可试：

```
lambda = 0.01, 0.02, 0.05
```

如果没有真实时间 delta，也可以用序列 age index。

****优先级 6：context dropout****

训练时随机丢一部分 history token：

```
p = 0.1  0.2
```

目的：

1.

```
2.
3.    test FP
```

特别适合 `ws128`，因为长 history 很容易夹杂噪声。

****优先级 7: top-k attention / sparse history****

不是所有 127 条历史都有用。可以让 attention 只保留权重最高的 top-k:

```
k = 16, 32, 64
```

或者先用最近 k 条:

```
recent_k = 32 / 64
```

这比简单扩大 window 更稳。你的结果已经显示: 更多 history 不一定等于更好。

****优先级 8: 改 threshold 策略****

你现在很多模型的 ranking 很强, 但 official F1 受 threshold 影响。可以增加:

```
threshold_metric = f0.5_label1
```

F0.5 更偏 precision:

```
F0.5    FP
F1      precision/recall
F2      recall
```

对你的任务, `F1` 和 `F0.5` 都值得试。尤其当模型 PR_AUC 很高但 FP 偏多时, F0.5 threshold 会更稳。

****优先级 9: 把 Stage1C logit/prob 作为 base****

这是很强的方向。现在 Stage2 的 `base\_head(current)` 是重新训练的, 不等于 Stage1C 原分类器。更理想的是:

```
final_logit = stage1_logit + context_delta
```

也就是:

```
Stage1C    current-flow
Stage2 context      Stage1C
```

这会最大程度保留 Stage1C 的 high precision。

如果你能在 Stage1 保存每个 flow 的:

```
stage1_prob_label1
stage1_logit_label1
```

Stage2 可以直接使用它作为 feature 或 base logit。

****我建议的实验顺序****

先跑这几组, 不要太发散:

```
1. separate gate, ws64, src=0.2, dst=0.1
2. separate gate, ws128, src=0.2, dst=0.05
3. separate gate, ws128, src=0.3, dst=0.1
4. separate gate, ws128, src=0.2, dst=0.0
5. source_host target_query_residual, scale=0.2, gate=-2
```


6. source_host target_query_residual, scale=0.1, gate=-3
7. current best separate gate + no length feature
8. current best separate gate + multiplicative fusion

判断标准:

PR_AUC
F1_1
FP/FN trade-off

目前你的 best candidate 是:

dual source/destination separate gate
ws128
src_scale=0.2
dst_scale=0.1
PR_AUC=0.8877
F1_1=0.8015

继续优化的核心不是“让模型更大”，而是:

context
destination
threshold
Stage1C precision

用户 2026-07-08 11:48:41

17. Relation-aware attention bias

如果你做 mixed context, 可以给 attention 加关系 bias:

same_source + b_src
same_destination + b_dst
same_endpoint + b_ep
time_distance - lambda * age

attention score:

score = qk / sqrt(d) + relation_bias

可否做这个功能? 详细解析

助手 2026-07-08 11:48:53

可以做, 而且这类 bias 很适合 mixed context: 它不会把 source/destination 强行混成一个 embedding, 而是在 attention 分数层面告诉模型“这条历史和当前流是什么关系”。我先看一下你现在的 `target\_query`、`dataset/context` 数据流, 确认最小改动点, 然后给你一个尽量不影响已有模型的新增实现。

助手 2026-07-08 11:49:03

我看到现在 mixed context 还是只把 `context\_z` 送进模型, 模型不知道每个历史 token 和当前 token 是 same_source 还是 same_destination。实现 relation-aware bias 的关键是: 在 `Dataset` 里额外构造每个历史 token 的关系特征, 然后新增一个模型类型专门消费这个 bias。

助手 2026-07-08 11:49:17

设计上我会把它做成一个新增 `model\_type=relation\_aware\_attention`, 这样不会碰你当前表现最好的 `source\_destination\_attention` 路线。它复用 mixed context 的 `context\_z/mask`, 额外传入

`relation\_features`，所以旧配置和旧实验都不会被破坏。

助手 2026-07-08 11:49:55

我准备加的是“分数级 bias”，不是把关系 one-hot 拼到 embedding 里。这样更贴合你的公式： $\text{score} = \text{qk}/\sqrt{d} + b_{\text{src}}*\text{same_source} + b_{\text{dst}}*\text{same_destination} + b_{\text{ep}}*\text{same_endpoint} - \text{lambda}*\text{age}$ ，并且仍然保留 base logits + gated delta 的保守结构，避免 Stage2 过度改写 Stage1C。

助手 2026-07-08 11:50:30

接下来我会改 5 个位置：`Dataset` 生成关系特征、`collate` 做同步 padding/truncation、`trainer` 只在新模型需要时传入关系特征、`target\_query.py` 加新模型、`run\_stage2.py` 暴露命令行参数。改动会保持向后兼容，旧模型不会收到第三个参数。

助手 2026-07-08 11:50:59

补丁没套上，是因为这个文件里原有中文注释编码显示有些乱，匹配上下文不稳定。我会用更小的 ASCII 上下文逐段套补丁。

助手 2026-07-08 11:51:31

现在 `Dataset` 已经能为普通 mixed context 生成 `[same\_source, same\_destination, same\_endpoint, age]`。这里的 `age` 是归一化位置年龄：当前 token 为 0，越旧越接近 1；这样 `lambda` 的大小更好调，不会因为 window=128 让 bias 数值过大。

助手 2026-07-08 11:51:47

同步 padding/truncation 也会跟着 `context\_z` 一起做，所以 relation bias 和每个历史 embedding 的位置不会错位。下一步我把 trainer 改成“只有新模型声明需要 relation_features 时才传第三个参数”。

助手 2026-07-08 11:53:22

新模型主体已经加进 factory 了。它的默认 bias 很保守：source 初始 +0.10、destination +0.05、endpoint 0、age penalty 0.10，训练中都会学习；这相当于给模型一个轻微先验，而不是硬规则。

助手 2026-07-08 11:54:15

命令行入口也加好了，所以你后面可以直接用 `--model\_type relation\_aware\_attention`，并且可以调 `--relation\_source\_bias\_init`、`--relation\_destination\_bias\_init`、`--relation\_endpoint\_bias\_init`、`--relation\_age\_lambda\_init`、`--relation\_bias\_scale`。我再补一下配置列名兼容和做一次编译/烟测。

助手 2026-07-08 11:54:24

还有一个小点：`Stage2Dataset` 以前不知道 config 里的列名，我会让它接收 `source\_col/destination\_col`，默认仍是 `source\_id/destination\_id`，这样如果你将来换 metadata 列名，relation bias 不会悄悄用错列。

助手 2026-07-08 11:54:31

我还会顺手让 `build\_datasets` 把 `cfg` 传下去，保持和 `ContextIndexBuilder` 使用同一套 `data.source\_col/data.destination\_col`。这属于小的兼容性修正，不改变你现有默认配置行为。

助手 2026-07-08 11:54:38

关系特征这部分只会在普通 mixed context 分支里生成；`source\_destination` 双路分支仍然走原来的双输入，不会受到这次新增字段影响。

助手 2026-07-08 11:54:51

`run\_stage2.py` 已经改好；我把 tuned 入口也同步一下，避免之后用 `run\_stage2\_tuned.py` 时出现两个脚本行为不一致。

助手 2026-07-08 11:55:10

代码路径都接上了。我现在做两件验证：先 `py\_compile` 抓语法问题，再构造一个很小的 fake batch，确认 relation-aware 模型输出有限值、shape 正常。

助手 2026-07-08 11:55:17

验证命令被当前 sandbox 拦住了，这是本地 Python 执行权限问题，不是代码报错。我会按审批流程重新跑同样的编译和烟测。

助手 2026-07-08 11:56:15

验证通过：新增文件的语法编译没问题，小型前向烟测输出是 `[3, 2]`，且所有 logits 都是 finite。现在我整理一下实现点、怎么跑、以及它为什么可能提升/可能不提升。

助手 2026-07-08 11:56:27

我核了一遍关键行：新增模型的 bias 是 per-head 可学习参数，每个 head 可以学出不同偏好；这比单个全局 `b\_src/b\_dst` 更灵活，但仍然是很轻量的参数，不会明显增加过拟合风险。

助手 2026-07-08 11:57:13

可以做，我已经在你的代码基础上加好了一个新模型类型：`relation\_aware\_attention`，尽量不动原有模型路线。

核心实现是：

```
score = qk / sqrt(d)
        + b_src * same_source
        + b_dst * same_destination
        + b_ep * same_endpoint
        - lambda_age * age
```

其中 `b\_src/b\_dst/b\_ep/lambda\_age` 都是可学习参数，而且是 per-head 的，也就是不同 attention head 可以学不同关系偏好。

改动位置：

- [dataset.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/dataset.py:47)
- 新增 `relation\_features = [same\_source, same\_destination, same\_endpoint, age]`。
- [dataset.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/dataset.py:169)

relation features 和 `context\_z` 同步 padding/truncation。

- [target_query.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/target_query.py:358)
新增 `Stage2RelationAwareAttention`。

- [target_query.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/target_query.py:497)
实现 relation bias。

- [target_query.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/target_query.py:568)
attention score 加入 relation bias。

- [trainer.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/trainer.py:145)
只有新模型声明 `uses\_relation\_features=True` 时才传第三个参数，旧模型不受影响。

- [model.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/model.py:813)
factory 支持 `relation\_aware\_attention`。

- [run_stage2.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/run_stage2.py:101)
CLI 支持新模型和 bias 初始化参数。

建议第一组实验用 mixed `endpoint` context:

```
!python /content/drive/MyDrive/s2/run_stage2.py \
--stage1_dir /content/drive/MyDrive/s1/0704/0704C_ar002_et12_20260511_001 \
--config /content/drive/MyDrive/s2/0708ar002_et12_20260511_001/endpoint_relation/stage2_config_used.yaml \
--out_dir /content/drive/MyDrive/s2/0708ar002_et12_20260511_001/endpoint_relation/ws128 \
--context_method endpoint \
--context_policy online \
--window_size 128 \
--model_type relation_aware_attention \
--context_scale 0.2 \
--gate_bias_init -2.0 \
--use_context_length_feature \
--relation_source_bias_init 0.10 \
--relation_destination_bias_init 0.05 \
--relation_endpoint_bias_init 0.00 \
--relation_age_lambda_init 0.10 \
--relation_bias_scale 1.0 \
--seed 130 \
--epochs 500 \
--batch_size 128
```

解析上，这个功能最可能提升的是 `endpoint` 或 `time\_only` 这种 mixed context，因为原来模型只看到一堆历史 embedding，不知道某条历史是同源、同目的、同端点还是只是时间上靠近。现在 attention 会优先看“关系更像”的历史流，同时对更旧的历史给轻微惩罚。

但它不一定比你当前最强的 `source\_destination\_attention + separate gate` 更好。原因是双路模型已经显式把 source/destination 拆开了；relation-aware bias 更像是给混合上下文补上关系结构。我的优先实验顺序是：

1. `endpoint + relation\_aware\_attention + ws128`
2. `endpoint + relation\_aware\_attention + ws64`
3. `time\_only + relation\_aware\_attention + ws128`
4. `endpoint + relation\_bias\_scale=0.5`
5. `endpoint + relation\_age\_lambda\_init=0.05`
6. `endpoint + relation\_endpoint\_bias\_init=0.05`

我已验证：

```
py_compile: passed
smoke forward: torch.Size([3, 2]), finite=True
```

所以代码层面可以直接跑。

用户 2026-07-08 12:21:25

Files mentioned by the user:

[INFO] Stage2 ----- seed: 130 [INFO] 随机种子已设置: 130 [INFO] Stage2 ----- d...:
C:\Users\XiaoyanXiong\codex/attachments/615da4e6-1782-4fce-a7c6-5aae1a00118b/pasted-text.txt

My request for Codex:

```
!python /content/drive/MyDrive/s2/run_stage2.py \  
--stage1_dir /content/drive/MyDrive/s1/0704/0704C_ar002_et12_20260511_001 \  
--config  
/content/drive/MyDrive/s2/0708ar002_et12_20260511_001/dual_host_attention/stage2_config_used.yaml  
\  
--out_dir /content/drive/MyDrive/s2/0708ar002_et12_20260511_001/dual_host_attention/separate_gate_  
ws128_src02_dst02 \  
--context_method source_destination \  
--context_policy online \  
--window_size 128 \  
--model_type source_destination_attention \  
--context_scale 0.2 \  
--source_context_scale 0.2 \  
--destination_context_scale 0.2 \  
--gate_bias_init -2.0 \  
--use_context_length_feature \  
--seed 130 \  
--epochs 500 \  
--batch_size 128
```

运行结果: Stage2 run summary

=====

best_epoch: 12
metric_for_best: pr_auc
best_score: 0.9136070305097918
context_method: source_destination
context_policy: online
window_size: 128

Final metrics:

train: loss=0.014378, acc=0.989138, macro_precision=0.956810, macro_recall=0.890122,
macro_f1=0.920614, weighted_precision=0.988737, weighted_recall=0.989138, weighted_f1=0.988710,
auc=0.9962332520613185, pr_auc=0.9426702809760865, precision_label1=0.9222291536702018,
recall_label1=0.7828773168578994, f1_label1=0.8468588886767233, cm=[[283251, 748], [2460, 8870]]
val: loss=0.022602, acc=0.986110, macro_precision=0.927902, macro_recall=0.896336,
macro_f1=0.911452, weighted_precision=0.985697, weighted_recall=0.986110, weighted_f1=0.985844,
auc=0.9922128098737344, pr_auc=0.913609912188287, precision_label1=0.8647342995169082,
recall_label1=0.798216276477146, f1_label1=0.8301449275362319, cm=[[40171, 224], [362, 1432]]
test: loss=0.027977, acc=0.983515, macro_precision=0.897428, macro_recall=0.896024,
macro_f1=0.896724, weighted_precision=0.983486, weighted_recall=0.983515, weighted_f1=0.983500,
auc=0.9910902260922356, pr_auc=0.88730242384894, precision_label1=0.8035357855717137,
recall_label1=0.8005681818181818, f1_label1=0.8020492386509179, cm=[[80171, 689], [702, 2818]]

结合我的代码和配置, 详细解析结果。还能继续提高性能吗? 为什么macro_recall基本上没有变化, 但是macro_pr

ecision却train-val-test变小，详细解析

助手 2026-07-08 12:21:39

我先把你这完整日志和当前双路模型代码再对一下，重点看阈值、混淆矩阵、source/destination scale 对 FP/FN 的影响。这个结果其实很接近你前一个 `src0.2\_dst0.1`，差异主要在 destination 分支权重加大后召回略升、precision 略降。

助手 2026-07-08 12:23:20

****总体结论****

这次 `src0.2\_dst0.2` 是一个很强但已经接近平台期的结果。按 `test f1\_label1` 看，它目前略高于你之前的 `src0.2\_dst0.1`；按 `test pr\_auc` 看，反而略低一点。所以不能说 `dst=0.2` 明显优于 `dst=0.1`，只能说它把模型往“多抓一点攻击”的方向推了一小步。

关键 test 结果：

model	P1	R1	F1_1	PR_AUC	AUC	CM
Stage1C	0.8069	0.7920	0.7994	0.8763	0.9890	[[80193,667],[732,2788]]
dual `src0.2_dst0.1`	0.8039	0.7991	0.8015	0.8877	0.9912	[[80174,686],[707,2813]]
dual `src0.2_dst0.2`	0.8035	0.8006	0.8020	0.8873	0.9911	[[80171,689],[702,2818]]

和 Stage1C 比：TP +30, FN -30, FP +22。也就是 Stage2 确实多抓到了一些攻击，但代价是多报了一点正常流。F1 +0.0026, PR_AUC +0.0110，这说明上下文分支对排序质量是真有帮助的。

和 `src0.2\_dst0.1` 比：TP +5, FN -5, FP +3。变化很小。`dst` 从 0.1 加到 0.2 后，recall 稍微提高，precision 稍微下降，符合 destination 分支之前表现出的“更偏召回”的特性。

****结合代码看****

你现在的模型在

[target_query.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/target_query.py:613) 里是双路结构：

```
summary_src = attention(current, source_history)
summary_dst = attention(current, destination_history)

logits =
    base_logits(current)
    + source_context_scale * source_gate * source_delta_logits
    + destination_context_scale * destination_gate * destination_delta_logits
```

这次配置里：

```
source_context_scale = 0.2
destination_context_scale = 0.2
gate_bias_init = -2.0
use_context_length_feature = True
window_size = 128
metric_for_best = pr_auc
```

`gate\_bias\_init=-2.0` 很重要，初始 gate 约等于 `sigmoid(-2)=0.119`，再加上 delta head zero init，模型一开始基本等于 Stage1 current-flow base branch，然后慢慢学习上下文修正。这就是为什么它比较稳，没有像早期 endpoint/time_only 那样大幅增加 FP。

你的 context 诊断也支持双路设计：

```
test label1 source_history_len mean = 117.88
test label0 source_history_len mean = 98.01

test label1 destination_history_len mean = 65.38
test label0 destination_history_len mean = 90.83
```

攻击流通常 source 历史更长，但 destination 历史更短。这个模式如果混成 endpoint，会被部分抵消；分开 source/destination 是合理的。

****为什么 macro_recall 基本没变，但 macro_precision 从 train 到 val/test 变小****

这个现象主要来自正类 precision 下降，而不是两个类别都在下降。

拆开看：

```
| split | P0 | R0 | P1 | R1 | macro_P | macro_R |
|---|---|---|---|---|---|---|
| train | ~0.9914 | ~0.9974 | 0.9222 | 0.7829 | 0.9568 | 0.8901 |
| val   | ~0.9911 | ~0.9945 | 0.8647 | 0.7982 | 0.9279 | 0.8963 |
| test  | ~0.9913 | ~0.9915 | 0.8035 | 0.8006 | 0.8974 | 0.8960 |
```

macro recall 是 $(R0 + R1) / 2$ 。从 train 到 test，`R1` 从 0.7829 升到 0.8006，但 `R0` 从 0.9974 降到 0.9915，两边抵消，所以 macro_recall 看起来几乎不动。

macro precision 是 $(P0 + P1) / 2$ 。`P0` 一直很稳，约 0.991；真正掉的是 `P1`：0.9222 -> 0.8647 -> 0.8035。也就是说模型在 test 上多把正常流判成攻击，导致正类 precision 明显下降。

从 FP rate 更直观：

```
train FP rate = 748 / 283999 = 0.263%
val   FP rate = 224 / 40395  = 0.555%
test  FP rate = 689 / 80860  = 0.852%
```

你的数据正类比例只有约 4%，所以哪怕 FP rate 只是从 0.26% 涨到 0.85%，precision 都会掉得很明显。因为 precision 的分母是 `TP + FP`，正类本来就少，FP 多一点影响很大。

****还能怎么提高****

优先级最高的不是大改模型，而是继续做很小的、有方向的 ablation：

1. 保留这个结构，扫 scale：

- `src0.2\_dst0.05`
- `src0.2\_dst0.1`
- `src0.2\_dst0.15`
- `src0.25\_dst0.1`
- `src0.3\_dst0.1`

2. 试 `window\_size=64`。你现在大量样本历史已经顶到 127，长历史可能带来 FP 噪声。ws64 很可能提升 precision，尤其是 test precision。

3. 如果目标是 `F1\_1`，可以把 `metric\_for\_best` 从 `pr\_auc` 改成 `f1\_label1`。日志里 epoch 22 的 val F1=0.8340，高于 best_epoch 12 的 0.8301，但因为 PR_AUC 没有更高，所以没被选中。

4. 降低训练正类推动力：你现在 `sampler\_pos\_frac=0.080`，`alpha=[1.0,1.05]`。如果想减少 FP，可以试：

- `sampler\_pos\_fraction=0.06`
- `alpha=[1.0,1.0]`
- 或 threshold metric 改成 `f0.5\_label1`

5. 给 source/destination 分开 gate bias。现在两个 gate 都是 -2.0 。可以尝试:

- source gate bias -2.0

- destination gate bias `-2.5` 或 -3.0``

这个很符合你现在的结果：destination scale 加大后 recall 涨一点，但 precision 下降一点。

6. 打开 `use\_multiplicative\_fusion=True` 做一次实验。当前是 `False`，source/destination branch 只用

`[current, summary, current-summary] + length`, 加上 `current\*summary`

可能捕捉“当前流和历史模式是否相似”。

我的判断：当前模型已经比 Stage1C 有稳定但不大的提升；后续提升空间还有，但大概率是 $0.001 \sim 0.004 F1$

这种级别。下一步最值得跑的是 `ws64 + src0.2\_dst0.1` 和 `ws128 + src0.25\_dst0.1`。

用户 2026-07-08 12:28:42

```
!python /content/drive/MyDrive/s2/run_stage2.py \
--stage1_dir /content/drive/MyDrive/s1/0704/0704C_ar002_et12_20260511_001 \
--config /content/drive/MyDrive/s2/0708ar002_et12_20260511_001/relation_aware_attention/stage2_config_used.yaml \
--out_dir /content/drive/MyDrive/s2/0708ar002_et12_20260511_001/relation_aware_attention/src/ws128 \
--context_method source_destination \
--context_policy online \
--window_size 128 \
--model_type relation_aware_attention \
--context_scale 0.2 \
--gate_bias_init -2.0 \
--use_context_length_feature \
--relation_source_bias_init 0.10 \
--relation_destination_bias_init 0.05 \
--relation_endpoint_bias_init 0.00 \
--relation_age_lambda_init 0.10 \
--relation_bias_scale 1.0 \
--seed 130 \
--epochs 500 \
--batch_size 128
[INFO] Stage2 ----- Training
[Stage2Trainer]---fit lr=3e-05, weight_decay=0.0003, patience=15, threshold=0.5, metric_for_best=pr_auc
Traceback (most recent call last):
File "/content/drive/MyDrive/s2/run_stage2.py", line 451, in <module>
main()
File "/content/drive/MyDrive/s2/run_stage2.py", line 358, in main
trainer.fit()
File "/content/drive/MyDrive/s2/stage2/trainer.py", line 436, in fit
train_loss = train_one_epoch(
^^^^^^^^^^^^^^^^^^^^
File "/content/drive/MyDrive/s2/stage2/trainer.py", line 200, in train_one_epoch
logits = forward_stage2_model(model, batch, device)
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/content/drive/MyDrive/s2/stage2/trainer.py", line 141, in forward stage2 model
```


****正确解决方式 1：跑 relation-aware mixed context****

我之前给你的 `relation\_aware\_attention` 是为 ****mixed context**** 设计的，最推荐搭配 `endpoint`，不是 `source\_destination`。

改成：

```
!python /content/drive/MyDrive/s2/run_stage2.py \  
  --stage1_dir /content/drive/MyDrive/s1/0704/0704C_ar002_et12_20260511_001 \  
  --config /content/drive/MyDrive/s2/0708ar002_et12_20260511_001/relation_aware_attention/stage2_config_used.  
yaml \  
  --out_dir /content/drive/MyDrive/s2/0708ar002_et12_20260511_001/relation_aware_attention/endpoint/ws128 \  
  --context_method endpoint \  
  --context_policy online \  
  --window_size 128 \  
  --model_type relation_aware_attention \  
  --context_scale 0.2 \  
  --gate_bias_init -2.0 \  
  --use_context_length_feature \  
  --relation_source_bias_init 0.10 \  
  --relation_destination_bias_init 0.05 \  
  --relation_endpoint_bias_init 0.00 \  
  --relation_age_lambda_init 0.10 \  
  --relation_bias_scale 1.0 \  
  --seed 130 \  
  --epochs 500 \  
  --batch_size 128
```

这个才符合 relation-aware attention 的设计：在一个 mixed endpoint history 里，每条历史 flow 会有：

```
same_source  
same_destination  
same_endpoint  
age
```

然后 attention score 加关系 bias。

****正确解决方式 2：继续跑 source/destination 双路****

如果你想继续使用：

```
--context_method source_destination
```

那模型应该用：

```
--model_type source_destination_attention
```

也就是你刚刚跑过的 separate gate 双路模型。这个路线目前是你结果最稳、也最强的一条。

****不要这样搭配****

```
--context_method source_destination  
--model_type relation_aware_attention
```

因为 `source\_destination` 已经把 source/destination 分开成两个输入分支了；而 `relation\_aware\_attention` 是给“混在一起的 context”加关系 bias。二者的数据格式不同。

****补充判断****

如果你用 `source\_host + relation\_aware\_attention`，也能跑，但意义不大，因为 source_host 里的历史 token 基本全是 `same\_source=1`，这个 bias 对同一条样本内所有 token 几乎是常数，softmax 里会被抵消。`relation\_aware\_attention` 最值得跑的是：

```
endpoint + relation_aware_attention  
time_only + relation_aware_attention
```

尤其是 `endpoint`。`source\_destination` 这条线继续用 `source\_destination\_attention`。

用户 2026-07-08 13:59:03

Files mentioned by the user:

```
## [INFO] Stage2 ----- seed: 130 [INFO] 随机种子已设置: 130 [INFO] Stage2 ----- d...:
C:\Users\XiaoyanXiong\.codex/attachments/e4b78a48-e662-472a-b80d-1cdeff3adb1e/pasted-text.txt
```

My request for Codex:

```
!python /content/drive/MyDrive/s2/run_stage2.py \
--stage1_dir /content/drive/MyDrive/s1/0704/0704C_ar002_et12_20260511_001 \
--config /content/drive/MyDrive/s2/0707ar002_et12_20260511_001/src_newM/stage2_config_used.yaml \
--out_dir /content/drive/MyDrive/s2/0707ar002_et12_20260511_001/src_newM/ws128 \
--context_method source_host \
--context_policy online \
--window_size 128 \
--model_type target_query_gated \
--seed 130 \
--epochs 500 \
--batch_size 128
```

运行结果。结合我的代码和配置，如何precision_label1让他从train-val-test
不会掉的太多同时要保证recall_label1不变差。详细解析

助手 2026-07-08 13:59:18

我先把这次 `source\_host + target\_query\_gated` 的完整日志读一下，尤其看阈值、oracle
threshold、train/val/test 混淆矩阵和配置细节。precision 从 train 到 test
掉得多，一般不是单一原因，通常是阈值迁移、正类先验差异、context 分布差异和模型容量共同作用。

助手 2026-07-08 13:59:31

日志里这个模型的 test 阈值迁移 gap 只有 `0.0011`，说明“阈值从 val 迁到
test”本身不是最大问题。更大的问题是：test 上正常样本更容易被 source_host 上下文推成正类，FP 明显多于
Stage1C，所以 precision 掉了。接下来我看一下尾部有没有 train/val/test final 全指标，方便精确拆 FP rate。

助手 2026-07-08 14:00:58

****核心结论****

你这次 `source\_host + target\_query\_gated` 的问题不是 recall 差，而是 ****test 上 FP 增多****，导致
`precision\_label1` 从 train -> val -> test 明显下降。

你现在的的结果是：

```
| split | P1 | R1 | F1_1 | CM |
|---|---|---|---|---|
| train | 0.9197 | 0.7991 | 0.8552 | [[283209,790],[2276,9054]] |
| val   | 0.8566 | 0.8088 | 0.8320 | [[40152,243],[343,1451]] |
| test  | 0.7945 | 0.8074 | 0.8009 | [[80125,735],[678,2842]] |
```

recall 其实很稳：`0.799 -> 0.809 -> 0.807`。precision 掉，是因为 FP rate 在涨：

```
train FP rate = 790 / 283999 = 0.278%
```

```
val   FP rate = 243 / 40395 = 0.602%
test  FP rate = 735 / 80860 = 0.909%
```

正类只有约 4%，所以 FP rate 从 `0.28%` 涨到 `0.91%`，precision 会掉很多。这个是类别不平衡下很典型的现象。

****为什么你的 target_query_gated 会这样****

在 [target_query.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/target_query.py:126) 里，`target\_query\_gated` 是：

```
current
summary = attention(current, source_history)
fused = current + gate * context_update
logits = cls_head(fused)
```

它的问题是：****context 直接改写 current representation****。没有 `base\_logits(current)` 保护，也没有 `context\_scale` 限制。也就是说，一旦 source history 在 train 上学到“某些 source 历史模式像攻击”，到 test 上正常流也有类似 source 历史，就容易被推成正类。

更关键：你传的这个模型 `target\_query\_gated` 目前不使用：

```
context_scale
gate_bias_init
use_context_length_feature
```

这些保守控制主要在 `target\_query\_residual` 和 `source\_destination\_attention` 里起作用。所以如果想减少 test FP，`target\_query\_gated` 不是最理想结构。

****如果要求 recall 不比当前 0.807 差****

只靠调阈值很难。因为 test oracle 结果显示：

```
val threshold = 0.6386: P1=0.7946, R1=0.8077
test oracle   = 0.6237: P1=0.7876, R1=0.8173
```

降低阈值能提高 recall，但 precision 更低。提高阈值能提高 precision，但 recall 很可能下降。所以要同时保持 recall 和提高 precision，重点应该是 ****减少 FP，而不是单纯调 threshold****。

****最推荐的改法****

第一优先：把 `target\_query\_gated` 换成 `target\_query\_residual`。

```
--model_type target_query_residual \
--context_scale 0.1 \
--gate_bias_init -3.0 \
--use_context_length_feature
```

或者稍微激进一点：

```
--model_type target_query_residual \
--context_scale 0.15 \
--gate_bias_init -2.5 \
--use_context_length_feature
```

原因：`target\_query\_residual` 是：

```
logits = base_logits(current) + context_scale * gate * delta_logits
```

它不会让 context 直接重写 current embedding，只允许 context 做小幅 logit 修正。这个结构更适合你的目标：****precision 掉得少，同时保留 source_host 带来的 recall 提升****。

第二优先：试 `window\_size=64`。

你的 source history 分布里，test/val 大量样本已经顶到 127：

```
test mean source_history = 98.84
test p50 = 127
test p95 = 127
```

长历史会带来更多“看起来像攻击的正常 source 模式”，容易增加 FP。`ws64` 可能减少旧历史噪声，同时保留近期 source 行为。

推荐命令：

```
--context_method source_host \
--window_size 64 \
--model_type target_query_residual \
--context_scale 0.15 \
--gate_bias_init -2.5 \
--use_context_length_feature
```

第三优先：降低训练时正类推动力。

你现在是：

```
sampler_pos_fraction = 0.080
manual_alpha = [1.0, 1.05]
focal_gamma = 1.0
```

这套配置偏向保护 recall，但会增加 FP。可以试：

```
training:
  sampler_pos_fraction: 0.06
  alpha: [1.0, 1.0]
  focal_gamma: 1.0
```

如果 recall 掉太多，再把 `sampler\_pos\_fraction` 回到 `0.07`。

****如果目标是不比 Stage1C recall 差****

Stage1C test recall 是 `0.7920`。那你空间更大。当前最好的双路 separate gate 已经更稳：

```
source_destination src0.2 dst0.1:
P1=0.8039, R1=0.7991, F1=0.8015
```

它 precision 比 source_host 高，recall 也高于 Stage1C，但略低于 source_host 当前的 `0.8074`。如果你的“recall 不变差”是指不低于 Stage1C，那双路模型更推荐。

****最值得跑的实验顺序****

1. `source\_host + target\_query\_residual + ws128 + context\_scale=0.15 + gate\_bias=-2.5`
2. `source\_host + target\_query\_residual + ws64 + context\_scale=0.15 + gate\_bias=-2.5`
3. `source\_host + target\_query\_residual + ws128 + context\_scale=0.1 + gate\_bias=-3.0`
4. `source\_host + target\_query\_residual + ws64 + context\_scale=0.2 + gate\_bias=-2.0`
5. 当前 `target\_query\_gated` 保持不变，但训练改成 `sampler\_pos\_fraction=0.06, alpha=[1.0,1.0]`

我的判断：想让 train-val-test precision gap 缩小，最有效的是 ****不要继续用原始 target_query_gated 作为主模型****，而是换成更保守的 `target\_query\_residual`。它更符合你的目标：少增加 FP，同时尽量保住 source_host 带来的 recall。

用户 2026-07-08 14:15:16

以为现在的模型和结果，如果我想 F1_class1, R1_class1, P1_class1 都提高到 0.85, 有可能吗？想详细解析

助手 2026-07-08 14:15:47

短答: **只靠现在这套 Stage2 调参, 把 `P1/R1/F1\_class1` 同时推到 0.85, 可能性不高**。不是完全不可能, 但需要明显提升排序能力, 不是调 threshold、window size、scale 这种小改能做到的级别。

你现在最强附近大概是:

```
| 模型 | P1 | R1 | F1_1 | CM |
|---|---|---|---|---|
| Stage1C | 0.8069 | 0.7920 | 0.7994 | [[80193,667],[732,2788]] |
| source_host target_query_gated | 0.7945 | 0.8074 | 0.8009 | [[80125,735],[678,2842]] |
| dual src0.2 dst0.1 | 0.8039 | 0.7991 | 0.8015 | [[80174,686],[707,2813]] |
| dual src0.2 dst0.2 | 0.8035 | 0.8006 | 0.8020 | [[80171,689],[702,2818]] |
```

如果要 `P1=R1=0.85`, test 集正类有 3520 个, 负类 80860 个, 大概需要:

```
TP = 2992
FN = 528
FP = 528
TN = 80332
```

而你当前最好附近是:

```
TP = 2818
FN = 702
FP = 689
TN = 80171
```

所以你需要同时做到:

```
TP +174
FN -174
FP -161
```

这很难, 因为通常提高 recall 会增加 FP, 提高 precision 会增加 FN。你现在 test oracle threshold 也只能把 F1 推到约 `0.802`, 说明**不是阈值没选好, 而是分数排序本身还不够把 0/1 拉开**。

从目前结果看, Stage2 确实有提升, 但提升幅度是:

```
F1_class1: 0.7994 -> 0.8020
PR_AUC:    0.8763 -> 0.8877
```

这是“稳定小提升”, 不是朝 0.85 突破的迹象。想上 0.85, 需要更强的信息源或训练目标变化。

最可能接近 0.85 的方向:

1. **Stage1 先变强**

Stage2 是基于 Stage1 embedding 修正。如果 Stage1 单流已经卡在 $F1 \approx 0.799$, Stage2 只靠历史上下文很难凭空补出 0.05。优先尝试 Stage1 更强 encoder、更多特征、hard negative mining。

2. **显式减少 FP 的训练目标**

加 `F0.5` threshold、precision-constrained threshold, 或者训练时降低 `sampler\_pos\_fraction` / `alpha`, 但这通常会牺牲 recall。

3. **做 hard negative mining**

把当前 FP 的正常流保存出来, 分析它们的 source/destination/history 特征, 再针对这些样本加权训练。这个比盲目改 attention 更可能提升 precision。

4. **融合 Stage1C + Stage2**

不让 Stage2 完全替代 Stage1, 而是做 ensemble/stacking:

```
final_prob = w * stage1_prob + (1-w) * stage2_prob
```

或者只在 Stage1 不确定区间调用 Stage2。这个有机会减少 FP，同时保留一部分 recall。

5. **增加更强上下文结构**

比如 relation-aware endpoint、source/destination separate gate、recency bias、history top-k，而不是简单 full window attention。

我的判断：

****短期目标合理设为 $P1 \approx 0.81-0.83$, $R1 \approx 0.80-0.82$, $F1 \approx 0.805-0.815$ 。****

$0.85/0.85/0.85$ 属于明显更高一级，需要 Stage1、数据特征、训练策略一起升级，单靠现在 Stage2 架构微调不太现实。

用户 2026-07-08 14:18:52

如何代码，使我的短期目标能够实现？详细解析

助手 2026-07-08 14:19:06

我会把“短期目标”拆成可落地的代码目标：少 FP、尽量不牺牲

R1。最直接的改法不是再换一个大模型，而是给你现有训练/评估加两个控制杆：`precision-with-recall-floor` 阈值选择，以及 source/destination 分开 gate bias，这样可以把 precision 往上拉，同时用 recall 下限约束住不让他掉太多。

助手 2026-07-08 14:19:33

我会直接做这些代码增强，并保持默认行为不变：不传新参数时，你之前的实验结果路径不变。新功能会让你可以用 `threshold_metric=precision_at_recall` 选阈值、给 source/destination 单独 gate bias、以及可选打开 multiplicative fusion；这些都是为了把 FP 压下来，同时用 recall floor 护住 R1。

助手 2026-07-08 14:19:58

阈值选择函数已经扩展了：现在支持

`precision_at_recall`、`f1_at_recall`、`f0.5_at_recall`、`f0.5_label1`。这不是测试集调参，它仍然只在 val 上选阈值；只是 val 上会优先找“召回不低于某个下限时 precision 最高”的阈值。

助手 2026-07-08 14:20:24

我继续把这个阈值搜索接到训练流程里。这样 `fit()` 每个 epoch 选 val threshold、final evaluate、oracle diagnosis 三处都会用同一套阈值策略，避免训练中和最终评估不一致。

助手 2026-07-08 14:20:47

现在阈值约束已经接到训练和最终评估了。下一步我会给双路模型加 `source_gate_bias_init` / `destination_gate_bias_init`，这样可以让 destination 分支更保守一点，减少它带来的 FP，而 source 分支仍然保留召回贡献。

助手 2026-07-08 14:21:26

`run_stage2.py` 里已经有这些命令行参数了：`--threshold_metric precision_at_recall`、`--threshold_recall_floor 0.80`、`--source_gate_bias_init`、`--destination_gate_bias_init`、`--use_multiplicative_fusion`。我把 tuned

入口也同步，避免两个脚本跑出来能力不一致。

助手 2026-07-08 14:21:57

我再加一组训练侧的 CLI 控制：`sampler\_pos\_fraction` 和 focal/alpha。你的 FP 偏多时，最常用的短期操作就是把训练采样里的正类比例从 0.08 轻微降到 0.06 或 0.07；这个以前只能改 YAML，现在可以直接从命令行实验。

助手 2026-07-08 14:22:12

采样和 focal alpha 的命令行覆盖也加上了；我同步到 tuned 脚本，然后做一次编译和小型阈值函数烟测，确认新参数路径没有语法问题。

助手 2026-07-08 14:22:47

改动完成了。我现在跑语法编译和一个小的阈值选择测试：用假分数确认 `precision\_at\_recall` 会在满足 recall floor 的阈值里挑 precision 更高的那个。

助手 2026-07-08 14:23:11

编译通过了。小烟测超时主要是导入 `trainer.py` 会拉起 torch/sklearn，10 秒不够；我加长一点时间再跑一次。

助手 2026-07-08 14:23:39

烟测也通过：示例里它选到了 precision=1.0 且 recall=0.75 的阈值，说明“先保 recall floor，再尽量提高 precision”的逻辑是生效的。我再快速列一下改动点，方便给你可直接跑的命令。

助手 2026-07-08 14:24:22

我已经把代码加好了，目标就是服务你这个短期目标：

```
P1 ≈ 0.81-0.83
R1 ≈ 0.80-0.82
F1_class1 ≈ 0.805-0.815
```

这不是保证一定到，但现在代码上有了更合理的控制杆。

****我改了什么****

1. 在 [trainer.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/trainer.py:299)

加了新的阈值选择策略：

```
precision_at_recall
f1_at_recall
f0.5_at_recall
recall_at_precision
f0.5_label1
```

最重要的是：

```
threshold_metric = precision_at_recall
threshold_recall_floor = 0.80
```

含义是：在 val 上先保证 `recall\_label1 >= 0.80`，然后在满足这个条件的阈值里选 precision 最高的。这样比单纯 `f1\_label1` 更适合你现在的的问题，因为你现在主要是 FP 偏多、precision 掉得多。

2. 在 [target_query.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/target_query.py:738) 给双路模型加了：

```
source_gate_bias_init
destination_gate_bias_init
```

以前 source/destination 两个 gate 都用同一个 `gate\_bias\_init=-2.0`。现在可以让 destination 更保守，例如：

```
source_gate_bias_init = -2.0
destination_gate_bias_init = -3.0
```

这样 destination 分支不容易把正常流推成攻击，有助于减少 FP。

3. 在 [run_stage2.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/run_stage2.py:157) 加了命令行参数：

```
--source_gate_bias_init
--destination_gate_bias_init
--use_multiplicative_fusion
--metric_for_best
--threshold_metric
--threshold_recall_floor
--threshold_precision_floor
--threshold_fbeta_beta
--sampler_pos_fraction
--focal_gamma
--negative_alpha
--positive_alpha
```

也同步到了 `run\_stage2\_tuned.py`。

已验证：

```
py_compile: passed
precision_at_recall smoke test: passed
```

****最推荐你先跑这个****

基于你目前最稳的双路模型：

```
!python /content/drive/MyDrive/s2/run_stage2.py \
  --stage1_dir /content/drive/MyDrive/s1/0704/0704C_ar002_et12_20260511_001 \
  --config /content/drive/MyDrive/s2/0708ar002_et12_20260511_001/dual_host_attention/stage2_config_used.yaml \
  --out_dir /content/drive/MyDrive/s2/0708ar002_et12_20260511_001/dual_host_attention/
ws128_src02_dst01_precision_at_recall80 \
  --context_method source_destination \
  --context_policy online \
  --window_size 128 \
  --model_type source_destination_attention \
  --source_context_scale 0.2 \
  --destination_context_scale 0.1 \
  --gate_bias_init -2.0 \
  --source_gate_bias_init -2.0 \
  --destination_gate_bias_init -3.0 \
  --use_context_length_feature \
  --threshold_metric precision_at_recall \
  --threshold_recall_floor 0.80 \
  --seed 130 \
  --epochs 500 \
  --batch_size 128
```

这组的逻辑是：保持你目前最好的 `src0.2\_dst0.1`，但让 destination gate 更保守，同时阈值选择不再盲目追 F1，而是 “R1 至少 0.80，然后尽量提高 P1”。

****第二组实验****

如果第一组 P1 仍不够，就继续降低训练时正类推动力：

```
--sampler_pos_fraction 0.06 \  
--negative_alpha 1.0 \  
--positive_alpha 1.0
```

完整思路是：你现在 FP 偏多，说明训练对正类略积极。`sampler\_pos\_fraction=0.06` 会比当前 0.08 更保守，通常 precision 会升，recall 可能小幅降，所以配合 `threshold\_recall\_floor 0.80` 比较稳。

****第三组实验****

如果 recall 掉了，再把 source 加强一点：

```
--source_context_scale 0.25 \  
--destination_context_scale 0.1 \  
--source_gate_bias_init -2.0 \  
--destination_gate_bias_init -3.0
```

source 分支在你数据里更像“召回贡献”，destination 分支更容易带 FP，所以这个方向比简单把两个 scale 都加大更合理。

****为什么这些代码更接近短期目标****

你当前最好的附近是：

```
P1 ≈ 0.803  
R1 ≈ 0.800  
F1 ≈ 0.802
```

要到短期目标，不需要大幅提高 TP，主要需要把 FP 从大约 `686-735` 压到更低，同时别让 FN 明显上升。新代码做的正是这件事：

```
precision_at_recall:      R1    P1  
destination_gate_bias_init=-3.0:    destination    FP  
sampler_pos_fraction=0.06:
```

如果这几组都只能把 F1 推到 `0.803-0.806`，那说明短期目标上沿 `0.815` 需要更强的 Stage1 或 hard negative mining；但这些改动是目前最该先做的。

用户 2026-07-08 14:59:42

如果stage2做不到，我该如何考虑修正stage1呢？有什么好的方案吗？

助手 2026-07-08 14:59:57

我会把 Stage1 当成“真正决定上限”的部分来看：Stage2 现在只能做小幅上下文修正，想把 class1 的 P/R/F1 一起推高，Stage1 的单流表征和 hard negative 区分能力必须先变强。我先快速看一下你 `s1` 的模型、loss、训练入口，给你更贴合现有代码的改法。

助手 2026-07-08 15:00:17

我看到 Stage1 里其实已经有一些很有用的基础件：`asymmetric`、`cb\_focal`、`hard\_neg\_ce`、方案 C 的 packet+flow fusion、attention pooling。也就是说，下一步不是“推倒重写”，而是把 Stage1 训练目标从普通 F1 优化，转向“hard negative + 排序能力 + 更稳的单流表示”。

助手 2026-07-08 15:01:04

可以，而且我建议你吧 Stage1 当成现在的“性能天花板”来修。Stage2 目前只是利用上下文把 Stage1 的 embedding/概率做小幅修正；如果 Stage1 本身对单条 flow 的 0/1 排序不够好，Stage2 很难把 `P1/R1/F1` 同时推到更高。

****先看目标差距****

Stage1C test 是:

```
P1 = 0.8069
R1 = 0.7920
F1 = 0.7994
CM = [[80193, 667], [732, 2788]]
```

如果想 $P1 \approx R1 \approx 0.85$, test 正类 3520 个, 负类 80860 个, 大约需要:

```
TP ≈ 2992
FN ≈ 528
FP ≈ 528
```

和 Stage1C 比, 需要:

```
TP +204
FN -204
FP -139
```

这说明不是简单调 threshold

能解决。你需要让模型排序能力真正变强: 更多攻击样本排到正常样本前面, 同时减少正常样本被打高分。

****第一步: 先做 Stage1 错误诊断****

不要先盲目改模型。先把 Stage1 的错误拆成四类:

```
TP:
FP:
FN:
TN:
```

重点分析:

```
FP  FN      protocol / port / duration / packet_count / byte_count / flow_length / iat / flag
FP              source/destination
FN              payload
```

如果 FP/FN 集中在特定流形态, Stage1 可以修; 如果 FP/FN 和 TP/TN 特征高度重叠, 0.85 就可能需要新特征或更干净标签。

****最推荐的 Stage1 方向 1: Hard Negative Mining****

你的代码里已经有这个 loss:

[losses.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s1/stage1/losses.py:245)

```
class HardNegativeMiningCELoss
```

它的意义是: 保留所有正类样本, 同时只让模型重点学习最容易被误报的负类。你的问题正是 FP 偏多, 所以这个非常适合。

建议先试:

```
training:
  loss: hard_neg_ce
  neg_keep_ratio: 0.30
  label_smoothing: 0.0
  alpha_mode: none
```

如果 precision 上升但 recall 下降, 再试:

```
neg_keep_ratio: 0.40
label_smoothing: 0.02
```

这个方向比继续加大 focal 正类权重更合理。因为你现在不是抓不到攻击, 而是正常样本被打高分太多。

****方向 2: Asymmetric Focal Loss****

你的 Stage1 也已有 asymmetric loss:

[trainer.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s1/stage1/trainer.py:86)

建议试:

```
training:
  loss: asymmetric
  alpha_mode: none
  asl_gamma_pos: 0.0
  asl_gamma_neg: 3.0
  asl_alpha_pos: 0.50
```

含义: 不要过度压正类; 重点压难负类。

如果 recall 掉太多, 把:

```
asl_alpha_pos: 0.55
asl_gamma_neg: 2.0
```

****方向 3: 改 Stage1 的 threshold 选择策略****

你 Stage1 里现在 `find\_optimal\_threshold` 最后仍然返回 F1 最优阈值:

[trainer.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s1/stage1/trainer.py:768)

它虽然检查了 `target\_precision=0.85,target\_recall=0.85`, 但最后还是:

```
best_result = max(results, key=lambda x: x['f1'])
return best_result['threshold'], results
```

所以这里可以改成和 Stage2 一样:

```
precision_at_recall
f1_at_recall
recall_at_precision
```

例如短期目标可以用:

```
val    recall_label1 >= 0.80    precision_label1    threshold
```

这不能改变排序能力, 但能让部署阈值更符合你的目标, 而不是盲目最大化 F1。

****方向 4: Stage1 做 ensemble/stacking****

你 repo 里已经有 flow-level baseline, 比如 LGBM/ExtraTrees 的结果目录。这个很重要。

如果 tree baseline 对 flow features 的 P/R/F1 比 Transformer 更高, 说明 packet Transformer 没充分利用 flow 统计特征。建议做融合:

```
final_prob = w * transformer_prob + (1-w) * lgbm_prob
```

或者把 LGBM prob 当成 Stage1 额外特征输入 MLP/融合头。

这个方向很可能比继续调 Transformer 层数更有效, 因为树模型对 tabular flow features 很强。

****方向 5: 改 Stage1 模型结构, 但要保守****

你的 Stage1 已经有方案 C、flow fusion、attention pooling:

- [AttentionPooling](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s1/stage1/model.py:304)
- [FlowFusion](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s1/stage1/model.py:419)
- [LogitResidualFusion](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s1/stage1/model.py:565)

我最推荐的是 ****logit residual fusion**** 思路:

```
packet_logits + small_gate * flow_logits
```

不要让 flow feature 直接大幅改写 packet embedding。这样更稳，和你 Stage2 里 residual correction 的经验一致。

如果当前 Stage1C 不是用 `LogitResidualFusion`，可以优先把 C 改成这个方向，而不是 concat/gated embedding fusion。

****方向 6: 增加 auxiliary heads****

一个更强的 Stage1 设计是同时输出：

```
packet_head logits
flow_head logits
fused_head logits
```

训练 loss：

```
loss = fused_loss + 0.3 * packet_loss + 0.3 * flow_loss
```

好处是：packet 分支和 flow 分支都被监督，不会一个分支被另一个分支盖掉。对 Stage2 也有好处，因为导出的 embedding 更稳定。

****推荐实验顺序****

1. Stage1C 原结构 + `loss=hard\_neg\_ce, neg\_keep\_ratio=0.30`
2. Stage1C 原结构 + `loss=asymmetric, asl\_gamma\_neg=3.0, asl\_alpha\_pos=0.50`
3. Stage1C + threshold 改成 `precision\_at\_recall\_floor=0.80`
4. Stage1C + `LogitResidualFusion`
5. Stage1C + auxiliary packet/flow/fused heads
6. Transformer prob + LGBM prob ensemble
7. hard negative mining 二阶段训练：先用当前 Stage1C 训练，再用 FP hard negatives 微调 5-10 epoch

****我的判断****

如果 Stage1 不改数据和特征，只改 loss/threshold，可能把：

```
F1_class1: 0.799 -> 0.805~0.815
```

如果加 hard negative mining + logit residual fusion + tree ensemble，有机会到：

```
P1/R1/F1: 0.82
```

但要同时到 `0.85/0.85/0.85`，大概率需要：

```
flow features

hard negative
Transformer + tree ensemble
```

最该先做的是 hard negative mining，因为它正好针对你现在的核心问题：FP 太多。

用户 2026-07-08 15:03:42

stage1应该做到尽可能的将0和1分开，让生成的embedding，尽可能的可分。详细解析

助手 2026-07-08 15:04:22

对，你这个判断是对的：****Stage1 的核心任务不只是把当前 flow 分类对，而是生成一个对 0/1 尽可能可分、对 val/test 也稳定的 embedding****。Stage2 后面吃的就是这个 embedding；如果 Stage1 embedding 里 0/1 已经混在一起，Stage2 再怎么 do attention，也只能做小修小补。

但这里有一个关键点：

****不是让 train embedding 分得越开越好，而是让 val/test embedding 也分得开。**** 否则就是过拟合。

****Stage1 应该优化什么****

现在 Stage1 基本是：

```
packet sequence / flow features -> z_embedding -> classifier logits
loss = CE/Focal/Asymmetric      loss
```

分类 loss 只要求 logits 能分对，不一定要求 embedding 本身结构好。所以会出现：

```
embedding    0/1
Stage2 embedding
```

你想要的是：

```
label 0 embedding
label 1 embedding
0    1    embedding
      hard negatives
```

****最推荐的改法：CE/Focal + Supervised Contrastive Loss****

Stage1 loss 可以改成：

```
loss = classification_loss +  $\lambda$  * supervised_contrastive_loss(z, y)
```

其中：

```
classification_loss:
supervised_contrastive_loss:      embedding
```

直觉是：

```
z
z
batch
```

这是最贴合你目标的改法。

推荐初始配置：

```
training:
  loss: focal
  alpha_mode: mild
  focal_gamma: 1.0
  label_smoothing: 0.0
  use_supcon: true
  supcon_weight: 0.05
  supcon_temperature: 0.10
```

如果分类指标不掉，再试：

```
supcon_weight: 0.10
```

如果 recall 掉，降回：

```
supcon_weight: 0.02
```

****第二个好方案：Center Loss****

Center loss 更简单：

```
loss = classification_loss +  $\lambda$  * center_loss(z, y)
```

它让每个类别 embedding 靠近自己的类别中心。优点是稳定，缺点是攻击类可能不是一个单峰分布。攻击类型如果很多，强行拉成一个中心可能反而伤 recall。

所以它适合小权重：

```
center_loss_weight: 0.001 ~ 0.01
```

我会把它排在 SupCon 后面。

****第三个方案：ArcFace / CosFace margin classifier****

普通 classifier 是：

```
logits = W z
```

ArcFace/CosFace 会要求正确类别不仅分对，还要有 margin：

```
class 1      class 0
```

这会让 embedding 的角度空间更可分。对 Stage2 embedding 很有价值。

推荐方向：

```
normalize z
normalize classifier weight
cosine margin
```

但这个比 SupCon 改动更大，建议作为第二阶段尝试。

****不要只做一件事：盲目加大 class1 权重****

加大 class1 权重通常会：

```
recall
FP
precision
embedding          1
```

你现在的問題已經是 FP 控制困難，所以 Stage1 不應該繼續簡單“更偏正類”。應該做的是：

```
hard negative
embedding margin
```

****必須加的診斷****

每次 Stage1 訓練後，除了看 P/R/F1，還要看 embedding 可分性：

```
1. train/val/test   PCA/UMAP/t-SNE
2. class centroid distance
3. intra-class distance:
4. inter-class distance:
5. Fisher ratio =      /
6.  z      LogisticRegression / KNN   embedding
7. FP/FN   embedding
```

如果 train 分得很開，test 仍然混，說明過擬合。

如果 val/test 的 Fisher ratio 提高，同時 PR_AUC 提高，說明 Stage1 embedding 真的變好了。

****我的推薦路線****

優先級最高：

```
Stage1C + SupConLoss
Stage1C + hard_neg_ce
Stage1C + asymmetric focal
Stage1C + SupConLoss + hard negative sampler
Stage1C + ArcFace/CosFace
```

最推薦第一版目標：

```
classification_loss + 0.05 * SupConLoss(z, y)
```

因为它直接优化你想要的东西: **让 Stage1 embedding 的 0/1 更可分**。

****最终判断****

Stage2 做不到大幅突破时, Stage1 应该从“分类器优化”升级成“分类 + 表征可分性优化”。你现在最该加的不是更复杂 attention, 而是:

Supervised Contrastive Loss
Hard Negative Mining
Embedding separability diagnostics

这三件事会真正提高 Stage2 的输入质量。

用户 2026-07-08 15:15:00

在我s1代码的基础上, Stage1C + SupConLoss, 如何实现这个功能?

助手 2026-07-08 15:15:17

可以实现, 而且这次我建议直接接到你现有 Stage1 训练流程里, 而不是另写一套训练脚本: 模型继续正常输出 `logits`, 同时从 forward 里取 `embedding z`, 训练时用 `classification\_loss + supcon\_weight \* SupConLoss(z, y)`。我先确认你当前 Stage1C 的 forward 返回格式和 trainer 训练循环, 然后做最小侵入式修改。

助手 2026-07-08 15:15:34

好, 关键点确认了: 你的 `Stage1TimeAwareTransformer.forward()` 已经支持 `return\_embeddings=True`, 返回 `(logits, z, h)`, 所以 SupCon 不需要改模型结构, 只要训练时打开这个返回并把 `z` 喂给 contrastive loss。这个很干净。

助手 2026-07-08 15:15:51

接下来我要改两处核心代码: `losses.py` 加 `SupervisedContrastiveLoss`, `trainer.py` 在训练时按配置打开 `return\_embedding=True` 并把 `z` 加入 loss。默认配置不启用 SupCon, 所以不会影响你之前的 Stage1C 结果。

助手 2026-07-08 15:17:03

我先把 `s1` 的训练入口、loss 和模型输出路径对齐一下, 然后直接在现有 Stage1C 上加一个可开关的 SupConLoss。这样你可以保持原来的 Stage1C 配置不动, 只通过 YAML 或命令行切到 `Stage1C + SupConLoss`。

助手 2026-07-08 15:17:16

我看了一下现有结构: `model.forward(..., return\_embedding=True)` 已经能返回 `logits, z, h`, 所以 SupCon 可以直接作用在 Stage1C 的 embedding `z` 上, 不需要重写模型主体。接下来我会加一个独立的 `SupervisedContrastiveLoss`, 再让训练循环在开启配置时使用 `CE/Focal +  $\lambda$  \* SupCon`。

助手 2026-07-08 15:17:21

还有一个小点我也顺手处理: `flow\_token` 分支在 `return\_embedding=True` 时没有给 `h` 赋值, 导出 embedding 或 SupCon 解包第三个返回值时可能踩到这个坑。哪怕你当前 Stage1C 不是

`flow\_token`，这里修一下更稳。

助手 2026-07-08 15:17:36

`SupervisedContrastiveLoss` 已经作为独立 loss 加进去了：它只看 batch 内同标签样本，把同类 embedding 拉近、异类推远；如果某个 batch 没有正样本对，会自动跳过，不会产生 NaN。下一步把它接到训练 loop。

助手 2026-07-08 15:17:58

补丁因为注释里的编码字符匹配失败了，我改用更小的补丁分段接入，避免碰那些乱码注释。代码逻辑不受影响。

助手 2026-07-08 15:18:49

训练主流程现在会读取

`training.use\_supcon/supcon\_weight/supcon\_temperature/supcon\_warmup\_epochs`。我继续改单个 epoch 的 forward：开启 SupCon 时拿 `z` 计算额外 loss，关闭时完全保持原逻辑。

助手 2026-07-08 15:19:22

我再给 `run\_stage1.py` 加命令行开关，这样你不用复制一份配置文件也能直接跑对比实验；当然 YAML 里写同样字段也可以。

助手 2026-07-08 15:19:35

核心代码已经接好了。现在我做一次语法检查，再用一个小张量跑一下 SupCon 的前向，确认没有形状或数值问题。

助手 2026-07-08 15:19:49

本机这个 PowerShell 里 `python` 命令没挂到 PATH，而且当前目录看起来不是 Git 仓库根，所以我换成本机已安装的 Python 绝对路径做检查。

助手 2026-07-08 15:20:09

语法检查通过。再跑一个最小 SupCon smoke test，确认同标签/异标签计算路径能正常返回有限 loss。

助手 2026-07-08 15:20:38

SupCon smoke test 也通过了，返回的是正常有限值。最后我快速看一下改动位置和可能的调用点，确保没有漏掉 Optuna/embedding 导出这种旁路。

助手 2026-07-08 15:20:50

我再做一个数值稳定性的小优化：模型前向和分类 loss 仍用 AMP，SupCon 的相似度矩阵放在 AMP 外面按 float32 算，训练时更不容易因为温度较小出现极端值。

助手 2026-07-08 15:20:58

还是那段注释编码影响上下文匹配。我用更窄的匹配改同一段逻辑。

助手 2026-07-08 15:21:25

这个稳定性调整也完成了：分类部分仍然享受 AMP，SupCon 部分现在在外面算。再跑一次语法检查确认。

助手 2026-07-08 15:21:46

最终语法检查通过，改动集中在 4 个文件：loss、trainer、model 的一个小返回修正、以及 run_stage1 命令行参数。现在可以直接用现有 Stage1C 配置加 SupCon 参数跑新实验。

助手 2026-07-08 15:22:12

我顺手把 Optuna 训练路径也接上同样的开关。你主命令走的是普通 `train\_model`，但这样以后做 Stage1C 的 HPO 时不会出现“普通训练支持 SupCon、Optuna 不支持”的分裂。

助手 2026-07-08 15:23:21

已在你的 `s1` 代码里实现了 `Stage1C + SupConLoss`，并保持原有 Stage1C 主体不变。

****改动文件****

- [losses.py](<C:\Users\XiaoyanXiong\Desktop\NIDSTransformer\s1\stage1\losses.py:286>): 新增 `SupervisedContrastiveLoss`
- [trainer.py](<C:\Users\XiaoyanXiong\Desktop\NIDSTransformer\s1\stage1\trainer.py:63>): 训练时支持 `classification loss +  $\lambda$  \* SupConLoss`
- [run_stage1.py](<C:\Users\XiaoyanXiong\Desktop\NIDSTransformer\s1\run_stage1.py:57>): 新增命令行参数
- [model.py](<C:\Users\XiaoyanXiong\Desktop\NIDSTransformer\s1\stage1\model.py:990>): 修复 `flow\_token` 分支下 `return\_embedding=True` 时 `h` 未定义的潜在问题

核心训练目标现在是：

```
loss = cls_loss + current_supcon_weight * supcon_loss(z, y)
```

其中 `z` 是 Stage1 导出的 flow embedding。SupCon 会让同标签 flow 的 embedding 更接近，不同标签的 embedding 更远，所以它不是替代 Focal/CE，而是辅助 Stage1C 学出更可分的 embedding。

推荐先这样跑：

```
!python /content/drive/MyDrive/s1/run_stage1.py \
--packet_csv /content/drive/MyDrive/s1/data/xxx-stage1_packets.csv \
--flow_csv /content/drive/MyDrive/s1/data/xxx-stage1_flows.csv \
--config /content/drive/MyDrive/s1/0704/0704C_ar002_et12_20260511_001/stage1_config_C.yaml \
--out_dir /content/drive/MyDrive/s1/0708_stage1C_supcon_w005_t010 \
--export_embeddings \
--use_supcon \
--supcon_weight 0.05 \
--supcon_temperature 0.10 \
--supcon_warmup_epochs 5
```

也可以写进 YAML：

```
training:
  use_supcon: true
  supcon_weight: 0.05
  supcon_temperature: 0.10
  supcon_warmup_epochs: 5
```

建议实验顺序：

实验	supcon_weight	temperature	目的
baseline	0	-	原 Stage1C
mild	0.03	0.10	最稳, 不容易伤 precision
recommended	0.05	0.10	首选
stronger	0.10	0.10	看 embedding 是否进一步分开
softer	0.05	0.20	如果 recall/precision 波动大

注意一点很关键: SupCon 是 batch 内 pair loss, 所以 `batch\_size=128` 可以, `256` 更好。如果你启用 weighted sampler, 不建议正类比例直接 0.5; 可以先试 `sampler\_pos\_fraction: 0.25` 或 `0.30`, 否则可能提高 recall 但 precision 掉。

我已经做过验证:

- `py\_compile` 通过
- SupConLoss 小张量前向测试通过, loss 是有限值

跑完新的 Stage1 后, 需要用新的 embedding 重新跑 Stage2。这个方案真正要看的不是单独 Stage1 accuracy, 而是: `Stage1 PR\_AUC/F1\_class1` 是否提高, 以及 Stage2 用新 embedding 后 `P1/R1/F1\_class1` 是否同步提高。

用户 2026-07-08 16:33:14

Files mentioned by the user:

[INFO] Stage2 ----- seed: 130 [INFO] 随机种子已设置: 130 [INFO] Stage2 ----- d...:
C:\Users\XiaoyanXiong\codex/attachments/ccfe7dfc-4530-49fc-9b8f-6d7c3875cb59/pasted-text.txt

My request for Codex:

```
!python /content/drive/MyDrive/s2/run_stage2.py \  
--stage1_dir /content/drive/MyDrive/s1/0704/0704C_ar002_et12_20260511_001 \  
--config /content/drive/MyDrive/s2/0708ar002_et12_20260511_001/source_destination_attention/stage2_  
_config_used.yaml \  
--out_dir /content/drive/MyDrive/s2/0708ar002_et12_20260511_001/source_destination_attention/ws128_  
_src02_dst01_precision_at_recall80 \  
--context_method source_destination \  
--context_policy online \  
--window_size 128 \  
--model_type source_destination_attention \  
--source_context_scale 0.2 \  
--destination_context_scale 0.1 \  
--gate_bias_init -2.0 \  
--source_gate_bias_init -2.0 \  
--destination_gate_bias_init -3.0 \  
--use_context_length_feature \  
--threshold_metric precision_at_recall \  
--threshold_recall_floor 0.80 \  
--seed 130 \  
--epochs 500 \  
--batch_size 128
```

运行结果: Stage2 run summary

=====

best_epoch: 12
metric_for_best: pr_auc
best_score: 0.913327537303977
context_method: source_destination
context_policy: online
window_size: 128

Final metrics:

train: loss=0.014376, acc=0.989226, macro_precision=0.954960, macro_recall=0.893303,
macro_f1=0.921697, weighted_precision=0.988831, weighted_recall=0.989226, weighted_f1=0.988834,
auc=0.9961891208131032, pr_auc=0.9425145488793735, precision_label1=0.9182751540041068,
recall_label1=0.7894086496028243, f1_label1=0.8489795918367347, cm=[[283203, 796], [2386, 8944]]
val: loss=0.022563, acc=0.985921, macro_precision=0.924806, macro_recall=0.897568,
macro_f1=0.910689, weighted_precision=0.985543, weighted_recall=0.985921, weighted_f1=0.985687,
auc=0.9922250220543702, pr_auc=0.9133370886017915, precision_label1=0.8584229390681004,
recall_label1=0.8010033444816054, f1_label1=0.828719723183391, cm=[[40158, 237], [357, 1437]]
test: loss=0.027798, acc=0.983397, macro_precision=0.895180, macro_recall=0.897864,
macro_f1=0.896517, weighted_precision=0.983454, weighted_recall=0.983397, weighted_f1=0.983425,
auc=0.991240870514132, pr_auc=0.8876581028960786, precision_label1=0.7988716502115656,
recall_label1=0.8045454545454546, f1_label1=0.8016985138004247, cm=[[80147, 713], [688, 2832]]

结合我的代码和配置，详细解析结果。还有什么优化空间吗？能将F1_class1, R1_class1,P1_class1都提高到0.85吗？详细解析

助手 2026-07-08 16:33:29

我先对你这次的结果做数值拆解，同时看一下本地`s2`里 source/destination attention、threshold search、gate 参数实际是怎么接入的。这样判断“还能不能到0.85”会更靠近你的代码真实行为，而不是只看表面指标。

助手 2026-07-08 16:35:40

这次结果的核心结论：`source\_destination\_attention + precision\_at\_recall>=0.80`已经把模型稳定推到`R1≈0.80`，但没有真正突破 Stage1C 的 class1 可分性上限。它比 Stage1C 的排序能力更好，`PR\_AUC`明显更高，但最终`F1\_class1`只小幅提升。

****1. 和 Stage1C 对比****

model	P1	R1	F1_class1	PR_AUC
Stage1C	0.8069	0.7920	0.7994	0.8763
当前 Stage2	0.7989	0.8045	0.8017	0.8877
差值	-0.0081	+0.0125	+0.0023	+0.0114

所以：可以说 Stage2 提高了 ranking/separability，也提高了 recall；但不能说最终分类 F1 有明显提升，因为`+0.0023`很小，单 seed 下可能接近波动。

****2. 这次配置做了什么****

你的模型在

[target_query.py](<C:\Users\XiaoyanXiong\Desktop\NIDSTransformer\s2\stage2\target_query.py:601>)
里是双路 residual attention：

```
logits = base_logits
        + source_context_scale * source_gate * source_delta_logits
        + destination_context_scale * destination_gate * destination_delta_logits
```

这次参数很保守：

参数	值	含义
`source_context_scale`	0.2	source 分支最多小幅修正 base logits
`destination_context_scale`	0.1	destination 更保守
`source_gate_bias_init`	-2.0	初始 gate≈0.119
`destination_gate_bias_init`	-3.0	初始 gate≈0.047
`sampler_pos_fraction`	0.08	训练采样中正类比例较低，偏 precision 稳定
`alpha`	[1.0, 1.05]	class1 只轻微加权
`threshold_metric`	precision_at_recall	在 recall≥0.80 的阈值中选 precision 最高

这个设计方向是对的：你没有让 context 直接覆盖 Stage1 embedding，而是让 context 做小的 gated logit correction。

3. Confusion Matrix 说明

当前 test:

```
TN=80147, FP=713
FN=688, TP=2832
```

对应：

```
P1 = 2832 / (2832 + 713) = 0.7989
R1 = 2832 / (2832 + 688) = 0.8045
F1 = 0.8017
```

对比 Stage1C:

```
Stage1C: TP=2788, FP=667, FN=732
: TP=2832, FP=713, FN=688
```

也就是说 Stage2 多抓到了 `44` 个 class1，但同时多误报了 `46` 个 class0。因此 F1 只涨了一点点。

4. 为什么 val precision 高，但 test precision 掉了

val:

```
P1=0.8584, R1=0.8010, FP=237
```

test:

```
P1=0.7989, R1=0.8045, FP=713
```

recall 几乎没掉，说明 class1 检出能力是稳定的。precision 掉，主要是 test 里的 hard negative 更多。

负类误报率：

```
train FPR = 796 / 283999 = 0.28%
val FPR = 237 / 40395 = 0.59%
test FPR = 713 / 80860 = 0.88%
```

这非常关键：问题不是阈值没选好，而是 test negative 的分布更难，更多 benign flow 被打到 class1 高分区间。

你的 threshold diagnosis 也证明了这一点：

```
TEST AT VAL THRESHOLD F1 = 0.8017
TEST ORACLE F1          = 0.8021
gap = 0.0004
```

阈值几乎没有优化空间了。

****5. 能不能 P1/R1/F1 都到 0.85? ****

以当前模型，基本不能。

如果要 $R1 \geq 0.85$ ，test 正类总数是 3520，需要：

$$TP \geq 0.85 * 3520 = 2992$$

当前 $TP=2832$ ，所以至少还要多抓 160 个正类。

如果同时要 $P1 \geq 0.85$ ，在 $TP=2992$ 时：

$$FP \leq 2992 * (1 - 0.85) / 0.85 \approx 528$$

当前 $FP=713$ ，所以还要至少减少 185 个误报。

也就是说，不是简单调阈值，而是要同时做到：

TP +160
FP -185

这需要明显更好的 score 排序。当前 test oracle 都只有 $F1 \approx 0.802$ ，所以 Stage2 现有 embedding + threshold 已经接近上限。

****6. 还有什么优化空间****

短期 Stage2 还能试，但预期是小幅提升，不是直接到 0.85。

优先级最高的 Stage2 实验：

```
--metric_for_best f1_label1 \  
--threshold_metric f1_at_recall \  
--threshold_recall_floor 0.80 \  
--threshold_steps 1001
```

原因：你现在 $\text{metric_for_best}=\text{pr_auc}$ ，但最终目标是 F1/P/R。日志里 epoch 13 的 val F1 比 best epoch 12 更高，但因为 PR_AUC 低一点，没有被选中。

第二组，控制 FP：

```
--threshold_metric recall_at_precision \  
--threshold_precision_floor 0.80 \  
--threshold_steps 1001
```

第三组，减少 destination 噪声：

```
--source_context_scale 0.25 \  
--destination_context_scale 0.05 \  
--source_gate_bias_init -2.0 \  
--destination_gate_bias_init -3.5
```

因为你的统计里 class1 的 $\text{destination_history_len}$ 明显比 class0 短，destination 分支可能更容易引入不稳定 FP。

第四组，训练采样小网格：

```
| 目标 | sampler_pos_fraction | alpha |  
|---|---|---|  
| 更稳 precision | 0.06 | [1.0, 1.03] |  
| 当前平衡 | 0.08 | [1.0, 1.05] |  
| 尝试 recall | 0.10 | [1.0, 1.08] |
```

不要一下子把正类采样提太高，否则 recall 会涨，但 precision 大概率继续掉。

****7. 真正要冲 0.85，该看 Stage1****

当前 Stage2 已经说明：context 有帮助，但不够大。要冲 $P1/R1/F1 \geq 0.85$ ，重点应该放在 Stage1

embedding 可分性上:

- `Stage1C + SupConLoss`
- hard negative mining
- class1/class0 margin loss
- embedding center separation
- Stage1 score + Stage2 score ensemble
- 专门分析 test FP: 看这 713 个 FP 在 Stage1 里是不是本来就高分

我更建议下一步先跑你刚加的 `Stage1C + SupConLoss`，重新导出 embedding，再跑这个 `source\_destination\_attention`。如果 Stage1 PR_AUC 能从 `0.876` 提到 `0.90+`，Stage2 才有现实机会把 class1 F1 推到 `0.82~0.84`；要稳定到 `0.85`，Stage1 的 embedding 分离必须明显改善。

用户 2026-07-08 16:39:09

该如何修正代码，能让precision也想recall一样稳定，不掉？详细解析

助手 2026-07-08 16:39:27

我先把 `s2` 里 loss、threshold 选择、训练保存 best 的路径重新看一遍。你这个问题本质是 “test 里的 hard negative 被打高分”，所以代码层面要同时动三处：训练目标、best epoch 选择、阈值选择。

助手 2026-07-08 16:39:41

现有代码里已经有 `precision\_at\_recall`，但它只管 “阈值选择”；模型训练本身还是普通 focal loss，best checkpoint 还是按 `pr\_auc` 存。这会导致模型排序很好但 FP 控制不够稳。我准备加一个更偏 precision 的训练项：对负类样本中高于 margin 的 class1 概率额外惩罚。

助手 2026-07-08 16:40:13

我会加一个 “precision 稳定器” 而不是只调阈值：对负类样本，如果模型给 class1 的概率超过一个 margin，就额外罚；同时可以对正类低概率加一个轻量保护，避免 recall 被一起压下去。这个改动不碰你的模型结构，属于训练目标层面的保守修正。

助手 2026-07-08 16:40:21

我会把参数做成可选的，默认全部为 0，不会影响你已有实验。开启时才生效：`false\_positive\_penalty\_weight > 0` 会压低负类的高 class1 概率；`positive\_margin\_penalty\_weight` 可以保护正类概率，防止 recall 被压坏。

助手 2026-07-08 16:41:12

接着我加一个更保守的阈值目标：`precision\_lcb\_at\_recall`。它不是最大化验证集表面 precision，而是最大化 precision 的 Wilson lower confidence bound。直觉上，它会避开那些验证集上看起来漂亮、但样本支撑不够稳的阈值。

助手 2026-07-08 16:42:01

现在训练 loss 和阈值目标都支持新参数了。我继续把这些命令行参数写入 config override，确保 Colab 命令行可以直接打开，不用手改 YAML。

助手 2026-07-08 16:42:14

`run\_stage2.py` 已经可以直接用这些新参数。我也把同样改动同步到 `run\_stage2\_tuned.py`，避免以后你切入口时参数不一致。

助手 2026-07-08 16:42:48

代码改动完成，我跑一次语法检查和一个最小 loss 前向测试，确认新 loss 能正常反传路径、阈值函数参数也没破坏原接口。

助手 2026-07-08 16:42:57

语法检查通过。再跑一个最小张量测试，确认新 loss 包装器返回有限值，并且新增的 `precision\_lcb\_at\_recall` 阈值搜索能正常选阈值。

助手 2026-07-08 16:43:28

最小测试也通过了。现在我整理一下具体怎么用、为什么它能让 precision 更稳，以及哪些参数该先试。

助手 2026-07-08 16:43:33

改动点我已经确认在 4 个文件里，默认关闭，不影响旧实验。下面我会给你一个推荐命令和参数网格，重点是让 precision 不再靠阈值硬撑，而是在训练时降低高分 FP。

助手 2026-07-08 16:43:56

可以修，但要先明确一点：`precision` 不像 `recall` 那样稳定，主要不是阈值问题，而是 test 里的 hard negative 更多。你上一轮已经证明 threshold transfer gap 只有 `0.0004`，说明“换阈值”几乎救不了 precision。要让 precision 稳，需要让模型训练时学会：`不要把负类样本推到 class1 高概率区域`。

我已经给你的 `s2` 加了两类代码修正，默认关闭，不影响旧实验。

****已修改****

- [losses.py](<C:\Users\XiaoyanXiong\Desktop\NIDSTransformer\s2\stage2\losses.py:116>): 新增 `PrecisionRegularizedLoss`
- [trainer.py](<C:\Users\XiaoyanXiong\Desktop\NIDSTransformer\s2\stage2\trainer.py:19>): 把原本 focal/CE 包装成 precision-aware loss
- [trainer.py](<C:\Users\XiaoyanXiong\Desktop\NIDSTransformer\s2\stage2\trainer.py:360>): 新增 `precision\_lcb\_at\_recall` 阈值目标
- `run\_stage2.py` / `run\_stage2\_tuned.py`: 新增命令行参数

核心新增 loss 是：

```
loss = base_loss
      + fp_weight * mean(relu(P(class1|negative) - fp_margin)^2)
      + pos_weight * mean(relu(pos_margin - P(class1|positive))^2)
```

意思是：

- 负类如果 class1 概率太高，就罚，减少 FP，提高 precision 稳定性
- 正类如果 class1 概率太低，轻微罚，保护 recall 不被压坏

****推荐第一组命令****

先用温和参数，不要一上来压太狠：


```
!python /content/drive/MyDrive/s2/run_stage2.py \
--stage1_dir /content/drive/MyDrive/s1/0704/0704C_ar002_et12_20260511_001 \
--config /content/drive/MyDrive/s2/0708ar002_et12_20260511_001/source_destination_attention/
stage2_config_used.yaml \
--out_dir /content/drive/MyDrive/s2/0708ar002_et12_20260511_001/source_destination_attention/
ws128_precision_regularized_v1 \
--context_method source_destination \
--context_policy online \
--window_size 128 \
--model_type source_destination_attention \
--source_context_scale 0.2 \
--destination_context_scale 0.1 \
--gate_bias_init -2.0 \
--source_gate_bias_init -2.0 \
--destination_gate_bias_init -3.0 \
--use_context_length_feature \
--metric_for_best precision_label1 \
--threshold_metric precision_lcb_at_recall \
--threshold_recall_floor 0.80 \
--threshold_precision_lcb_z 1.0 \
--false_positive_penalty_weight 0.08 \
--false_positive_penalty_margin 0.55 \
--false_positive_topk_ratio 0.5 \
--positive_margin_penalty_weight 0.02 \
--positive_margin 0.45 \
--sampler_pos_fraction 0.06 \
--negative_alpha 1.0 \
--positive_alpha 1.03 \
--seed 130 \
--epochs 500 \
--batch_size 128
```

****为什么这些参数这样设****

| 参数 | 建议 | 作用 |

|---|---|---|

| `false\_positive\_penalty\_weight` | `0.08` | 压高分负类，直接减少 FP |

| `false\_positive\_penalty\_margin` | `0.55` | 只罚 class1 概率超过 0.55 的负类 |

| `false\_positive\_topk\_ratio` | `0.5` | 只盯 batch 中最危险的一半负类 |

| `positive\_margin\_penalty\_weight` | `0.02` | 轻微保护正类，防 recall 掉 |

| `positive\_margin` | `0.45` | 正类概率低于 0.45 才罚 |

| `sampler\_pos\_fraction` | `0.06` | 比原来 0.08 更偏 precision |

| `positive\_alpha` | `1.03` | 比原来 1.05 更少推高 class1 |

| `metric\_for\_best` | `precision\_label1` | checkpoint 也按 precision-oriented val 指标保存 |

| `threshold\_metric` | `precision\_lcb\_at\_recall` | 选更稳的 precision，而不是表面最高 precision |

****调参顺序****

如果 recall 掉到 `<0.80`:

```
--positive_margin_penalty_weight 0.03
--sampler_pos_fraction 0.08
```

如果 precision 还是不稳:

```
--false_positive_penalty_weight 0.12
--false_positive_penalty_margin 0.50
--threshold_precision_lcb_z 1.28
```

如果 precision 上升但 F1 下降:

```
--false_positive_penalty_weight 0.05
--false_positive_penalty_margin 0.60
```

****最重要的判断标准****

不要只看 test P1。你要看这几个一起:

```
val P1 / test P1 gap
val FPR / test FPR gap
test FP      713
test TP      2800
```

你的目标不是单纯提高 precision，而是把：

```
FP: 713 -> 600
TP: 2832      2800+
```

这样 P1 会更稳，F1 才可能继续涨。当前代码修正的方向正是减少高分 false positive，同时尽量不牺牲 recall。

用户 2026-07-09 09:41:05

precision_label1从train:0.8745468422056859到val: 0.839766081871345到test: 0.7998286693318104在降低，但是recall_label1从train:0.8090909090909091到val: 0.8004459308807135到test: 0.7957386363636364却没有特别大的降低，这种情况该如何处理？详细解析recall别precision低那么多又是为什么？详细解析

助手 2026-07-09 09:41:22

我先按你给的 P/R 把它翻译成 TP/FP/FN 的含义，再解释为什么 recall 看起来稳定、precision 却一路掉。这个现象其实很典型：模型对正类的覆盖能力没怎么变，但 test 负类里更容易被误报的样本变多了。

助手 2026-07-09 09:42:22

这个现象的核心是：

> **模型对正类的识别能力基本稳定，所以 recall 没怎么掉；但 test 负类里被误判成 1 的 hard negative 更多，所以 precision 明显掉。**

按你给的数值和之前固定 split 数量反推，大概是：

```
| split | P1 | R1 | TP | FP | FN | FPR |
|---|---|---|---|---|---|---|
| train | 0.8745 | 0.8091 | 9167 | 1315 | 2163 | 0.46% |
| val   | 0.8398 | 0.8004 | 1436 | 274 | 358 | 0.68% |
| test  | 0.7998 | 0.7957 | 2801 | 701 | 719 | 0.87% |
```

真正变坏的是 **FPR**：

```
train FPR = 0.46%
val   FPR = 0.68%
test  FPR = 0.87%
```

而 recall 对应的是正类内部：

```
train miss rate = 19.09%
val   miss rate = 19.96%
test  miss rate = 20.43%
```

所以 recall 稳定，是因为正类 TP/FN 结构稳定；precision 掉，是因为负类 FP 变多。

为什么 precision 更容易掉

你的数据极度不平衡，test 里：

```
positive = 3520
negative = 80860
```

负类数量大约是正类的 `23 倍`。所以即使负类误报率只从 `0.46%` 涨到 `0.87%`，FP 数量也会明显增加。

precision 公式是：

```
precision = TP / (TP + FP)
```

它直接受 FP 影响。recall 公式是：

```
recall = TP / (TP + FN)
```

它完全不看 FP。所以当 test 里 hard negative 增多时，会出现：

```
recall  
precision
```

这不是矛盾，而是典型的负类分布 shift。

****为什么 recall 有时比 precision 低****

如果你说的是 train 上 `P1=0.8745 >

R1=0.8091`，原因是当前阈值和训练目标偏保守：模型宁愿少报一些正类，也不愿多报负类。

也就是：

```
-> FP    -> precision  
->          -> recall
```

如果降低阈值，recall 会升，但 FP 会增加，precision 会掉。你现在用 `threshold\_recall\_floor=0.80`，等于把 recall 锚定在 0.80 左右，所以 recall 自然稳定在这个区域。

****该如何处理****

不要只调 threshold。你上一轮的 threshold oracle gap

很小，说明阈值已经不是主要问题。要处理的是：****训练时减少高分 FP****。

我建议你现在用这组方向：

```
--metric_for_best precision_label1_lcb \  
--threshold_metric precision_lcb_at_recall \  
--threshold_recall_floor 0.80 \  
--threshold_precision_lcb_z 1.0 \  
--false_positive_penalty_weight 0.08 \  
--false_positive_penalty_margin 0.55 \  
--false_positive_topk_ratio 0.5 \  
--positive_margin_penalty_weight 0.02 \  
--positive_margin 0.45 \  
--sampler_pos_fraction 0.06 \  
--negative_alpha 1.0 \  
--positive_alpha 1.03
```

含义是：

- `false\_positive\_penalty\_weight`：专门惩罚负类被打成高 class1 概率
- `false\_positive\_penalty\_margin=0.55`：负类 class1 概率超过 0.55 才罚
- `false\_positive\_topk\_ratio=0.5`：只盯每个 batch 中最危险的一半负类
- `positive\_margin\_penalty\_weight=0.02`：轻微保护正类，避免 recall 被压坏
- `sampler\_pos\_fraction=0.06`：比 0.08 更偏 precision
- `positive\_alpha=1.03`：不要过度推高 class1

****调参判断****

如果 test precision 还是掉：

```
--false_positive_penalty_weight 0.12  
--false_positive_penalty_margin 0.50  
--threshold_precision_lcb_z 1.28
```

如果 recall 掉到 0.78 以下：

```
--positive_margin_penalty_weight 0.03
```

```
--sampler_pos_fraction 0.08
```

如果 precision 上升但 F1 下降:

```
--false_positive_penalty_weight 0.05  
--false_positive_penalty_margin 0.60
```

****最关键目标****

你现在 test 大概是:

```
TP=2801, FP=701
```

要让 precision 稳起来, 短期目标应该是:

```
TP    >= 2800  
FP    701    600
```

这样 precision 会从 `0.80` 往 `0.82~0.83` 走, 同时 recall 不会明显掉。要到 `0.85`, FP 需要降到大约 `500` 以下, 这通常需要 Stage1 embedding 本身更可分, 不只是 Stage2 调参。

用户 2026-07-09 10:49:35

Files mentioned by the user:

```
## [INFO] Stage2 ----- seed: 130 [INFO] 随机种子已设置: 130 [INFO] Stage2 ----- d...:  
C:\Users\XiaoyanXiong\.codex/attachments/c87a2268-6b98-492f-9393-03ba733bb853/pasted-text.txt
```

My request for Codex:

```
!python /content/drive/MyDrive/s2/run_stage2.py \  
--stage1_dir /content/drive/MyDrive/s1/0704/0704C_ar002_et12_20260511_001 \  
--config /content/drive/MyDrive/s2/0709ar002_et12_20260511_001/source_destination_attention/stage2  
_config_used.yaml \  
--out_dir /content/drive/MyDrive/s2/0709ar002_et12_20260511_001/source_destination_attention/ws128  
_precision_regularized_v1_0704C \  
--context_method source_destination \  
--context_policy online \  
--window_size 128 \  
--model_type source_destination_attention \  
--source_context_scale 0.2 \  
--destination_context_scale 0.1 \  
--gate_bias_init -2.0 \  
--source_gate_bias_init -2.0 \  
--destination_gate_bias_init -3.0 \  
--use_context_length_feature \  
--metric_for_best precision_label1 \  
--threshold_metric precision_lcb_at_recall \  
--threshold_recall_floor 0.80 \  
--threshold_precision_lcb_z 1.0 \  
--false_positive_penalty_weight 0.08 \  
--false_positive_penalty_margin 0.55 \  
--false_positive_topk_ratio 0.5 \  
--positive_margin_penalty_weight 0.02 \  
--positive_margin 0.45 \  
--sampler_pos_fraction 0.06 \  
--negative_alpha 1.0 \  

```

```
--positive_alpha 1.03 \  
--seed 130 \  
--epochs 500 \  
--batch_size 128
```

运行结果: Stage2 run summary

=====

best_epoch: 16
metric_for_best: precision_label1
best_score: 0.8570581257413997
context_method: source_destination
context_policy: online
window_size: 128

Final metrics:

train: loss=0.013799, acc=0.989405, macro_precision=0.953646, macro_recall=0.897337,
macro_f1=0.923466, weighted_precision=0.989027, weighted_recall=0.989405, weighted_f1=0.989055,
auc=0.9963594769752726, pr_auc=0.9439676900032123, precision_label1=0.9153246227083967,
recall_label1=0.7976169461606355, f1_label1=0.8524265434136679, cm=[[283163, 836], [2293, 9037]]
val: loss=0.022246, acc=0.986015, macro_precision=0.924221, macro_recall=0.899748,
macro_f1=0.911583, weighted_precision=0.985671, weighted_recall=0.986015, weighted_f1=0.985808,
auc=0.9915387733975376, pr_auc=0.9105891746160066, precision_label1=0.8570581257413997,
recall_label1=0.8054626532887402, f1_label1=0.8304597701149425, cm=[[40154, 241], [349, 1445]]
test: loss=0.027157, acc=0.983195, macro_precision=0.890422, macro_recall=0.902922,
macro_f1=0.896563, weighted_precision=0.983479, weighted_recall=0.983195, weighted_f1=0.983327,
auc=0.9908888925584063, pr_auc=0.886338528244378, precision_label1=0.7888949972512369,
recall_label1=0.8153409090909091, f1_label1=0.8018999720592345, cm=[[80092, 768], [650, 2870]]

详细解析结果, 还有提升的空间吗?

助手 2026-07-09 10:49:49

我先看一下完整日志里的阈值选择和 oracle diagnosis. 你这次表面上用了 precision regularizer, 但结果里 precision 没升、recall 反而升了, 关键通常藏在 “选出来的 threshold” 和 “test oracle” 里。

助手 2026-07-09 10:51:07

这次结果的结论很清楚: **precision regularized v1 没有真正把 precision 稳住, 它主要把模型推向了更高 recall**。F1 几乎没变, 但 P/R 的平衡方向变了。

****1. 当前结果拆解****

这次 test:

```
P1 = 0.7889  
R1 = 0.8153  
F1 = 0.8019  
CM = [[80092, 768],  
      [650, 2870]]
```

也就是:

```
TP = 2870  
FP = 768  
FN = 650
```

```
TN = 80092
```

和你上一版 `precision\_at\_recall80` 比:

model	P1	R1	F1	TP	FP	FN
--- ---:	---:	---:	---:	---:	---:	---
上一版	0.7989	0.8045	0.8017	2832	713	688
当前 v1	0.7889	0.8153	0.8019	2870	768	650
变化	-0.0100	+0.0108	+0.0002	+38	+55	-38

所以当前 v1 本质是:

```
      38
     55
F1
precision
recall
```

这说明: **这组 regularizer 参数没有成功压住 FP, 反而让 selected threshold 更低, recall 上去了, precision 下来了.**

****2. 为什么会这样****

你这次 val 选择出来的 threshold 是:

```
threshold = 0.613838
```

上一版大约是:

```
threshold = 0.658384
```

阈值明显降低了。阈值降低的直接结果就是:

```
      1
TP
FP
recall
precision
```

这正是你现在看到的结果。

更关键的是 test oracle:

```
TEST AT VAL THRESHOLD:
P1=0.7891, R1=0.8153, F1=0.8020, threshold=0.6138
```

```
TEST ORACLE:
P1=0.8028, R1=0.8017, F1=0.8023, threshold=0.6386
```

这说明当前模型其实在 test 上存在一个更偏 precision 的阈值 `0.6386`, 但验证集选择机制选了更低的 `0.6138`。不过 F1 差距只有 `0.00026`, 所以不是大问题; 真正问题还是 score ranking 没明显变好。

****3. regularizer 为什么没把 precision 提上来****

你用了:

```
false_positive_penalty_weight = 0.08
false_positive_penalty_margin = 0.55
false_positive_topk_ratio = 0.5
positive_margin_penalty_weight = 0.02
positive_margin = 0.45
sampler_pos_fraction = 0.06
positive_alpha = 1.03
```

这里有两个互相拉扯的力:

```
false_positive_penalty -> FP      precision
positive_margin_penalty ->         recall
```

```
threshold_recall_floor=0.80 -> recall
```

实际结果说明：`positive\_margin\_penalty + recall floor` 的效果更强，模型/阈值最后偏向于 recall，而不是 precision。

所以这次不是代码坏了，而是参数组合偏 soft，且阈值选择把收益转成了 recall。

****4. 一个重要异常：你的 Colab 代码可能不是最新****

你的命令里有：

```
--source_gate_bias_init -2.0
--destination_gate_bias_init -3.0
```

但日志里模型初始化只打印了：

```
gate_bias_init=-2.0
source_context_scale=0.2
destination_context_scale=0.1
```

没有打印：

```
source_gate_bias_init
destination_gate_bias_init
```

我本地最新版代码会打印这两个参数。这个现象说明：****你 Colab 里的 `target\_query.py` 可能还不是最新版，或者模型没有真正使用 separate gate bias. ****

这很重要。因为如果 `destination\_gate\_bias\_init=-3.0` 没生效，那么 destination 分支可能比你预期更活跃，会增加 FP。

建议你先确认 Colab 里 `Stage2SourceDestinationAttention.\_init\_` 是否真的有：

```
source_gate_bias_init = float(model_cfg.get("source_gate_bias_init", gate_bias_init))
destination_gate_bias_init = float(model_cfg.get("destination_gate_bias_init", gate_bias_init))
nn.init.constant_(self.source_gate[-2].bias, source_gate_bias_init)
nn.init.constant_(self.destination_gate[-2].bias, destination_gate_bias_init)
```

如果没有，先同步代码。这个可能比继续调参更重要。

****5. 下一步怎么调，目标是 precision 不掉****

我建议你跑 v2，把正类保护先关掉，让 regularizer 更专注 FP：

```
--metric_for_best precision_lcb \
--threshold_metric precision_lcb_at_recall \
--threshold_recall_floor 0.80 \
--threshold_precision_lcb_z 1.28 \
--false_positive_penalty_weight 0.12 \
--false_positive_penalty_margin 0.50 \
--false_positive_topk_ratio 0.3 \
--positive_margin_penalty_weight 0.0 \
--sampler_pos_fraction 0.05 \
--negative_alpha 1.0 \
--positive_alpha 1.0
```

预期效果：

```
FP
P1
R1    0.815    0.79~0.805
F1      precision
```

如果 recall 掉到 ` $<0.79$ `，再加一点点保护：

```
--positive_margin_penalty_weight 0.01
--positive_margin 0.45
```

不要一开始用 ` $0.02$ `，这次结果已经说明它偏 recall。

****6. 阈值选择也要改****

当前 `precision\_lcb\_at\_recall` 选了较低阈值。可以试两个替代：

方案 A，更偏 F1 稳定：

```
--threshold_metric f0.5_at_recall
--threshold_recall_floor 0.80
--threshold_fbeta_beta 0.5
```

方案 B，更偏 precision 稳定：

```
--threshold_metric precision_at_recall
--threshold_recall_floor 0.80
```

如果你发现 val threshold 仍然偏低，可以把 recall floor 微降一点：

```
--threshold_recall_floor 0.795
```

这会允许模型选择更高 threshold，通常 P1 会提升，R1 小幅下降。

****7. destination 分支建议更保守****

你当前 source/destination 统计里，destination 对 class1 的历史更短、更不稳定。我建议下一组同时试：

```
--source_context_scale 0.2
--destination_context_scale 0.05
--source_gate_bias_init -2.0
--destination_gate_bias_init -4.0
```

如果 Colab 代码已经支持 separate gate，这会进一步减少 destination 分支制造 FP。

****8. 还有提升空间吗****

有，但短期空间不大。

当前几个结果都在：

```
F1_class1 ≈ 0.801 ~ 0.802
P1 ≈ 0.789 ~ 0.803
R1 ≈ 0.802 ~ 0.815
PR_AUC ≈ 0.886 ~ 0.888
```

这说明 Stage2 已经接近当前 Stage1 embedding 的上限。后续 Stage2 调参可能把 precision 拉回 `0.81~0.82`，但想同时做到：

```
P1 >= 0.85
R1 >= 0.85
F1 >= 0.85
```

只靠这套 Stage2 很难。真正要突破，还是要让 Stage1 embedding 更可分，比如你前面已经加的 `Stage1C + SupConLoss`，重新导出 embedding 后再跑 Stage2。当前 v1 的意义是：它验证了你可以移动 P/R 平衡，但没有改善根本排序。